

RakshaSafe –Women Safety and Disaster Emergency Response System

A Project Report

Submitted in partial fulfillment of the Requirements for the award of Degree of
BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY)

BY

(Vivek Lorikprasad sahani)

(Roll No.42)

Under the esteemed guidance of

Prof. Sourabh Suryawanshi



DEPARTEMENT OF INFORMATION TECHNOLOGY

D.T.S.S. COLLEGE OF COMMERCE (AUTONOMOUS)

Kurar Village, Malad (E), Mumbai-400097

Academic Year – 2026-27

PROFORMA FOR THE APPROVAL OF PROJECT PROPOSAL

PRN No .: MU0341120240045529

Roll No:- 42

1. Name of the Student: Vivek Lorikprasad sahani

2. Title of the Project: RakshaSafe –Safety and Disaster Emergency Response System

3. Name of the Guide: Prof. Sourabh Suryawanshi

4. Teaching experience of the guide:_____

5. Is this your first submission?

Yes

☐

No

☐

Signature of the Student

Signature of the Guide

Date:_____

Date:_____

Signature of the Coordinator

Date:_____



**Dhirajlal Talakchand Sankalchand Shah College of Commerce &
P. D. Turakhia Junior College of Commerce & Science**

Kurar, Malad (East), Mumbai – 400 097. (M.S.) Tal. Borivali, Dist.: Mumbai.

Tel. Off. : M.: 8591377595 / 8591376800/8433888534

E-mail: principal@sanskarsarjan.org / admin@sanskarsarjan.org / library@sanskarsarjan.org

Re-accredited by NAAC with “A” Grade with 3.03 CGPA Permanently Affiliated to
University of Mumbai

Best College Award by University of Mumbai

[AUTONOMOUS]

DEPARTMENT OF INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that the project entitled **“RakshaSafe –Women Safety and Disaster Emergency Response System”** is the bonafide work of Mr. Vivek Lorikprasad sahani, bearing Roll Number: 42, submitted in partial fulfillment of the requirements for the award of the degree of BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY).

The work carried out by the student is found to be satisfactory and genuine to the best of our knowledge.

Internal Guide

Head of the Department

External Examiner

Date: _____

Seal of the College:

DECLARATION

I hereby declare that the project entitled “**RakshaSafe –Women Safety and Disaster Emergency Response System**”done at T.Y.B.Sc. I.T., D.T.S.S. College of Commerce (Autonomous), has not been in any case duplicated to submit to any other university for the award of any degree. To the best of my knowledge other than me, no one has submitted to any other university.

The project is done in partial fulfillment of the requirements for the award of the degree of BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY) to be submitted as Semester **Vth** project as part of our curriculum.

Place: _____

Date: _____

Signature of the Students:

Name: Vivek Lorikprasad sahani

Sign of student: _____

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, **Prof. Sourabh Suryawanshi**, for his valuable guidance, continuous support, and encouragement throughout the development of our project, **RakshaSafe –Women Safety and Disaster Emergency Response System**.

We also extend our sincere thanks to the **Head of Department and all the faculty members of the Department of Information Technology, D.T.S.S. College of Commerce (Autonomous)**, for their valuable support and for providing the necessary facilities and resources for the successful completion of this project.

We are also grateful to our **team members** for their cooperation, valuable ideas, and dedicated efforts throughout the development, testing, documentation, and successful completion of the project.

Finally, we would like to thank everyone who directly or indirectly supported us during the completion of this project.

Vivek Lorikprasad Sahani

(Roll Number.42)

TABLE OF CONTENTS

Abstract of the project	
Chapter 1: Introduction	
1.1 Background	
1.2 Objectives	
1.3 Purpose, Scope and Applicability	
1.4 Achievements	
1.5 Organization of Report	
Chapter 2: Survey of Technologies	
Chapter 3: Requirements and Analysis	
3.1 Problem Definition	
3.2 Requirements Specification	
3.3 Software and Hardware Requirements	
3.4 Planning and Scheduling	
3.5 Conceptual Models	
Chapter 4: System Design	
4.1 Modules Division	
4.2 System Interface	
Chapter 5: Implementation and Testing (Output Screens)	
Chapter 6: Test Cases and Test Reports	
Chapter 7: Conclusion and Future Enhancement	
Chapter 8: References	

List of Tables

1. Software Requirements (Table 3.3.1)
2. Hardware Requirements (Table 3.3.2)
3. Data Dictionary — (3.5.2)
4. Module Testing — Login to Administration (6.1)
5. Security Testing (6.2)
6. Test Report Summary (6.3)

List of Figures

1. Gantt Chart (3.4.1)
2. PERT Chart (3.4.2)
3. Conceptual Models
 - 3.1 Entity Relationship Diagram (3.5.1)
 - 3.2 Data Flow Diagram Level-0 and Level-1 (3.5.3)
 - 3.3 Class Diagram (3.5.4)
 - 3.4 Activity Diagram (3.5.5)
 - 3.5 Use Case Diagram (3.5.6)
 - 3.6 System Flowchart (3.5.7)
4. System Interface Flowcharts — Registration to Administration (4.2.1–4.2.12)
5. Output Screens — Login to Admin Dashboard (5.1) Modules 1–9

ABSTRACT

RakshaSafe is a web-based **Women Safety and Disaster Emergency Response System** developed to provide a single platform for emergency reporting, safety information and administrative coordination. The system allows users to register, manage emergency contacts, report SOS incidents, share available location information and access emergency resources. Administrators can monitor incidents, manage users, emergency contacts, facilities, rescue teams, assignments and reports. The system uses **MongoDB with Mongoose** for data storage, while **JWT and password hashing** provide secure authentication and access control.

RakshaSafe also includes a separate **Python FastAPI risk-assessment service**. It uses stored incident and area information to provide an AI-assisted risk assessment. The system can use browser/device geolocation to store available latitude and longitude, with accuracy depending on device, permission and network conditions.

The project is designed with a focus on simple user interaction, role-based administration, data protection and clear emergency workflows. Its purpose is to improve the organisation of emergency information and support timely decision-making while keeping real-world emergency response dependent on available services and authorities.

CHAPTER 1 : INTRODUCTION

1.1 Introduction

Every year, thousands of people - especially women and college students - find themselves where they need urgent help but have no easy, trusted way to raise an alarm. A shout or a phone call is often not possible. RakshaSafe is a centralized, web-based emergency response system built to close this gap: a user raises an SOS in one tap, chosen contacts are alerted by SMS and e-mail, and responders reach the incident with live location, a computed risk level and nearby resources.

In an emergency the delay between 'something is wrong' and 'someone trustworthy can act' is too long. RakshaSafe attacks that delay with a panic button, an intelligent risk score and a complete responder workflow across three layers: React frontend, Node.js/Express backend and a Python AI risk service.

1.2 Background

This project started as a final-year B.Sc. (Information Technology) requirement, but the motivation is real-world. Panic-button concepts are old; what changed is that nearly every student now carries a smartphone with GPS, mobile data and messaging apps. An emergency system no longer needs dedicated hardware - the phone itself is the panic button.

1.3 Objectives of the Project

- To design and implement a location-aware emergency reporting system usable by non-technical users under stress.
- To notify primary emergency contacts automatically over SMS and e-mail.
- To produce a meaningful, explainable risk score (0-100) for every incident.
- To give rescue teams and administrators a complete assignment and management workflow with full auditability.
- To verify the whole system with real automated and manual tests - 127 unit, 67 integration and 70 live system checks (264 in total).

1.4 Purpose

This document is the complete project report. It records what RakshaSafe must do (Chapter 3), how it is designed (Chapter 4), how it was built (Chapter 5), how it was tested and with

what results (Chapter 6), and what the project concludes (Chapter 7). An examiner can follow the journey from problem to a tested, working system.

1.5 Scope

1.5.1 In Scope

- User accounts, profiles, SOS incidents and the full incident lifecycle.
- Emergency contacts with SMS / e-mail notification and an in-app feed.
- Facilities, rescue teams, assignments and unsafe-area reports.
- Admin dashboard, report generation and a complete audit log.
- AI risk scoring with a documented deterministic model.

1.5.2 Out of Scope

- Native Android / iOS apps (a responsive web UI is provided instead).
- Procurement or integration with government emergency call centres.
- Training of responders or change management inside a real organisation.

1.6 Applicability

Primary users are students travelling to and from campus, office-goers in the late evening and residents of areas with weak formal emergency coverage. Secondary users are college security teams, NGO volunteers and local responders who want a structured way to receive, acknowledge and resolve alerts. A live production SMS gateway is expensive, so every provider is optional; the design records exactly what happened, including NOT_CONFIGURED outcomes.

1.7 Achievements of the Project

- A working, tested emergency response system with one-tap SOS and live location capture.
- Automatic SMS / e-mail alerts to primary emergency contacts, with honest delivery status recording, including NOT_CONFIGURED outcomes.
- A deterministic, explainable AI risk score (0-100) computed for every incident.
- A complete responder workflow: rescue-team assignment, status progression and notifications.
- An administrator dashboard with user, facility, team and report management plus an audit log.

- Executed test evidence: 264 checks (127 unit, 67 integration, 70 live system), all passing.

1.8 Organization of the Report

Chapter 2 surveys existing systems and technologies and records the survey exactly as it was prepared. Chapter 3 analyses the requirements - functional and non-functional, data, hardware, software and the project schedule. Chapter 4 presents the system design: architecture, modules, database collections, API routes, risk model, user interfaces and security. Chapter 5 implements that design - it lists the main parts of the real code with screenshots of the running website. Chapter 6 tests the implemented system and records the real results for every module. Chapter 7 concludes the project and lists its limitations and future scope.

CHAPTER 2 : SURVEY OF TECHNOLOGY

2.1 Safety at Your Fingertips: The History of and Case for Panic Buttons

Title: Safety at Your Fingertips: The History of and Case for Panic Buttons

Author: CM3 Building Solutions — Editorial Team

Publication Year / Date: 2026

Publisher / Source: CM3 Building Solutions

Summary:

This reference explains the development of panic buttons from physical alarms to modern digital safety solutions. The concept is relevant to RakshaSafe because its SOS feature allows users to report emergency incidents with available location details.

2.2 Cell Phone Personal Safety Alarm

Title: Cell Phone Personal Safety Alarm

Author: Erik N. Hagenstad

Publication Year / Date: 20 September 2016

Publisher / Source: United States Patent and Trademark Office (USPTO), U.S. Patent No. 9,451,061

Summary:

This reference describes a mobile safety system combining a panic button, warning functions, communication and location support. The concept is related to RakshaSafe because the system combines emergency reporting with location information.

2.3 Global Positioning System (GPS)

Title: Global Positioning System (GPS)

Author: Multiple contributors

Publication Year / Date: 1978

Publisher / Source: NASA

Summary:

GPS is a satellite-based positioning technology used to determine geographical location. RakshaSafe uses available device location through the Browser/Device Geolocation API for emergency incident management.

2.4 WhatsApp

Title: WhatsApp

Author: Jan Koum and Brian Acton

Publication Year / Date: 2009

Publisher / Source: WhatsApp

Summary:

WhatsApp is a messaging and communication platform. In RakshaSafe, WhatsApp-based communication can be supported through a configured communication provider when the required service is available.

2.5 The History of WhatsApp Business API

Title: The History of WhatsApp Business API

Author: Qiscus — Editorial Team

Publication Year / Date: 8 January 2021

Publisher / Source: Qiscus

Summary:

The WhatsApp Business API provides communication between organizations and users through automated messages. It was studied as a possible communication channel for RakshaSafe.

2.6 Twilio

Title: Twilio

Author: Jeff Lawson, Evan Cooke and John Wolthuis

Publication Year / Date: 2008

Publisher / Source: Twilio

Summary:

Twilio provides programmable communication services such as SMS and voice. The technology is relevant to RakshaSafe because external communication providers can be configured for emergency notifications.

2.7 Evolution of Speech Recognition Technology — From Audrey to Siri

Title: Evolution of Speech Recognition Technology — From Audrey to Siri

Author: Verbit — Editorial Team

Publication Year / Date: 19 March 2019

Publisher / Source: Verbit

Summary:

This reference explains the development of speech recognition from early systems to modern voice technologies. Speech recognition was studied as a possible future emergency interaction feature for RakshaSafe.

2.8 Google Translate

Title: Google Translate

Author: Franz Josef Och and Google Translate development team

Publication Year / Date: 28 April 2006

Publisher / Source: Google

Summary:

Translation technology converts information between different languages. It was studied for RakshaSafe because multilingual support can make the safety system easier to use.

2.9 Decision Tree and Random Forest Algorithms: Decision Drivers

Title: Decision Tree and Random Forest Algorithms: Decision Drivers

Author: History of Data Science — Editorial Team

Publication Year / Date: 2022

Publisher / Source: History of Data Science

Summary:

This reference explains classification techniques such as Decision Trees and Random Forests. These techniques were studied as possible approaches for analysing emergency information and identifying incorrect alerts.

2.10 Random Forests

Title: Random Forests

Author: Leo Breiman

Publication Year / Date: October 2001

Publisher / Source: Machine Learning Journal, Springer

Summary:

The research introduced the Random Forest method, which combines multiple decision trees for

classification and regression. It provides academic background for possible future analysis of emergency data.

2.11 MongoDB and Mongoose Technology

Title: MongoDB and Mongoose Technology

Author: MongoDB / Mongoose contributors

Publication Year / Date: 2009

Publisher / Source: MongoDB / Mongoose

Summary:

MongoDB is a document-based NoSQL database, while Mongoose helps manage MongoDB data in Node.js applications. RakshaSafe uses them to store and manage application information such as users, incidents, locations and reports.

2.12 Node.js and Express.js

Title: Node.js and Express.js

Author: Ryan Dahl / Express.js contributors

Publication Year / Date: Node.js — 27 May 2009; Express.js — 2010

Publisher / Source: Node.js / Express.js

Summary:

Node.js provides the backend runtime and Express.js supports REST API development. RakshaSafe uses them for backend processing, authentication, database communication and API services.

2.13 React.js, TypeScript, Vite and Tailwind CSS

Title: React.js, TypeScript, Vite and Tailwind CSS

Author: React / Microsoft / Vite / Tailwind CSS teams

Publication Year / Date: React — 2013; TypeScript — 2012; Vite — 2020; Tailwind CSS — 2017

Publisher / Source: React, Microsoft, Vite and Tailwind CSS projects

Summary:

These technologies are used for the RakshaSafe frontend. React supports reusable components, TypeScript provides type checking, Vite supports development and builds, and Tailwind CSS supports interface styling.

2.14 Python, FastAPI and AI Service Integration

Title: Python, FastAPI and AI Service Integration

Author: Python Software Foundation / Sebastián Ramírez

Publication Year / Date: FastAPI — 5 December 2018

Publisher / Source: Python / FastAPI

Summary:

Python and FastAPI provide the separate AI-related service layer of RakshaSafe. This structure allows AI-related processing to be managed independently from the main Node.js backend.

2.15 OpenStreetMap, Leaflet and Location-Based Services

Title: OpenStreetMap, Leaflet and Location-Based Services

Author: Steve Coast / Vladimir Agafonkin and contributors

Publication Year / Date: OpenStreetMap — 9 August 2004; Leaflet — 17 May 2011

Publisher / Source: OpenStreetMap / Leaflet

Summary:

OpenStreetMap provides geographical map data, while Leaflet displays interactive maps. RakshaSafe uses location and mapping technologies to display relevant incident and risk-related geographical information.

CHAPTER 3

REQUIREMENT AND ANALYSIS

3.1 Requirement Analysis

Requirement analysis defines RakshaSafe's users, functions and resources. It covers SOS, incidents, location, emergency resources, admin management, centralized storage, and functional, non-functional, hardware, software and database requirements.

3.2 Problem Definition

Problem Statement

Users may face emergencies such as harassment, accidents, medical emergencies, fire, floods and earthquakes where quick and organized information is required. Administrators may also find it difficult to monitor incidents and maintain updated emergency resources.

Main Problems:

- Lack of a centralized safety platform.
- Difficulty recording and tracking incidents.
- Difficulty finding emergency facilities and resources.
- Lack of organized incident history.
- Difficulty managing emergency resources.
- Lack of centralized admin monitoring.
- Risk of unauthorized access to sensitive data.
- Difficulty maintaining unsafe-area and risk information.
- Difficulty generating useful reports and statistics.

3.2.1 Lack of Centralized Safety Records

Emergency incidents may include details such as type, description, user, location, priority and status. Storing this information in one centralized database makes it easier to access, monitor and analyse when needed.

.

3.2.2 Difficulty in Sharing Location Information

Location helps identify where an incident occurred. RakshaSafe can capture and store the available latitude and longitude with the incident. Accuracy depends on the device, GPS, network and user permission; if unavailable, no false location is stored.

3.2.3 Difficulty in Finding Emergency Resources

During an emergency, users may need quick access to hospitals, shelters, police stations, fire stations and relief centres. RakshaSafe stores facility details such as name, type, address, contact number, location and operational status. Capacity and operating hours can also be stored when available. Registered response teams are maintained separately.

3.2.4 Difficulty in Monitoring Emergency Incidents

Administrators require a structured method for monitoring emergency incidents.

Raksha provides an administrative dashboard where authorized administrators can:

- **View emergency incidents**
- **View incident details**
- **View available location information**
- **Check incident priority**
- **Check current status**
- **Assign registered response resources**
- **Update incident status**
- **View incident history**
- **Monitor system activity**

The assignment and status features represent software-based coordination and record management. They do not guarantee physical response from external organizations.

3.2.5 Difficulty in Maintaining Incident History

Emergency incidents may require a chronological record for future reference, reporting and analysis.

Raksha maintains incident updates that record important activities such as:

- **Incident creation**
- **Status change**
- **Assignment**
- **Progress update**
- **Resolution**
- **Closure**
- **Cancellation**
- **Administrative comments**

3.2.6 Difficulty in Managing Emergency Resources

Hospitals, shelters, police stations, fire stations, relief centres and response teams store different types of information. RakshaSafe therefore keeps them as structured records: facilities use the **Facilities** collection, while response teams use **RescueTeams**. Administrators can manage these records according to their permissions. The system shows stored information and does not claim real-time availability unless a reliable live source is connected.

4.1.11 Notification Management

Notifications keep users informed about important events such as incident creation, acknowledgement, assignment, status changes, resolution and cancellation. RakshaSafe supports In-app, Email, SMS and WhatsApp channels when configured. Each notification stores details such as Incident ID, Contact ID, Channel, Status, Provider Response, Attempt Count, Last Attempt Time and Created Date. Statuses include **QUEUED, SENT, DELIVERED, FAILED, NOT_CONFIGURED** and **UNAVAILABLE**.

3.3 Proposed Solution

RakshaSafe provides a centralized web application where registered users can create emergency incidents and access emergency-resource information.

When an SOS is submitted, the system creates an incident record and stores the available details. If location permission is granted, the available location information is linked to the incident.

Administrators can monitor incidents, update their status, manage facilities and response teams, and maintain administrative records. The system also supports notifications through configured services, depending on provider availability. A Python-based risk assessment service may process available incident and location factors to generate a risk score and level for software-based decision support.

System Limitations

- Raksha does not claim to provide
- Automatic police FIR registration
- Guaranteed police response
- Guaranteed ambulance response
- Guaranteed rescue-team response
- Exact rescue ETA
- Guaranteed 24×7 emergency response
- Real-time hospital bed availability without a connected source
- Offline emergency communication unless specifically implemented

Actual emergency response depends on authorized external organizations, service providers, network connectivity and real-world availability.

3.4 Requirements Specification

Requirements specification defines the functions and quality requirements of Raksha.

The requirements are divided into:

1. **Functional Requirements**
2. **Non-Functional Requirements**
3. **User Requirements**
4. **System Constraints**

5. **Hardware Requirements**
6. **Software Requirements**
7. **Database Requirements**

3.4.1 Functional Requirements

The major functional requirements of Raksha are:

- User registration and login
- User authentication
- Role-based authorization
- User profile management
- Emergency contact management
- SOS and incident creation
- Incident information management
- Location processing
- Emergency facility management
- Rescue-team management
- Rescue assignment management
- Incident status management
- Notification management
- Incident history management
- Unsafe-area reporting
- Risk-zone management
- Risk assessment
- Administrative dashboard
- Analytics and reporting
- Resource search and filtering
- Administrative activity logging
- Secure access to protected information

3.4.1.1 User Authentication and Authorization

The system shall provide authentication so that only registered users can access protected features.

Requirements:

- The system shall allow users to register.
- The system shall allow users to log in.
- The system shall validate login credentials.
- Passwords shall not be stored in plain text.
- Passwords shall be securely hashed.

- The system shall identify the role of the logged-in account.
- Unauthorized users shall not access administrator functions.
- Users shall be able to log out.
- Protected API operations shall require proper authorization.
- Authentication shall use secure token-based access.

3.4.1.2 Role-Based Access Control

Raksha separates system operations according to user roles.

User

- A normal user can:
- Register
- Login
- Logout
- Manage profile
- Manage emergency contacts
- Create emergency incidents
- Share location where available
- View own incidents
- View incident status
- View notifications
- View emergency resources
- Submit unsafe-area reports
- View relevant risk information where provided

Administrator

- An administrator can:
- Login securely
- View emergency incidents
- View incident details
- View available locations
- Update incident status
- Manage facilities
- Manage rescue-team records
- Manage rescue assignments
- Review unsafe-area reports
- Manage risk-zone information
- View dashboard statistics

- View reports
- Monitor administrative activity

3.4.1.3 User Profile Management

The system shall provide basic profile functionality.

Users may:

- View profile information
- Update permitted profile information
- Manage account information
- View emergency history
- Manage emergency contacts
- Logout from the application

Only required information should be collected.

3.4.1.4 Emergency Contact Management

The system shall allow users to maintain emergency contact information.

An emergency contact may contain:

- Contact name
- Phone number
- Email address
- Relationship
- SMS notification preference
- Email notification preference

Emergency contact information shall only be accessible to authorized users and system functions.

3.4.1.5 SOS / Emergency Incident Management

SOS management is one of the main functions of Raksha.

When a user submits an SOS or emergency request, the application creates an incident record.

An incident can contain:

- Incident ID
- User ID
- Incident type
- Category
- Description
- Priority

- Location ID
- Current status
- Creation date and time
- Resolution information, where applicable

Incident Types

Examples include:

- Women Safety
- Medical Emergency
- Accident
- Fire
- Flood
- Earthquake
- Other Emergency
- Incident Status

The system uses the following detailed workflow:

REPORTED → ACKNOWLEDGED → ASSIGNED → IN_PROGRESS → RESOLVED → CLOSED

An incident may also be marked:

CANCELLED

where applicable.

In the database design, the SOS or emergency request is stored as an Incident record.

3.4.1.6 Location Processing

The system can request location permission from the user's browser or device.

When available, the system may capture:

- Latitude
- Longitude
- Address or location label
- City
- State
- Country
- Location accuracy
- Location capture time

The system shall:

- Request permission where required.
- Capture coordinates when available.

- Associate location data with the relevant incident.
- Handle unavailable location data.
- Avoid creating false location information.
- Restrict access to sensitive location information.
- Location accuracy depends on the user's device, browser permissions, GPS/network conditions and configured location services.

3.4.1.7 Emergency Facility Management

Raksha maintains information about emergency facilities.

The Facilities collection may contain:

- Facility name
- Facility type
- Location ID
- Phone number
- Capacity, where maintained
- Operational status
- Operating hours
- Created date
- Updated date

Facility types may include:

- Hospital
- Shelter
- Police Station
- Fire Station
- Relief Centre

The system does not claim real-time bed availability or actual operational response unless such information is connected to a reliable source.

3.4.1.8 Rescue Team Management

The system can maintain records of registered response teams.

A rescue-team record may contain:

- Team name
- Team type
- Contact information
- Email
- Active status

- Service area
- Specialization

Team types may include:

- Police
- Medical
- Fire
- Volunteer
- NGO

The system manages these records as software information and does not guarantee real-world team availability.

3.4.1.9 Rescue Assignment Management

Administrators can create a software-based assignment record between an incident and a registered response team.

The assignment may contain:

- Incident ID
- Team ID
- Assigned by
- Assigned date and time
- Assignment status
- Notes

Assignment status may include:

- ASSIGNED
- EN_ROUTE
- ON_SCENE
- COMPLETED
- CANCELLED

These statuses represent information recorded within the application and should not be interpreted as guaranteed physical movement or response.

3.4.1.10 Incident Status and Monitoring

Administrators can monitor emergency incidents through the dashboard.

The dashboard may display:

- Total incidents
- Reported incidents
- Acknowledged incidents

- Assigned incidents
- In-progress incidents
- Resolved incidents
- Closed incidents
- Cancelled incidents
- Recent incidents
- Incident categories
- Priority information

When an administrator changes an incident status, the system stores the change and can create an incident-update record and notification.

3.4.1.11 Notification Management

Notifications keep users updated about emergency events through **In-app, Email, SMS, or WhatsApp**. The system stores basic notification details and supports statuses such as **QUEUED, SENT, DELIVERED, FAILED, NOT_CONFIGURED, and UNAVAILABLE**.

.

3.4.1.12 Incident History

Raksha maintains a chronological activity record for incidents.

The history can contain:

- Incident creation
- Status change
- Assignment
- Progress update
- Resolution
- Closure
- Cancellation
- Administrative comment

The Incident Updates collection is used for maintaining this chronological history.

3.4.1.13 Unsafe Area Reporting

Users can report locations that they consider unsafe or risky.

An unsafe-area report may contain:

- Reporter ID
- Location ID
- Category
- Description

- Severity
- Verification status
- Created date
- Updated date

Administrators can review and verify submitted reports.

A user-submitted report should not automatically be treated as a confirmed dangerous location until it has been reviewed according to the implemented workflow.

3.4.1.14 Risk Zone Management

The system can maintain structured information about identified risk zones.

A risk-zone record may contain:

- Zone name
- Risk level
- Geographic boundary
- Risk factors
- Last assessment date
- Active status

Risk-zone information shall be managed according to administrator permissions.

3.4.1.15 Risk Assessment

Raksha can use a Python and Fast API-based risk assessment service to process available information related to an incident or location.

The service may produce:

- Risk score
- Risk level
- Model version
- Input factors
- Assessment time

The risk score may use a range such as 0–100.

Risk levels may include:

- LOW
- MEDIUM
- HIGH
- CRITICAL

Risk assessment is intended to support information analysis. It does not provide guaranteed predictions or replace professional emergency decisions.

3.4.1.16 Disaster Category Management

The system can maintain structured disaster and emergency categories.

A category may contain:

- Category name
- Category code
- Description
- Default priority
- Response-team requirement
- Active status
- This allows incident categories to be maintained without modifying the complete application structure.

3.4.1.17 Administrative Dashboard

The administrative dashboard provides a central management interface.

It can display:

- Total users
- Total incidents
- Reported incidents
- Active incidents
- Resolved incidents
- Cancelled incidents
- Available facilities
- Registered rescue teams
- Recent activities
- Notifications
- Basic statistics

Dashboard information shall be generated from actual stored system data.

3.4.1.18 Analytics and Reporting

The system can generate basic analytics from stored data.

Examples include:

- Number of incidents
- Incidents by type
- Incidents by priority
- Incidents by status
- Monthly incident count
- Number of facilities

- Number of shelters
- Number of registered response teams
- Unsafe-area reports
- Risk-level distribution

Analytics represent stored application data and should not be presented as guaranteed predictions of future emergencies.

3.4.1.19 Resource Search

Users can search and filter emergency resources.

Filtering may be based on:

- Resource type
- Facility name
- Area
- Location
- Service category
- Operational status

Where a configured map service is available, location-based information may also be displayed through a map interface.

3.4.1.20 Administrative Activity Logging

Administrative actions should be recorded for accountability.

An administrative log may contain:

- Administrator ID
- Action
- Target type
- Target ID
- Action details
- IP address, where collected
- Date and time

Administrative logs should be accessible only to authorized administrators.

3.4.1.21 Data Management

The system shall maintain structured records for:

- Users
- Emergency Contacts
- Incidents
- Locations

- Notifications
- Rescue Teams
- Rescue Assignments
- Incident Updates
- Unsafe Area Reports
- Risk Zones
- Risk Assessments
- Facilities
- Disaster Categories
- Administrative Logs
- Reports

Database operations shall be performed through authorized backend APIs.

3.4.2 Non-Functional Requirements

The major non-functional requirements are:

1. Security
2. Performance
3. Usability
4. Responsiveness
5. Reliability
6. Scalability
7. Maintainability
8. Compatibility
9. Data Integrity
10. Availability

3.4.2.1 Security

The system shall protect:

- User accounts
- Passwords
- Emergency incidents
- Location information
- Emergency contacts
- Administrative information
- Risk-related information

JWT-based authorization and secure password hashing shall be used for protected operations.

3.4.2.2 Performance

Common operations should provide reasonable response time under normal project usage, including:

- Login
- SOS submission
- Resource search
- Dashboard loading
- Status update
- Notification creation

3.4.2.3 Usability

The application should be simple and easy to understand.

Important functions should have clear labels such as:

- SOS / Emergency
- Submit Request
- View Status
- Find Hospital
- Find Shelter
- Emergency Contacts
- Admin Dashboard

3.4.2.4 Responsiveness

The application should support:

- Desktop
- Laptop
- Tablet
- Mobile phone

The interface should adjust according to screen size.

3.4.2.5 Reliability

After successful submission, the incident should remain stored in the database.

The system should handle failed external services without corrupting the incident record.

3.4.2.6 Scalability

The system should allow additional users, incidents, resources and administrative records without requiring a complete redesign.

3.4.2.7 Maintainability

The application should be divided into logical modules such as:

- Authentication
- User Management
- SOS/Incident Management
- Location
- Facilities
- Rescue Teams
- Notifications
- Risk Assessment
- Administration
- Analytics

3.4.2.8 Compatibility

The web application should work with commonly used modern browsers.

3.4.2.9 Data Integrity

The system should maintain accurate relationships between database records.

For example, an incident should reference a valid user and an available location record where applicable.

3.4.2.10 Availability

The application should remain available when the hosting environment, database, network and required external services are operational.

3.4.3 User Requirements

Raksha mainly supports two primary system roles:

1. User

The user is the person who uses Raksha to create emergency incidents and access available safety information.

Users shall be able to:

- Register
- Login
- Logout
- Manage profile
- Manage emergency contacts
- Create SOS incidents
- Select incident type
- Enter incident description

- Share location where available
- View incident status
- View incident history
- View notifications
- Search emergency resources
- Submit unsafe-area reports

2. Administrator

The administrator manages system information and monitors emergency incidents.

Administrators shall be able to:

- Login securely
- View incidents
- View incident details
- View available locations
- Update incident status
- Manage facilities
- Manage rescue teams
- Manage assignments
- Review unsafe-area reports
- Manage risk-zone information
- View dashboard statistics
- Generate reports
- Monitor administrative activities

3.4.4 User vs Administrator

Function	User	Administrator
Register	Yes	Usually pre-created
Login	Yes	Yes
Manage Profile	Yes	Limited
Create SOS	Yes	Optional
View Own Incidents	Yes	Yes
View All Incidents	No	Yes
Update Incident Status	No	Yes
Manage Facilities	No	Yes
Manage Rescue Teams	No	Yes
Create Assignments	No	Yes
View Notifications	Yes	Yes
Submit Unsafe Area Report	Yes	Yes
Verify Reports	No	Yes
Manage Risk Zones	No	Yes
View Dashboard	Limited	Yes

Analytics	Limited	Yes
Generate Reports	No/Limited	Yes

3.4.5 System Constraints

The following constraints apply to Raksha:

- The system requires internet connectivity for normal web-based functionality.
- Location functionality depends on browser/device permission.
- Location accuracy depends on the device and available location services.
- Map functionality depends on the configured map service.
- External email, SMS and WhatsApp services depend on provider availability and configuration.
- Database availability affects application functionality.
- Hosting performance affects response time.
- Unauthorized users must not access protected information.
- Location information must be protected.
- Emergency response cannot be guaranteed by the software.
- Automatic police FIR generation is not assumed.
- Exact rescue ETA is not assumed.
- 24×7 police or ambulance integration is not assumed unless actually implemented.
- Real-time hospital bed availability is not assumed unless connected to a reliable source.
- Offline emergency functionality is not assumed unless specifically implemented.
- Risk assessment results are software-generated decision-support information and are not guaranteed predictions.
- Notification delivery depends on configured external communication providers.

3.4.6 Hardware Requirements

Raksha does not require specialized hardware.

Component/hardware	Requirement
Processor	Intel Core i5 / AMD Ryzen 5 or equivalent
RAM	Minimum 8 GB
Storage	Minimum 256 GB SSD
Display	1080p or higher recommended
Network	Stable Internet connection
Development Device	Normal laptop or desktop
Mobile Device	Smartphone with location capability recommended

3.4.7 Software Requirements

The final software stack should reflect the technologies actually used in the implemented project.

Component	Technology
Frontend	React.js
Frontend Language	TypeScript
Build Tool	Vite
Styling	Tailwind CSS
Backend Runtime	Node.js
Backend Framework	Express.js
API Architecture	REST API
AI Backend	Python
AI Framework	FastAPI
AI Integration	Configured AI Service API
Database	MongoDB/superbase
ODM	Mongoose
Authentication	JSON Web Token (JWT)
Password Security	bcrypt
Location	Browser/Device Geolocation API
Maps	Configured Map Service
Communication	Configured Email/SMS/WhatsApp Providers
Development Environment	Visual Studio Code, Bolt
Version Control	Git, GitHub
Testing	Jest / Supertest, where implemented
Deployment	Configured Cloud Hosting Environment

3.4.8 Database Requirement and Collection Mapping

RakshaSafe uses **MongoDB** to store application data, with **Mongoose** for schemas and validation. Relationships are managed through application-level references instead of traditional foreign keys.

The proposed database contains the following collections:

No.	Collection	Purpose
1	Users	User accounts and profile information
2	EmergencyContacts	User emergency contacts
3	Incidents	SOS and emergency incident records
4	Locations	Geographic location information
5	Notifications	Notification records and delivery status
6	RescueTeams	Registered response-team information

7	RescueAssignments	Incident-to-team assignment records
8	IncidentUpdates	Incident history and status updates
9	UnsafeAreaReports	User-submitted unsafe-area reports
10	RiskZones	Structured risk-zone information
11	RiskAssessments	Risk scores and assessment results
12	Facilities	Hospitals, shelters, police/fire stations and relief centres
13	DisasterCategories	Emergency/disaster categories
14	AdminLogs	Administrative activity records
15	Reports	Generated reports and analytics records

3.5.1 Planning and Scheduling

Sr. No.	Project Activity	Start Date	End Date	Duration	Project Done By
1	Project Planning & Requirements	15-Jun-2026	22-Jun-2026	8 Days	Vivek & Vaibhav – Topic selection, planning and requirements
2	System Analysis & Design	23-Jun-2026	28-Jun-2026	6 Days	Vivek – System flow; Vaibhav – UML and design
3	Database Design	29-Jun-2026	03-Jul-2026	5 Days	Vivek – Database structure; Vaibhav – Collections and fields
4	UI & Frontend Design	04-Jul-2026	27-Jul-2026	24 Days	Vivek – User interface; Vaibhav – Admin interface
5	Backend Development	09-Jul-2026	25-Jul-2026	17 Days	Vivek – APIs; Vaibhav – Backend and database connection

6	SOS & Location Module	28-Jul-2026	03-Aug-2026	7 Days	Vivek – SOS; Vaibhav – Location features
7	Emergency & Rescue Modules	04-Aug-2026	14-Aug-2026	11 Days	Vivek – Resource search; Vaibhav – Rescue team and assignments
8	Risk & Disaster Management	15-Aug-2026	21-Aug-2026	7 Days	Vivek & Vaibhav – Risk zones, assessment and disaster features
9	Admin Dashboard & Integration	22-Aug-2026	03-Sep-2026	13 Days	Vivek – System support; Vaibhav – Dashboard; Both – Integration
10	Testing & Debugging	04-Sep-2026	11-Sep-2026	8 Days	Vivek & Vaibhav – Testing and error fixing
11	Documentation & Final Review	12-Sep-2026	27-Sep-2026	16 Days	Vivek – Requirements; Vaibhav – Design; Both – Final review
12	Final Corrections, Deployment & Submission	28-Sep-2026	30-Sep-2026	3 Days	Vivek & Vaibhav – Final corrections, deployment and submission

3.5.1 Gantt Chart

The Gantt Chart shows the planned schedule of the Raksha project, including requirement analysis, system design, development, database implementation, testing, documentation, and final deployment. It helps monitor the progress and completion of each project activity.

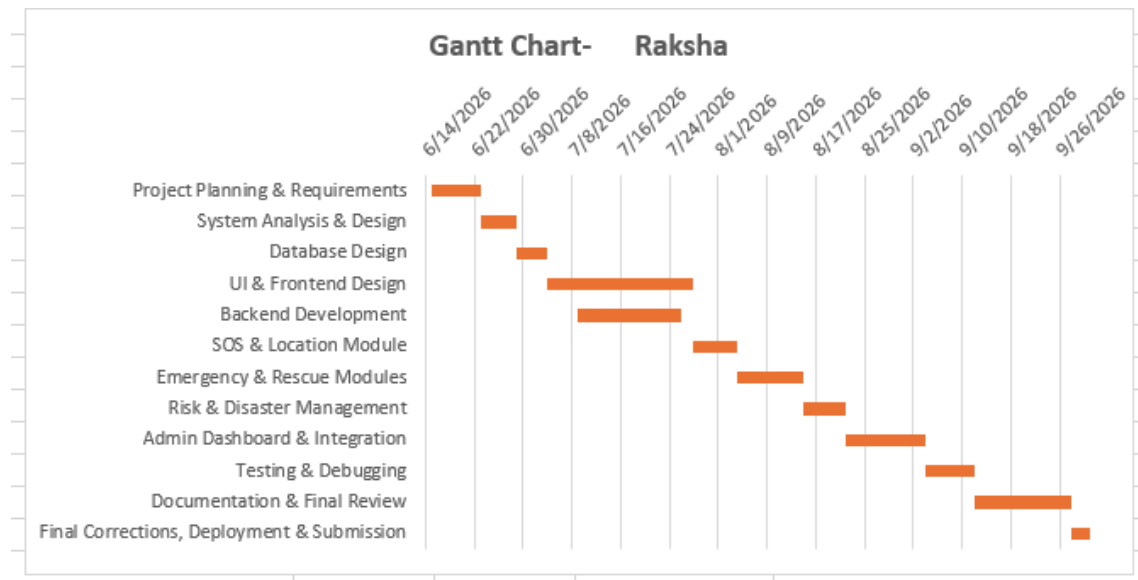


Figure 3.1: Gantt chart - planned project schedule (15 Jun to 30 Sep 2026).

3.5.2 PERT CHART

PERT (Program Evaluation and Review Technique) is used to estimate the time required to complete the Raksha – Women Safety and Disaster Management System. It represents the major project activities in a network form and shows the sequence in which the activities are carried out.

Optimistic Time (O): Minimum expected time

Most Likely Time (M): Normal expected time

Pessimistic Time (P): Maximum expected time

Expected Time is calculated as:

$$TE = (O + 4M + P) / 6$$

Standard Deviation:

$$S.D. = (P - O) / 6$$

PERT Time Estimation Table – RAKSHA

Task	Project Activity	Optimistic Time (O)	Most Likely Time (M)	Pessimistic Time (P)	Expected Time (TE)	S.D.
T1	Project Planning & Requirements	6	8	10	8.00	0.67
T2	System Analysis & Design	5	6	8	6.17	0.50
T3	Database Design	4	5	6	5.00	0.33

T4	UI & Frontend Design	20	24	28	24.00	1.33
T5	Backend Development	14	17	20	17.00	1.00
T6	SOS & Location Module	6	7	9	7.17	0.50
T7	Emergency & Rescue Modules	9	11	13	11.00	0.67
T8	Risk & Disaster Management	6	7	9	7.17	0.50
T9	Admin Dashboard & Integration	11	13	15	13.00	0.67
T10	Testing & Debugging	6	8	10	8.00	0.67
T11	Documentation & Final Review	14	16	18	16.00	0.67
T12	Final Corrections, Deployment & Submission	2	3	4	3.00	0.33

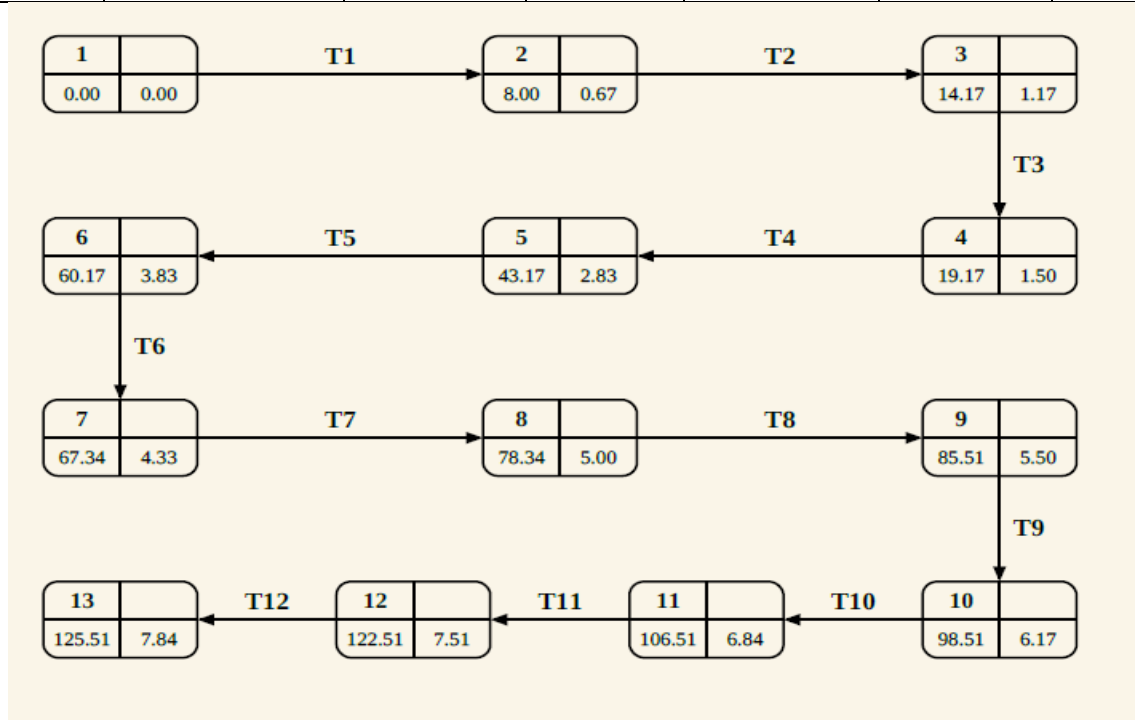


Figure 3.2: PERT network chart - RAKSHA project, with the expected times (T1-T12).

3.6 Conceptual Models

The conceptual model of Raksha Safe shows how the main parts of the system work together. A user can register, add emergency contacts, report an emergency, and use the SOS feature. The system stores the incident and available location details, and shows nearby rescue resources such as police, hospitals, fire services, and volunteers. The AI module provides a

basic risk assessment to support the emergency process. The admin manages incidents, users, rescue teams, assignments, notifications, and reports. Overall, the model shows how information moves between the user, system, AI module, rescue resources, and admin.

3.6.1 ER Diagram

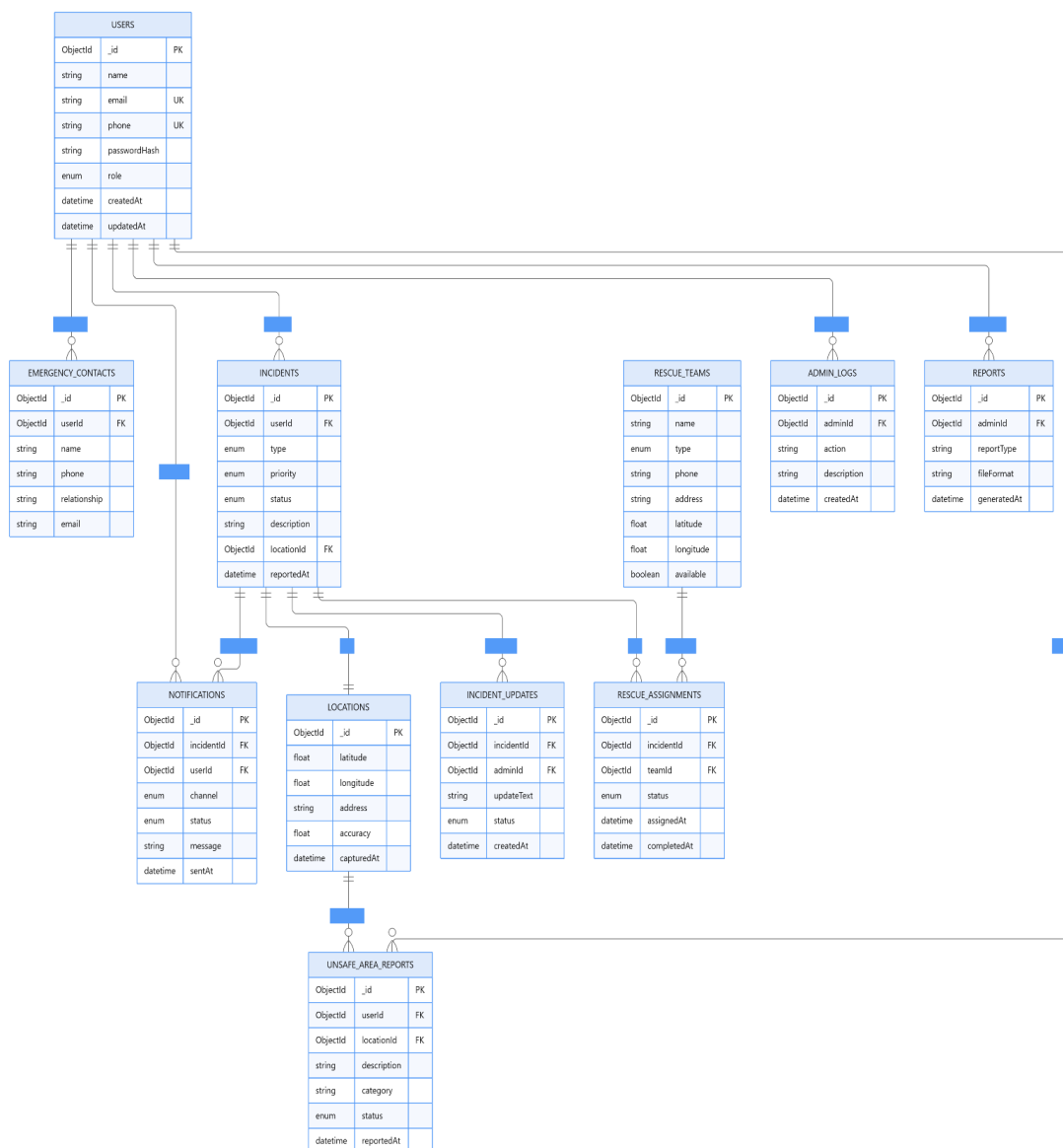


Figure 3.3: ER diagram - conceptual model of RakshaSafe.

3.6.2 Data Dictionary

The Data Dictionary describes the fields, data types and constraints of the major MongoDB collections used in the Raksha – Women Safety and Disaster Management System. The system contains 15 main collections for managing users, emergency incidents, locations, notifications, rescue resources, risk information, administrative activities and reports.

1. Users

Sr. No.	Field	Type	Constraint
---------	-------	------	------------

1	_id	ObjectId	Primary Key, Auto-generated
2	Name	String	Required
3	Email	String	Required, Unique
4	phone	String	Required, Unique
5	passwordHash	String	Required, Hashed
6	Role	String	Required, Enum: USER / ADMIN
7	language	String	Optional
8	createdAt	Date	Auto-generated
9	updatedAt	Date	Auto-generated

2. Emergency Contacts

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated
2	userId	ObjectId	Reference → Users
3	Name	String	Required
4	Phone	String	Required
5	Email	String	Optional
6	relationship	String	Optional
7	notifyViaSms	Boolean	Required
8	notifyViaEmail	Boolean	Required
9	createdAt	Date	Auto-generated
10	updatedAt	Date	Auto-generated

3. Incidents

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated
2	userId	ObjectId	Reference → Users
3	Type	String	Required, Safety / Disaster
4	category	String	Required
5	description	String	Required
6	priority	String	Required, LOW / MEDIUM / HIGH / CRITICAL
7	status	String	Required, Enum
8	locationId	ObjectId	Reference → Locations, Optional
9	createdAt	Date	Auto-generated
10	resolvedAt	Date	Optional

4. Locations.

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated

2	latitude	Number	Required
3	longitude	Number	Required
4	address	String	Optional
5	City	String	Optional
6	state	String	Optional
7	country	String	Optional
8	accuracy	Number	Optional
9	createdAt	Date	Auto-generated
10	updatedAt	Date	Auto-generated

5. Notifications

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated
2	incidentId	ObjectId	Reference → Incidents
3	contactId	ObjectId	Reference → EmergencyContacts, Optional
4	channel	String	Required, Email / SMS / WhatsApp / In-App
5	Status	String	Required, Enum
6	providerResponse	String	Optional
7	attemptCount	Number	Required
8	lastAttemptAt	Date	Optional
9	createdAt	Date	Auto-generated
10	updatedAt	Date	Auto-generated

6. RescueTeams

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated
2	name	String	Required
3	teamType	String	Required, Police / Medical / Fire / NGO / Volunteer
4	phone	String	Required
5	email	String	Optional
6	isActive	Boolean	Required
7	specializations	Array	Optional
8	createdAt	Date	Auto-generated
9	updatedAt	Date	Auto-generated

7. RescueAssignments

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated

2	incidentId	ObjectId	Reference → Incidents
3	teamId	ObjectId	Reference → RescueTeams
4	assignedBy	ObjectId	Reference → Users
5	assignedAt	Date	Auto-generated
6	status	String	Required, Enum
7	notes	String	Optional
8	createdAt	Date	Auto-generated
9	updatedAt	Date	Auto-generated

8. IncidentUpdates

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated
2	incidentId	ObjectId	Reference → Incidents
3	updatedBy	ObjectId	Reference → Users
4	statusFrom	String	Optional
5	statusTo	String	Required
6	comment	String	Optional
7	createdAt	Date	Auto-generated

9. UnsafeAreaReports

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated
2	reportedBy	ObjectId	Reference → Users
3	locationId	ObjectId	Reference → Locations
4	category	String	Required
5	description	String	Required
6	severity	String	Required
7	isVerified	Boolean	Required
8	createdAt	Date	Auto-generated
9	updatedAt	Date	Auto-generated

10. RiskZones

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated
2	name	String	Required
3	riskLevel	String	Required
4	geometry	Object	Required
5	factors	Array	Optional
6	lastAssessedAt	Date	Optional
7	isActive	Boolean	Required

8	createdAt	Date	Auto-generated
9	updatedAt	Date	Auto-generated

11. RiskAssessments

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated
2	locationId	ObjectId	Reference → Locations
3	riskScore	Number	Required, Range: 0-100
4	riskLevel	String	Required, LOW / MEDIUM / HIGH / CRITICAL
5	modelVersion	String	Required
6	inputFactors	Array	Optional
7	assessedAt	Date	Auto-generated
8	createdAt	Date	Auto-generated

12. Facilities

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated
2	name	String	Required
3	facilityType	String	Required, Enum
4	locationId	ObjectId	Reference → Locations
5	phone	String	Required
6	capacity	Number	Optional
7	isOperational	Boolean	Required
8	operatingHours	String	Optional
9	createdAt	Date	Auto-generated
10	updatedAt	Date	Auto-generated

13. Disaster Categories

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated
2	Name	String	Required
3	Code	String	Required, Unique
4	description	String	Optional
5	defaultPriority	String	Required
6	requiresResponseTeam	Boolean	Required
7	isActive	Boolean	Required
8	createdAt	Date	Auto-generated
9	updatedAt	Date	Auto-generated

14. AdminLogs

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated
2	adminId	ObjectId	Reference → Users
3	action	String	Required
4	targetType	String	Required
5	targetId	ObjectId	Optional
6	details	String	Optional
7	ipAddress	String	Optional
8	createdAt	Date	Auto-generated

15. Reports

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated
2	generatedBy	ObjectId	Reference → Users
3	Title	String	Required
4	reportType	String	Required
5	filters	Object	Optional
6	dataSnapshot	Object	Optional
7	format	String	Required, PDF / CSV / JSON
8	createdAt	Date	Auto-generated
9	expiresAt	Date	Optional

3.6.3 DFD LEVEL 0 — CONTEXT DIAGRAM

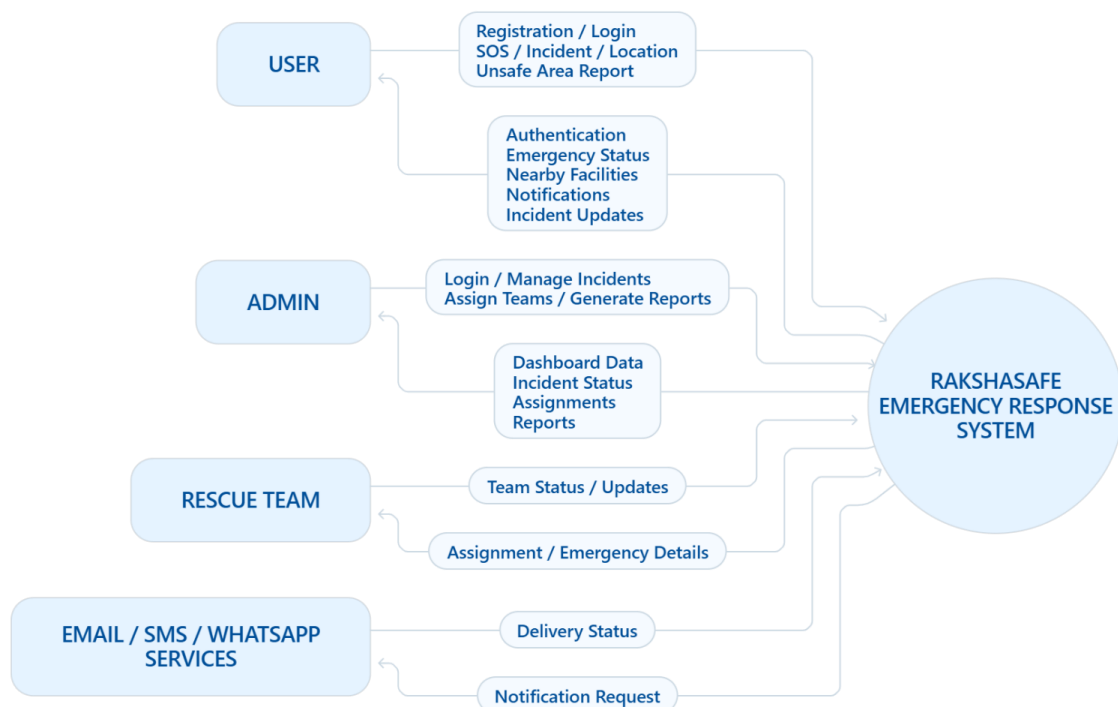
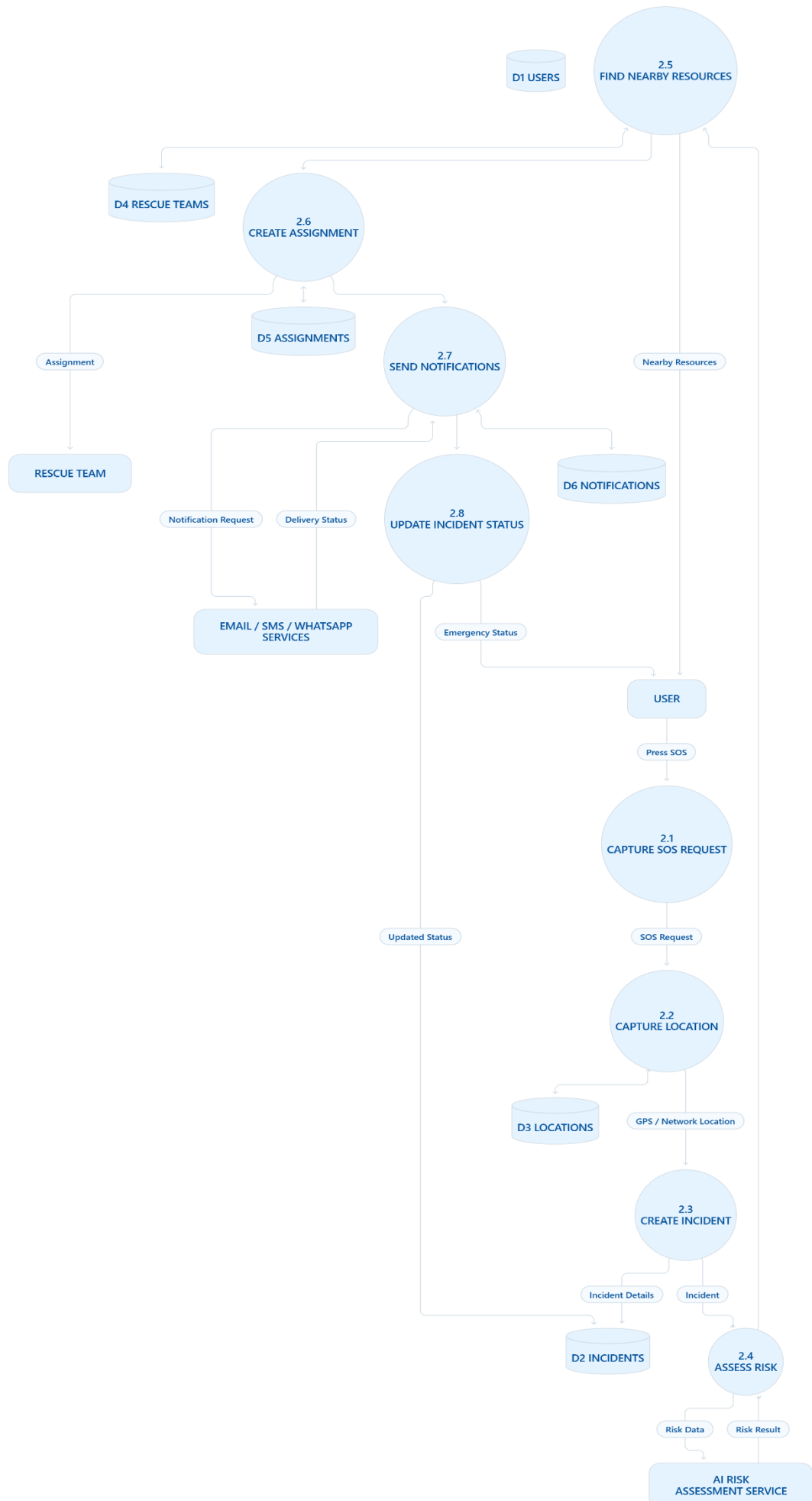
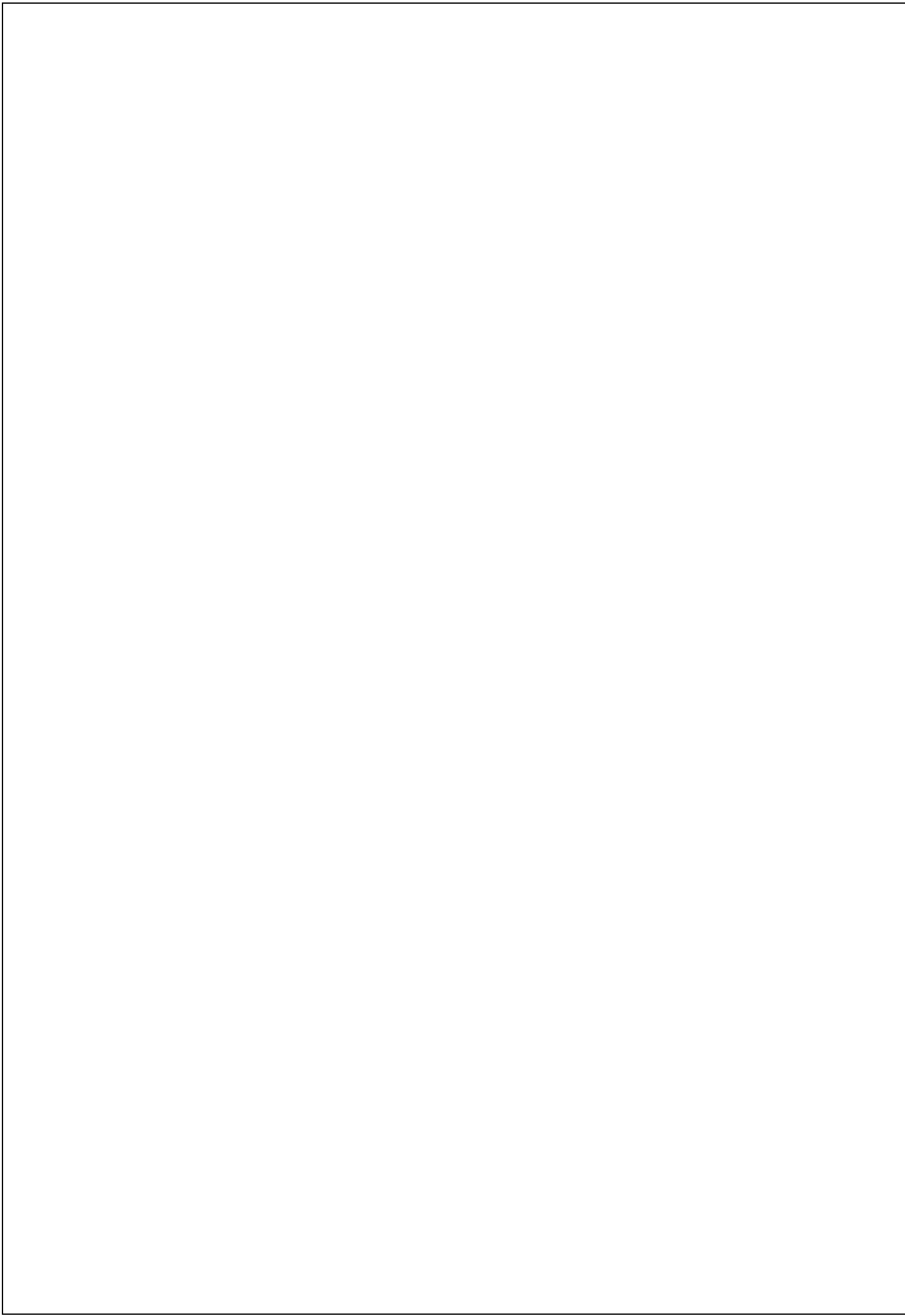


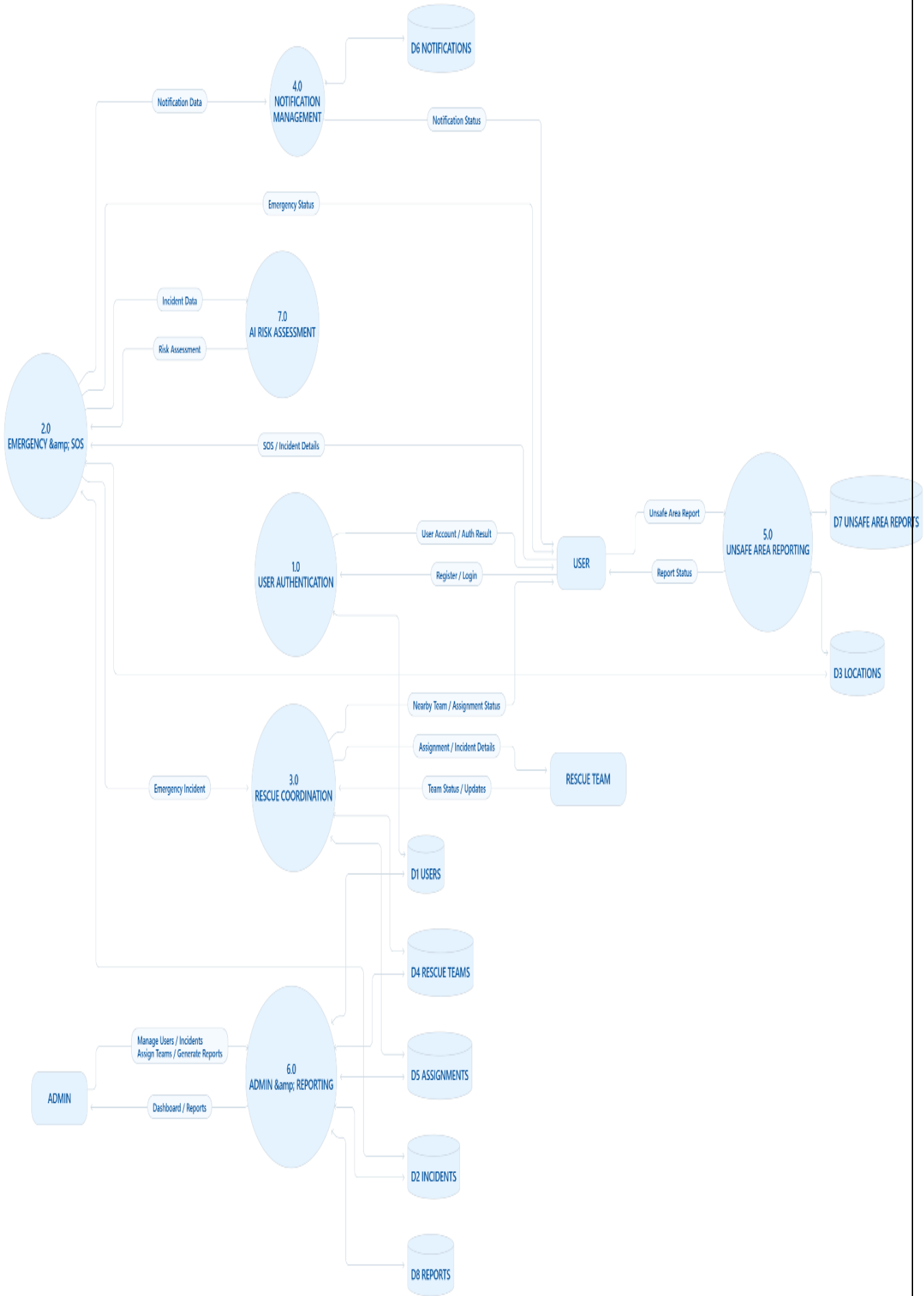
Figure 3.4: DFD Level 0 - context diagram of RakshaSafe.

DFD LEVEL 1 — MAIN MODULES





DFD LEVEL 2 — SOS / EMERGE



CHAPTER 4 :SYSTEM DESIGN

Module	What it covers	Main screens / API
Authentication	Registration, login, logout and JWT sessions for all three roles	Login, Register; /api/auth/*
User Management	Profiles, language preference and account state	Profile; /api/auth/me, /api/auth/profile
SOS / Incident Management	Emergency reporting, status workflow, history and ownership	SOS, Incident Detail, Dashboard; /api/incidents/*
Location	Coordinate capture, stored points and readable addresses	SOS location step, Location cards; /api/geocode/*
Emergency Contacts	Per-user contacts with call links and alert channels	Emergency Contacts, Admin Emergency Contacts; /api/emergency-contacts/*, /api/admin/emergency-contacts
Emergency Resources	Searchable directory and nearby lookup of help	Resources, Nearby sections; /api/facilities, /api/nearby-resources, /api/nearby
Facilities	Hospitals, shelters, police and fire stations, relief centres	Resources, Admin Facilities; /api/facilities, /api/admin/facilities
Rescue Teams	Registered teams with type, contact, members and base	Resources, Admin Teams; /api/rescue-teams, /api/admin/rescue-teams
Rescue Assignments	Incident-to-team assignment records and their lifecycle	Incident Detail, Admin incident detail; /api/admin/incidents/:id/assignments, /api/responder/assignments
Unsafe-Area Reporting	User reports with admin verification queue	Unsafe Reports, Admin Unsafe Reports; /api/unsafe-reports/*, /api/admin/unsafe-reports
Risk Zones	Administrator-maintained boundary records with risk levels	Admin masters; RiskZones collection
Risk Assessment (AI)	Deterministic raksha-risk-v1 scoring of incident locations	Risk panels; /api/incidents/:id/risk, FastAPI POST /risk/assess
Notifications	In-app and provider-channel delivery records with status	Notifications; /api/notifications, /api/responder/notifications
Weather Enrichment	Informational current weather for incident coordinates	Weather cards; /api/weather (Open-Meteo, no key)

Administration	Dashboard, users, reports, activity logs and analytics	Admin Dashboard, Users, Reports; /api/admin/*
----------------	--	---

4.1 System Architecture

RakshaSafe's system design shows how its main components work together to manage incidents, locations, resources, rescue assignments, notifications and risk assessment. The system uses a **React web application, Node.js backend, Python AI service**, and **MongoDB** for storing application data. The backend handles authentication, validation and database updates.

4.1.1 Overall System Working

The overall working of RakshaSafe is divided into the following stages:

1. User Authentication

- a. New users register with name, email, phone and password. Existing users, administrators and responders sign in using their registered email or phone and password.
- b. Passwords are securely hashed using bcrypt and never stored in plain text.
- c. After successful authentication, the backend issues a JSON Web Token (JWT). Users are redirected according to their role: USER → user dashboard, ADMIN → admin dashboard, and RESPONDER → responder workspace.

2. SOS and Incident Creation

- a. The signed-in user opens the SOS page and provides the incident type, category, description and priority (LOW, MEDIUM, HIGH or CRITICAL).
- b. The backend validates the request, creates an Incidents record with status REPORTED, and returns the stored incident. It then appears on the user dashboard and admin incident list.

3. Location Capture and Association

- a. With user permission, the browser Geolocation API provides latitude, longitude and accuracy. These coordinates are sent with the incident and stored as a Locations record linked through `locationId`.
- b. A server-side reverse-geocode step provides a readable address and location details. If permission is denied or the location is unavailable, the incident is recorded without coordinates; no false location is assumed or stored.

4. Emergency Resource Lookup

- a. The system lists nearby Facilities and active RescueTeams around the real query point, sorted by straight-line distance.

b. Each card shows available name, type, address and distance. Call **and** Directions links are shown only when real phone numbers or coordinates exist. If an external places provider is configured, its results are merged with stored resources; otherwise, the system reports its unavailability and continues with stored resources.

5. Rescue Assignment

a. An administrator assigns a registered, active rescue team to an incident, optionally with notes. b. Assignment status follows ASSIGNED → EN_ROUTE → ON_SCENE → COMPLETED, with CANCELLED where applicable. Every change is a software record in RescueAssignments; the system never claims automatic dispatch, guaranteed response or an exact arrival time.

6. AI Risk Assessment

a. For an incident with stored coordinates, the user or admin can request an assistive risk assessment. The backend sends priority, incident type, nearby unsafe-area reports and active incidents to the FastAPI AI service.

b. The deterministic raksha-risk-v1 model returns a 0–100 risk score, risk level, model version, input factors and assessment time. The validated result is stored in RiskAssessments and shown to users and admins with a decision-support disclaimer.

7. Notifications

a. Incident events such as creation, acknowledgement, assignment, status changes, resolution and cancellation create Notification records for the incident owner and configured emergency-contact channels.

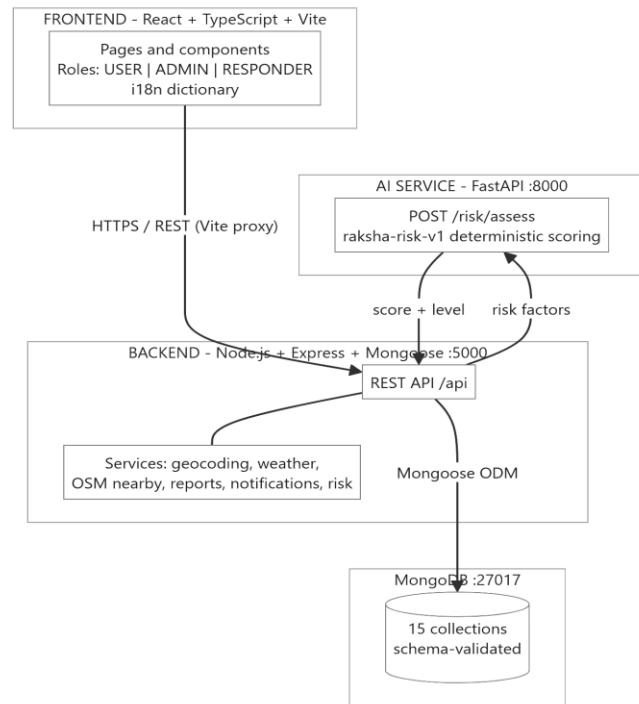
b. Each notification stores its channel, status, provider response, attempt count and timing. Delivery depends on configured providers, and a delivery failure does not affect the incident record.

8. Administration and Reporting

a. Administrators manage users, emergency contacts, incidents, facilities, rescue teams, unsafe-area reports, risk zones, reports and admin activity logs through the admin dashboard.

b. Incident status follows REPORTED → ACKNOWLEDGED → ASSIGNED → IN_PROGRESS → RESOLVED → CLOSED, with CANCELLED where applicable. Each transition is stored in IncidentUpdates with an optional comment.

Figure 4.1: RakshaSafe System Architecture



RakshaSafe System Architecture

4.1.2 Technology Used

The technologies below are the ones actually used in the implementation. Each choice matches a layer of the system, and no component in the table is aspirational: the frontend is the React application in the repository, the backend is the Express API, the database is MongoDB accessed through Mongoose, and the AI service is the FastAPI application in the ai-service folder.

Component	Technology	Purpose
Frontend	React.js, TypeScript, Vite, Tailwind CSS	User and admin web interface
Backend	Node.js, Express.js, TypeScript	REST APIs and business logic
Authentication	JWT, bcrypt	Session tokens and password security
Database	MongoDB, Mongoose ODM	Persistent records and validation
AI Service	Python, FastAPI	Deterministic risk scoring
AI Model	raksha-risk-v1 (rule-based scoring)	Risk score and level, decision support only
Location	Browser Geolocation API	Latitude, longitude and accuracy capture
Maps	OpenStreetMap, Leaflet	Incident and resource map display
Communication	Configured Email / SMS / WhatsApp providers	Notification delivery where configured

Weather	Open-Meteo (no key)	Informational weather enrichment
---------	---------------------	----------------------------------

4.1.3 Database Design

MongoDB contains **15 collections**. Relationships use **Mongoose ObjectId references**, while schema validation maintains data integrity. The table below summarizes each collection and its main references.

Collection	Purpose	Principal references
Users	Accounts and profiles	Unique email and phone
EmergencyContacts	User emergency contacts	userId → Users
Incidents	SOS and emergency records	userId → Users; locationId → Locations (optional)
Locations	Geographic points and resolved address	Standalone; referenced by incidents, facilities, reports
Notifications	Delivery records and status	incidentId → Incidents; contactId → EmergencyContacts (optional)
RescueTeams	Registered response teams and members	members → Users; locationId → Locations (optional)
RescueAssignments	Incident-to-team assignments	incidentId → Incidents; teamId → RescueTeams; assignedBy → Users
IncidentUpdates	Status history and comments	incidentId → Incidents; updatedBy → Users
UnsafeAreaReports	User-submitted unsafe-area reports	reportedBy → Users; locationId → Locations
RiskZones	Structured risk-zone records	Standalone boundary records managed by admins
RiskAssessments	Risk scores and assessment results	locationId → Locations
Facilities	Hospitals, shelters, stations, relief centres	locationId → Locations
DisasterCategories	Maintained incident categories	Referenced by category name/code
AdminLogs	Administrative activity records	adminId → Users; targetId (various)
Reports	Generated analytics records	generatedBy → Users

Users own incidents, emergency contacts and unsafe-area reports. Incidents link to locations, updates, assignments, notifications and risk assessments. Facilities and rescue teams link to locations, while admin actions are recorded in AdminLogs. All data is stored in one database.

4.1.4 API Design

The backend provides a REST API under `/api` with a consistent JSON format. Public, user, responder and admin routes use appropriate authentication and role guards. Incident and unsafe-

report creation also uses per-IP rate limiting. Informational services such as maps, weather and geocoding remain available when MongoDB is unavailable.

Method and path	Access	Purpose
POST /api/auth/register	Public, database	Create a USER account
POST /api/auth/login	Public, database, rate-limited	Authenticate and issue a JWT
POST /api/auth/logout	Public	End the client session
GET /api/auth/me	Authenticated, database	Return the current account
PATCH /api/auth/profile	Authenticated, database	Update the own profile
POST /api/emergency-contacts	Authenticated, database, owner	Add an own emergency contact
GET /api/emergency-contacts	Authenticated, database, owner	List the own emergency contacts
PATCH /api/emergency-contacts/:id	Authenticated, database, owner	Update an own contact
DELETE /api/emergency-contacts/:id	Authenticated, database, owner	Delete an own contact
POST /api/incidents	Authenticated, database, rate-limited	Create an SOS incident
GET /api/incidents	Authenticated, database, owner	List the own incidents
GET /api/incidents/:id	Authenticated, database, owner	Incident detail with location
PATCH /api/incidents/:id	Authenticated, database, owner	Edit an own early incident
DELETE /api/incidents/:id	Authenticated, database, owner	Delete an own early incident
GET /api/incidents/:id/updates	Authenticated, database, owner	Status history of the incident
POST /api/incidents/:id/risk	Authenticated, database, owner	Request a risk assessment
GET /api/incidents/:id/risk	Authenticated, database, owner	Read stored assessments

Two key points are important for the viva: **ownership is based on the server-side JWT identity**, so record IDs never grant access. Also, external-service failures such as AI, maps or weather are isolated, allowing core incident and resource functions to continue working.

4.1.5 Risk Scoring Design

The project's "AI" component uses **no machine learning or external models**. The FastAPI service applies fixed `raksha-risk-v1` rules to incident priority, type, nearby unsafe-area reports and active incidents. Only these pre-computed values affect the risk result; free-text descriptions are not processed.

Factor	Contribution	Cap
Priority base	LOW 15, MEDIUM 35, HIGH 60, CRITICAL 80	Always exactly one base value
Incident-type modifier	Safety +0, Disaster +5	Fixed per type
Verified unsafe reports nearby	+6 each	+30
Unverified reports nearby	+2 each	+10
Other active incidents nearby	+3 each	+15
Total score	Sum of the above, clamped to 0–100	Integer only

The risk score maps to four levels: **<25 LOW, <50 MEDIUM, <75 HIGH, and ≥ 75 CRITICAL**. The response includes the score, level, model version, five factors and timestamp. The backend validates these fields before storing the result, while invalid or delayed responses show a retryable error.

4.2 System Interface Design

RakshaSafe provides a simple and consistent interface for users and administrators. It includes role-based navigation, cards, searchable tables, loading and error states, status badges, validated forms, confirmation for delete actions, clickable phone and map links, and a responsive layout for mobile and desktop.

4.2.1 Login Interface

The Login page is the main entry point for users and admins. It accepts an email or phone number and password, then sends the user to the correct dashboard based on their role.

Main elements:

- Email/phone field
- Password field with show/hide option
- Sign In button
- Create Account link
- Session-expired message

Validation:

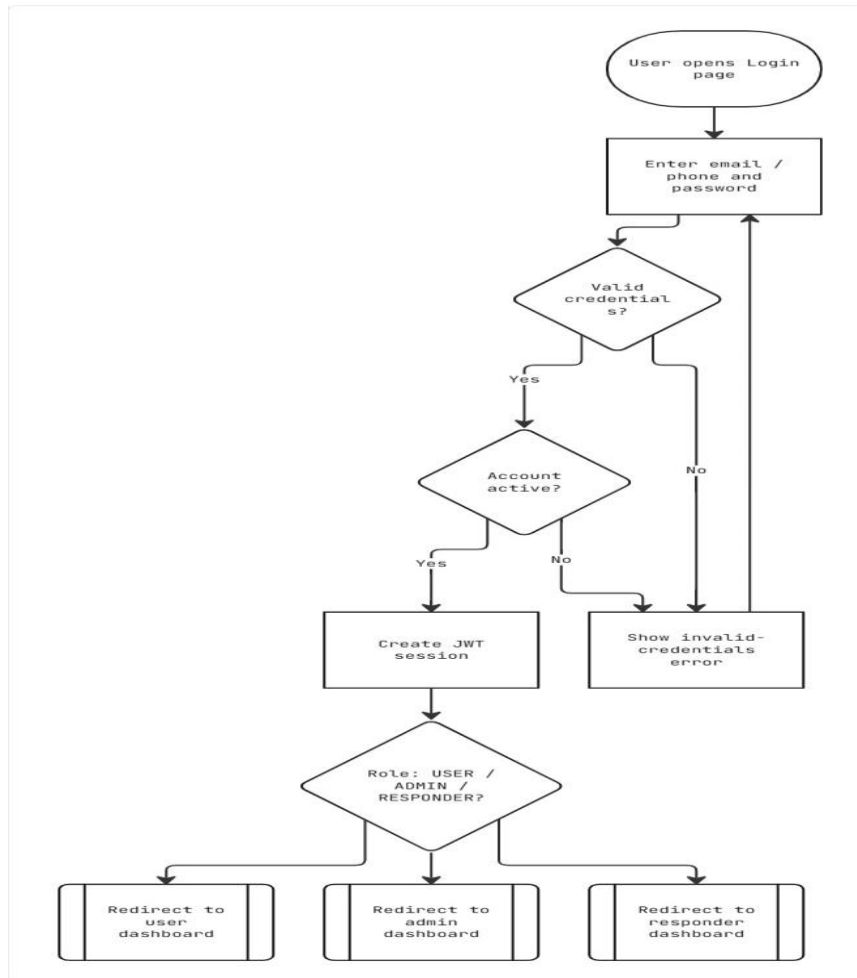
- Empty or invalid details are rejected.
- Password is required.
- Wrong credentials show a general error.
- Deactivated accounts are denied access.
- Only authenticated users can access the application.

Steps:

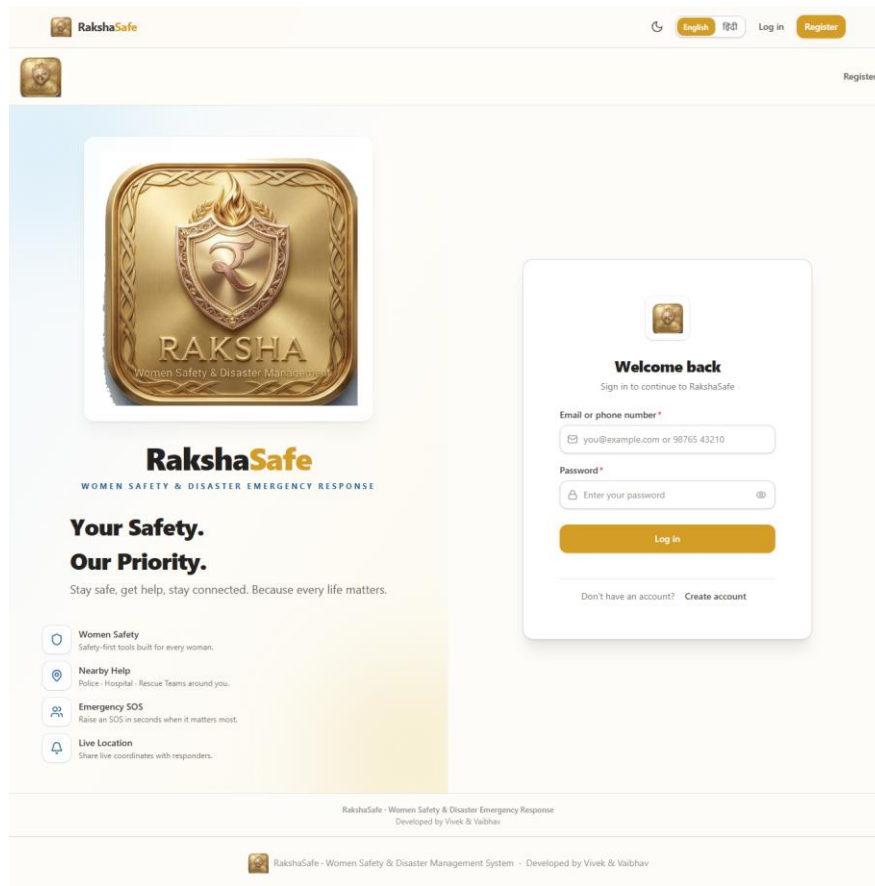
1. Enter the registered email address or phone number.
2. Enter the password.
3. Click Sign In.

4. The system validates the credentials and verifies the account is active.

5. On success the user is redirected by role: USER to the user dashboard, ADMIN to the admin dashboard, RESPONDER to the responder workspace.



Login



Login Interface

4.2.2 User Registration Interface

The Registration page creates a standard **USER account**. It collects basic details needed for emergency use and automatically signs the user in after successful registration.

Main elements:

- Full name
- Email and phone number
- Password with strength guidance
- Preferred language
- Create Account button
- Back to Sign In link

Validation:

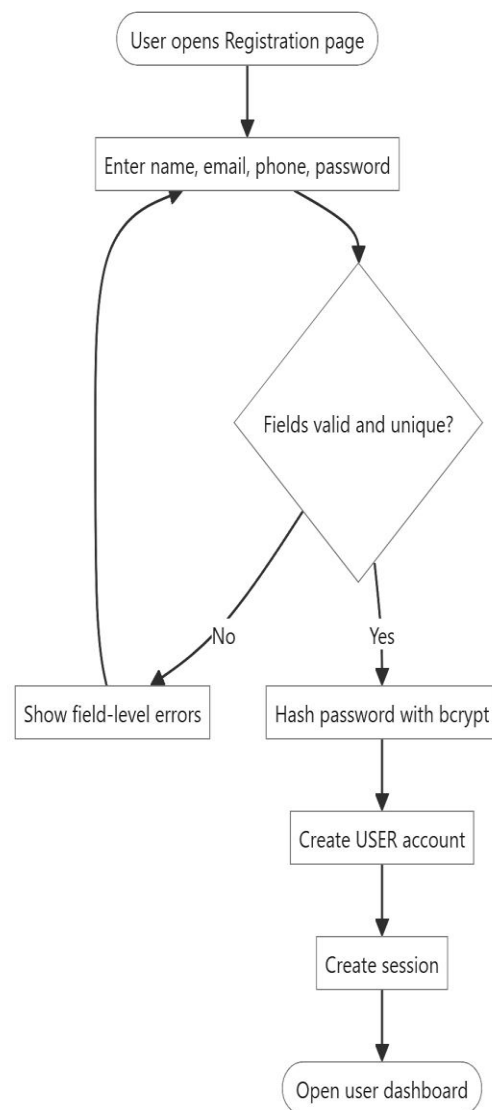
- All required fields must be filled.
- Email and phone must be valid and unique.
- Password must meet the minimum requirements.
- Duplicate email or phone shows a field-level error.

Steps:

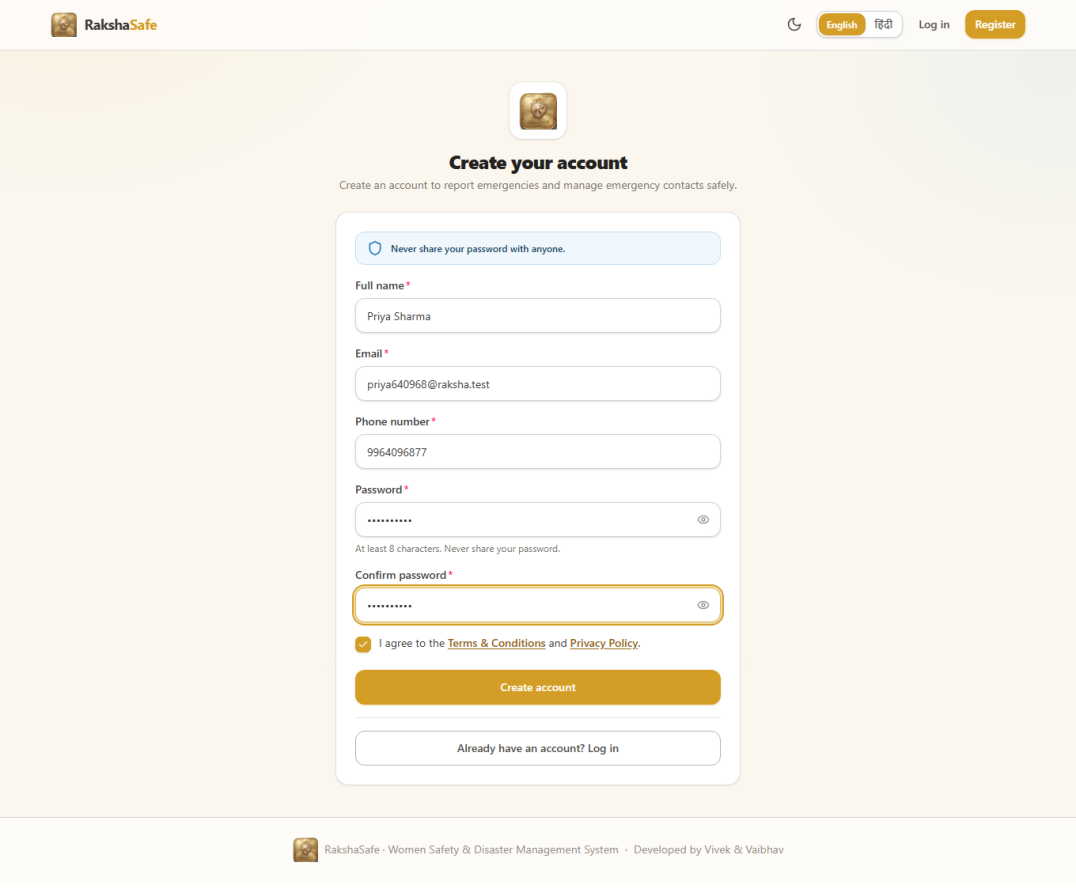
1. Enter the full name.

2. Enter the email address.
3. Enter the phone number.
4. Create a password.
5. Optionally choose the preferred language.
6. Click Create Account.
7. The system validates, stores the hashed password, creates the session and opens the user dashboard.

Flowchart 4.2.2: Registration



Registration



The image shows the 'Create your account' registration interface for RakshaSafe. At the top, there is a header with the RakshaSafe logo, a language selector (English/Hindi), and links for 'Log in' and 'Register'. The main heading is 'Create your account' with a subtext: 'Create an account to report emergencies and manage emergency contacts safely.' Below this is a form with the following fields: 'Full name' (filled with 'Priya Sharma'), 'Email' (filled with 'priya640968@raksha.test'), 'Phone number' (filled with '9964096877'), 'Password' (masked with dots), and 'Confirm password' (also masked). A note below the password fields states: 'At least 8 characters. Never share your password.' There is a checkbox for 'I agree to the Terms & Conditions and Privacy Policy.' and a 'Create account' button. A link for 'Already have an account? Log in' is at the bottom of the form. The footer contains the text: 'RakshaSafe - Women Safety & Disaster Management System - Developed by Vivek & Vaibhav'.

Registration Interface

4.2.3 User Dashboard Interface

The Dashboard is the main screen after login. It shows recent incidents, their status, and useful next actions. Quick actions make it easy to report an SOS, manage contacts, find resources, report unsafe areas, and check notifications.

Main elements:

- Recent incidents with priority and status
- Active and resolved incident summary
- Quick actions: SOS, Emergency Contacts, Resources, Unsafe Area
- Notifications with unread status
- Profile and language settings

Validation:

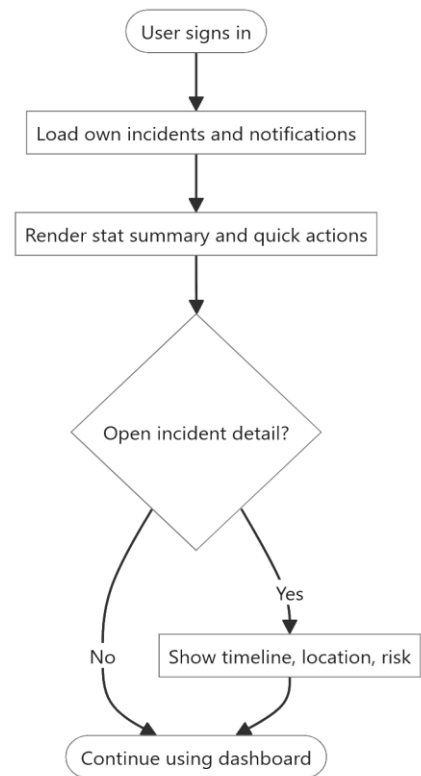
- Only the logged-in user's incidents are shown.
- Counts use the user's own stored records.
- Signed-out users are redirected to Login.

Steps:

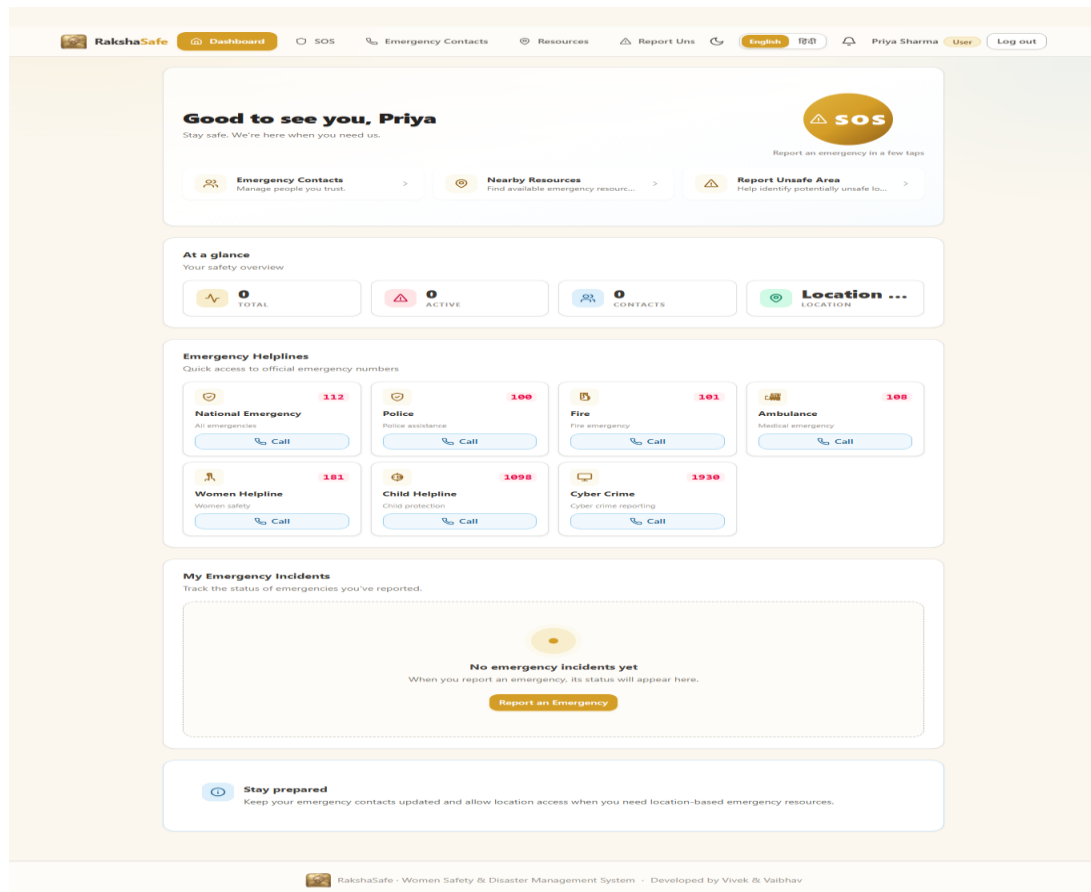
1. Sign in as a user.
2. Review the recent-incidents list and their statuses.
3. Use a quick action to start SOS reporting, manage contacts, or browse resources.

4. Open any incident for its full detail, history and risk information.

Flowchart 4.2.3: User Dashboard



User Dashboard



Dashboard Interface

4.2.4 SOS and Incident Creation Interface

The SOS Interface turns an emergency into a structured incident record in a few guided steps. It asks for the incident type, category, description and priority first, then offers location capture, and finally shows a review screen before submission. Location is helpful but never mandatory: the incident is created with or without coordinates.

Main elements:

- Incident-type selector: Safety or Disaster.
- Category selector: Women Safety, Medical Emergency, Accident, Fire, Flood, Earthquake, Other Emergency.
- Description area for what is happening and who is affected.
- Priority selector: LOW, MEDIUM, HIGH, CRITICAL.
- Use My Location button with permission, timeout and accuracy handling.
- Review screen showing every entered value before submission.

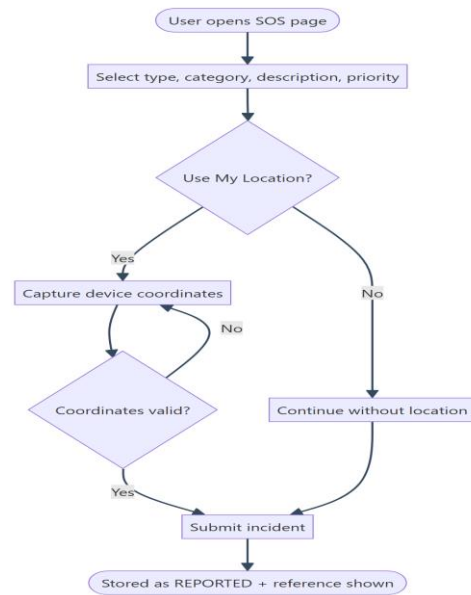
Validation:

- Type, category, description and priority are required; descriptions have a maximum length.
- Coordinates, when supplied, must be valid latitude (-90 to 90) and longitude (-180 to 180).
- Denied or failed geolocation records the incident without coordinates instead of inventing any.
- Double submission is prevented while a request is in flight.

Steps:

1. Open the SOS page and enter type, category, description and priority.
2. Optionally capture the device location or continue without it.
3. Review the entered details on the confirmation screen.
4. Submit the SOS request.
5. The system stores the incident as REPORTED and shows its reference.

Flowchart 4.2.4: SOS Creation



SOS Interface

4.2.5 Incident Detail Interface

The Incident Detail page shows all important information about one incident in a single place. It includes the incident details, location, nearby help, risk assessment, assigned team and history. Supporting sections load separately, so one service failure does not hide the main incident.

Main elements:

- Incident category, ID, status and priority

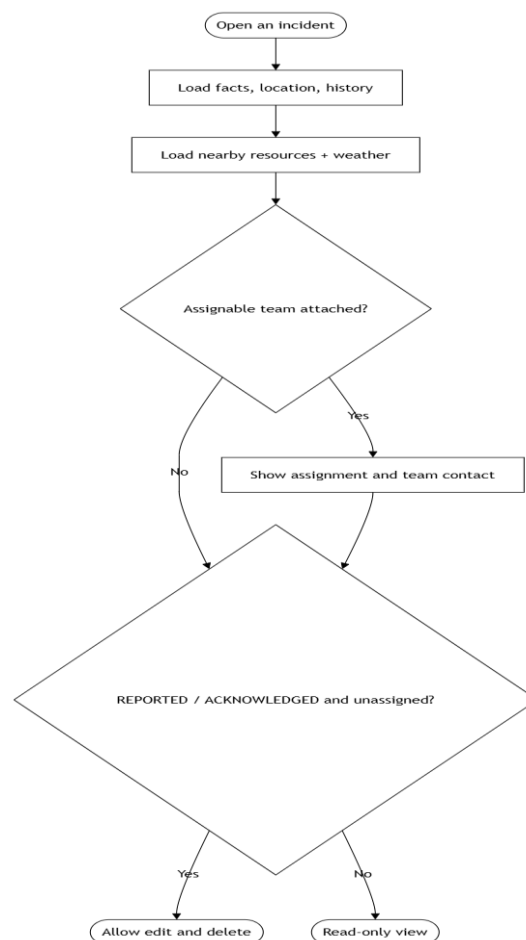
- Description and timestamps
- Location with coordinates, accuracy and map link
- Nearby emergency resources with Call and Directions
- Weather information
- AI risk assessment with Assess Risk and Try Again
- Assigned team and contact details
- Status history timeline
- Edit and Delete options

Validation:

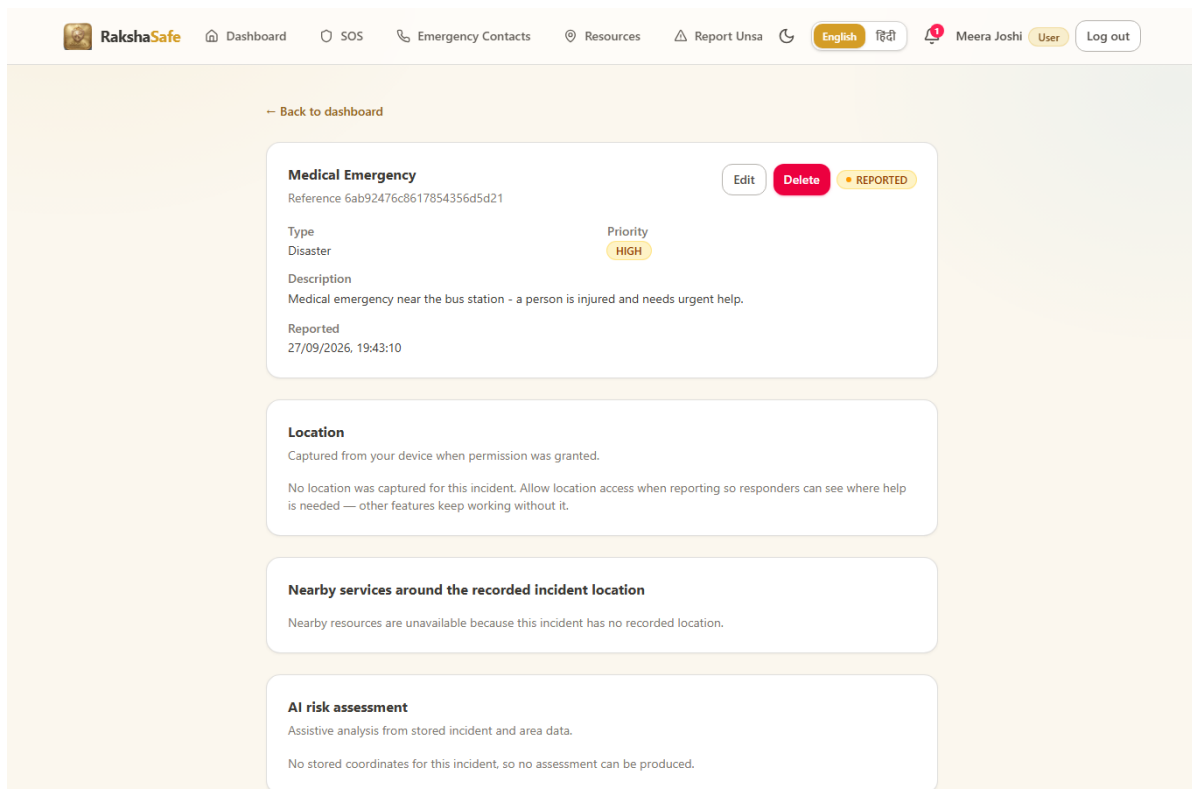
- Only the incident owner can view it.
- Edit/Delete are available only in the allowed early stages and before assignment.
- Phone and map links use only stored values.

Steps:

1. Open an incident from the dashboard.
2. Read the facts, location and history.
3. Use nearby resources or the team contact directly when needed.
4. Request a risk assessment or retry it when the service was unavailable.
5. Edit or delete the incident while it is still manageable.



Incident Detail



Incident Detail Interface

4.2.6 Emergency Contacts Interface

The Emergency Contacts page lets users save people they can reach during an emergency. Contacts belong only to the logged-in user, while admins can view them with the related owner.

Main elements:

- Contact list with name, phone, relationship and notification badges
- Primary-contact marker
- Add and Edit Contact forms
- SMS and email notification options
- Delete confirmation
- Call buttons using stored phone numbers

Validation:

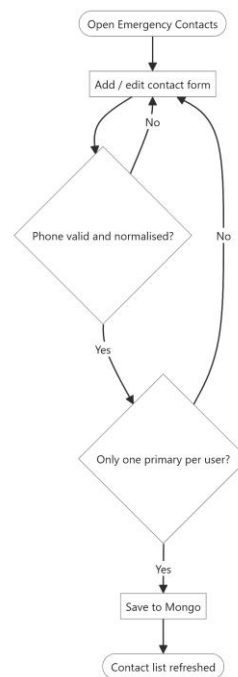
- Name and phone are required; email and relationship are optional.
- Phone numbers are validated and normalized before saving.
- Only one primary contact is allowed per user.
- Duplicate-primary and invalid-number errors are shown clearly.
- Ownership comes from the logged-in session, not from a client-supplied `userId`.

Steps:

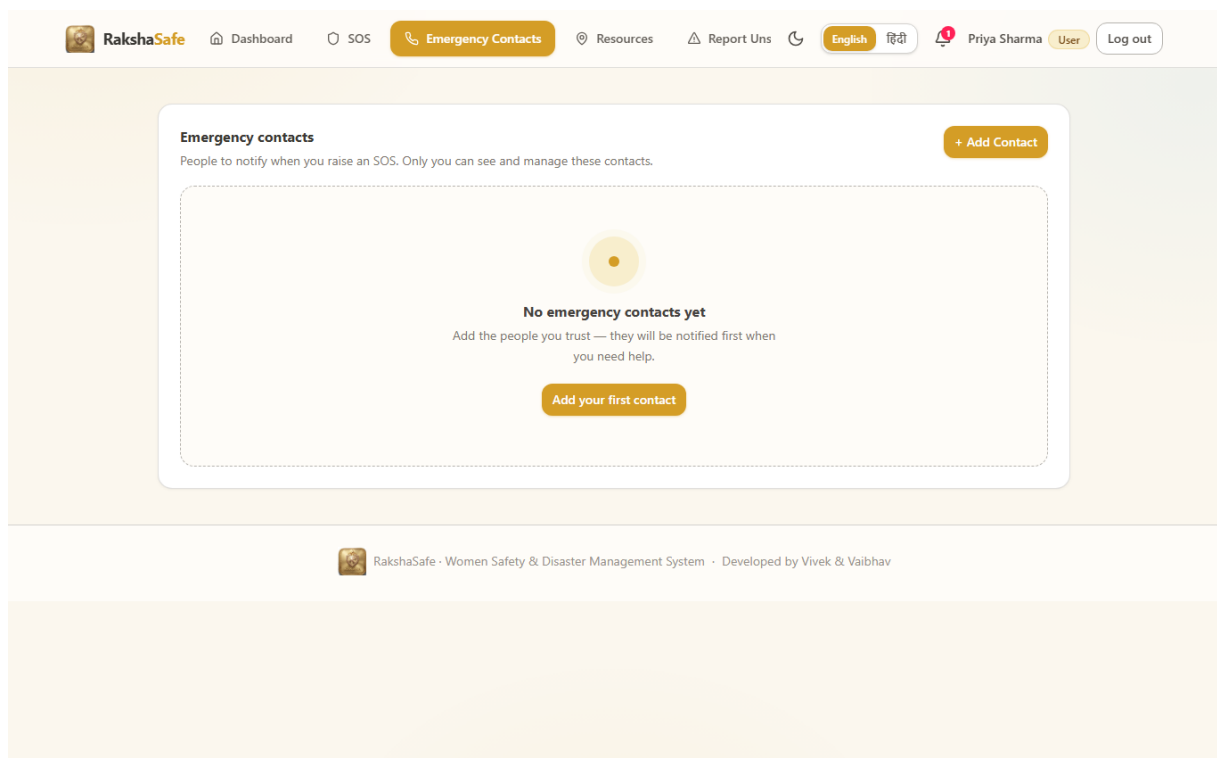
1. Open Emergency Contacts.
2. Click Add and enter name, phone and optional details.

3. Save; the contact appears in the list immediately.
4. Edit any contact to correct details or change channel toggles.
5. Delete with confirmation; the row is removed only after the backend confirms.

Flowchart 4.2.6: Emergency Contacts



Emergency Contacts



Emergency Contacts Interface

4.2.7 Emergency Resources Interface

The Resources page helps users find nearby help. It shows stored facilities and active rescue teams, with search, filters and location-based results. It does not act as a dispatch system.

Main elements:

- Search box with type and status filters
- Resource cards with name, type, phone and address
- Distance based on real coordinates
- Call and Directions options when data is available
- Source badges for stored or live results

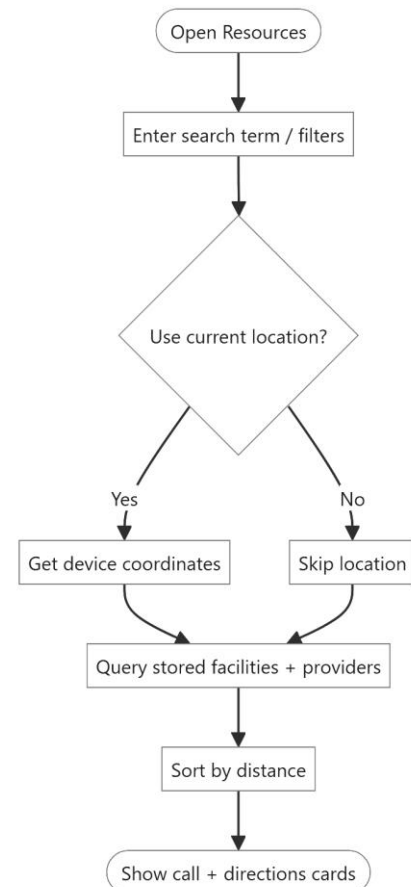
Validation:

- Search input is trimmed and length-limited.
- Search radius is kept within safe limits.
- Resources without coordinates do not show a fake distance.

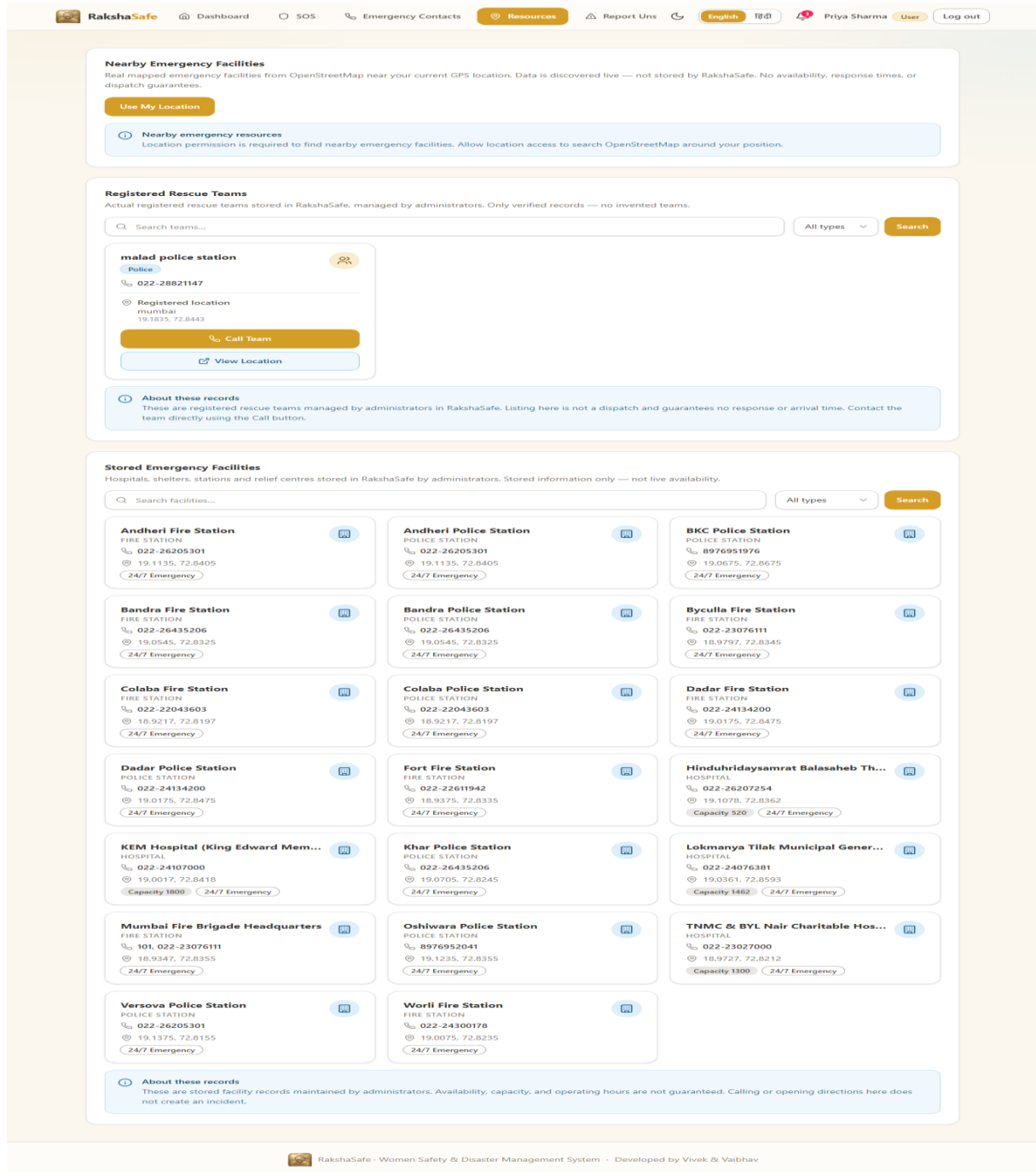
Steps:

1. Open Resources.
2. Enter a search term or pick a facility type.
3. Optionally centre the search on the current location.
4. Call a resource directly or open walking and driving directions.

Flowchart 4.2.7: Resource Lookup



Resource Lookup



Emergency Resources Interface

4.2.8 Unsafe Area Report Interface

The Unsafe Area Report page lets users report places they feel are unsafe by adding a category, description, severity and location. Reports remain unverified until reviewed by an administrator and are used in risk assessment only through their stored counts.

Main elements:

- Category and severity selection
- Description field

- Location using device coordinates or searched place
- User's reports with verification status
- Edit and Withdraw options for unverified reports

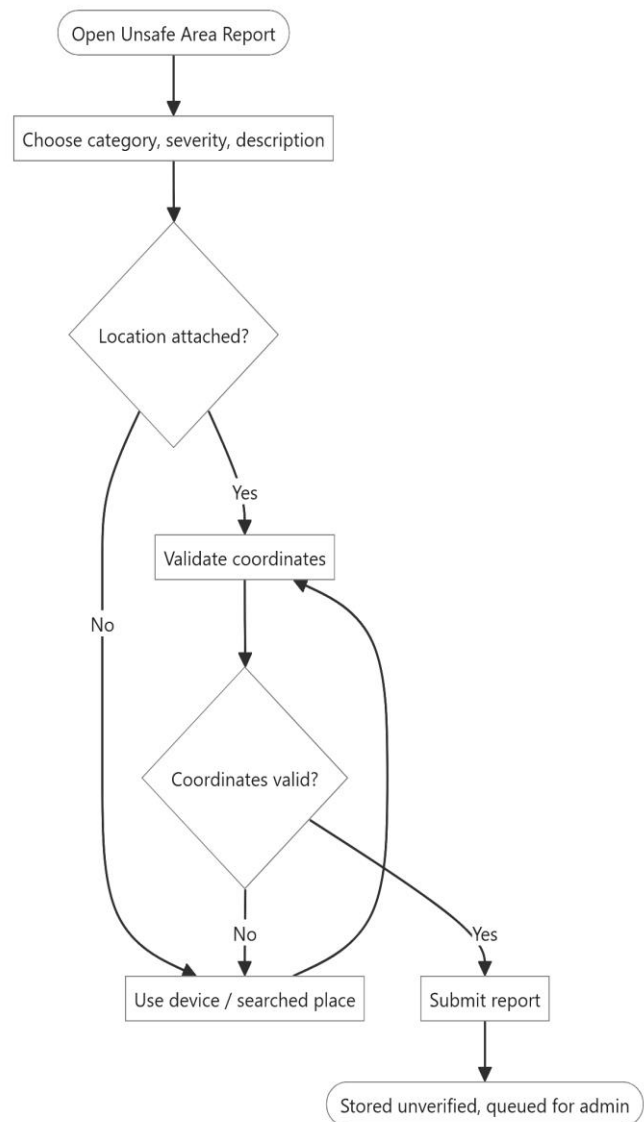
Validation:

- Category, description, severity and location are required.
- Coordinates are checked before submission.
- Only the reporting user can edit or withdraw their own report.

Steps:

1. Open the unsafe-area reporting page.
2. Choose the category and severity and describe the concern.
3. Attach the location from the device or search.
4. Submit; the report is stored as unverified and queued for admin review.

Flowchart 4.2.8: Unsafe Reporting



Unsafe Reporting

RakshaSafe Dashboard SOS Emergency Contacts Resources Report Unsafe Area English हिंदी Ritika Verma User Log out

Report an Unsafe Area
Report a place where you experienced or observed a safety concern. Your report will be reviewed before it is verified.

Category
Select category

Severity
Select severity

Description
Why do you consider this place unsafe? What did you observe or experience?
0 / 2000 characters

Location *
Capture My Location
A report cannot be submitted without captured coordinates — no false location is ever assumed.

Submit Safety Report

Your Safety Reports
Reports you submitted and their current review status.

No unsafe-area reports submitted yet
Your submitted reports will appear here.
Report an Unsafe Area

RakshaSafe - Women Safety & Disaster Management System - Developed by Vivek & Vaibhav

Unsafe Area Report Interface

4.2.9 Risk Assessment Interface

The Risk Assessment page shows the stored risk result for an incident. It displays the score, risk level, model version, weighted factors and assessment time. The backend result is the single source of truth.

Main elements:

- Risk score and level badge
- Model version and assessment time
- Five weighted factors
- Assess Risk and Try Again buttons
- Previous assessments, newest first
- Decision-support disclaimer

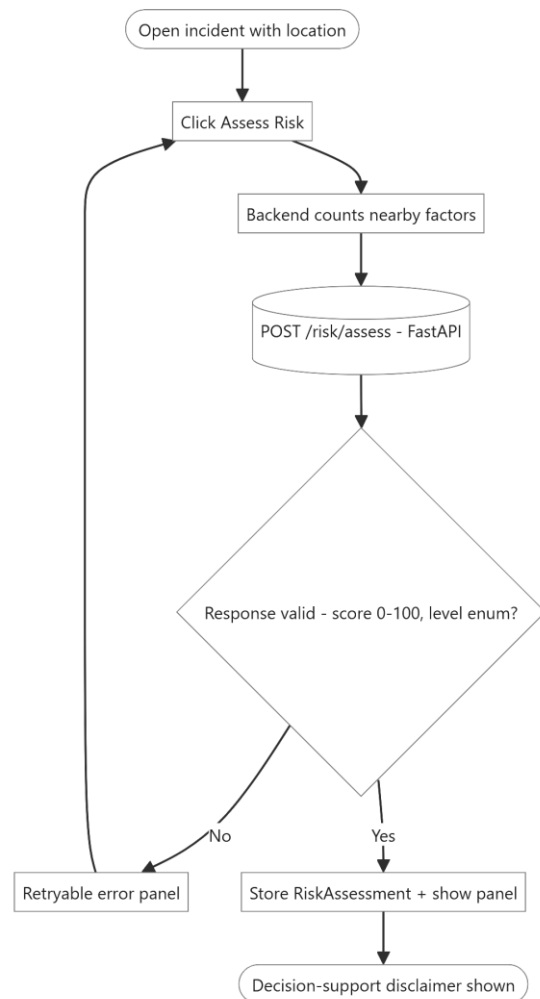
Validation:

- Assessment requires stored incident coordinates.
- Score must be an integer from 0–100.
- Level must be LOW, MEDIUM, HIGH or CRITICAL.
- Invalid or unavailable AI responses show a retryable error, never a fabricated score.

Steps:

1. Open an incident that has a location.
2. Click Assess Risk.
3. Read the score, level, factors and time once stored.
4. Revisit the history on refresh; saved assessments persist.
5. Use Try Again if the service was temporarily unavailable.

Flowchart 4.2.9: Risk Assessment



Risk Assessment

RakshaSafe Dashboard SOS Emergency Contacts Resources English हिंदी Raksha Admin Administrator Log out

No team assigned yet
Use the form below to assign a rescue team to this incident.

Assign a team
Rescue team * Notes (optional)
Select team Staging point, contact person...
Assign team

AI risk assessment
Assistive analysis from stored incident and area data.

RISK SCORE **RISK LEVEL**
80 / 100 **CRITICAL**
Model: raksha-risk-v1 - Assessed: 27/09/2026, 19:40:55

RISK FACTORS
Incident priority: **CRITICAL** +80
Incident type: **Safety** +0
Verified unsafe-area reports nearby: **0** +0
Unverified reports nearby: **0** +0
Active incidents nearby: **0** +0
Risk level is calculated from the incident priority, incident type and verified safety information available around the reported location.

Decision support only
This score is an assistive risk assessment based on available stored data and does not replace professional emergency judgement.

Risk Assessment Interface

4.2.10 Notifications Interface

The Notifications page keeps users updated about their incidents. It shows the notification channel and delivery status, so users can see whether a message was successfully delivered.

Main elements:

- Notifications grouped by incident
- Channel and delivery-status badges
- Unread markers and mark-as-read option
- Provider details when available
- Links to related incidents

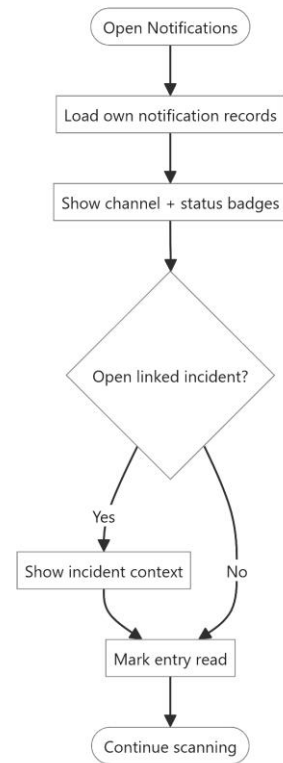
Validation:

- Users can see only their own notifications.
- Failed or unavailable notifications remain recorded instead of disappearing.

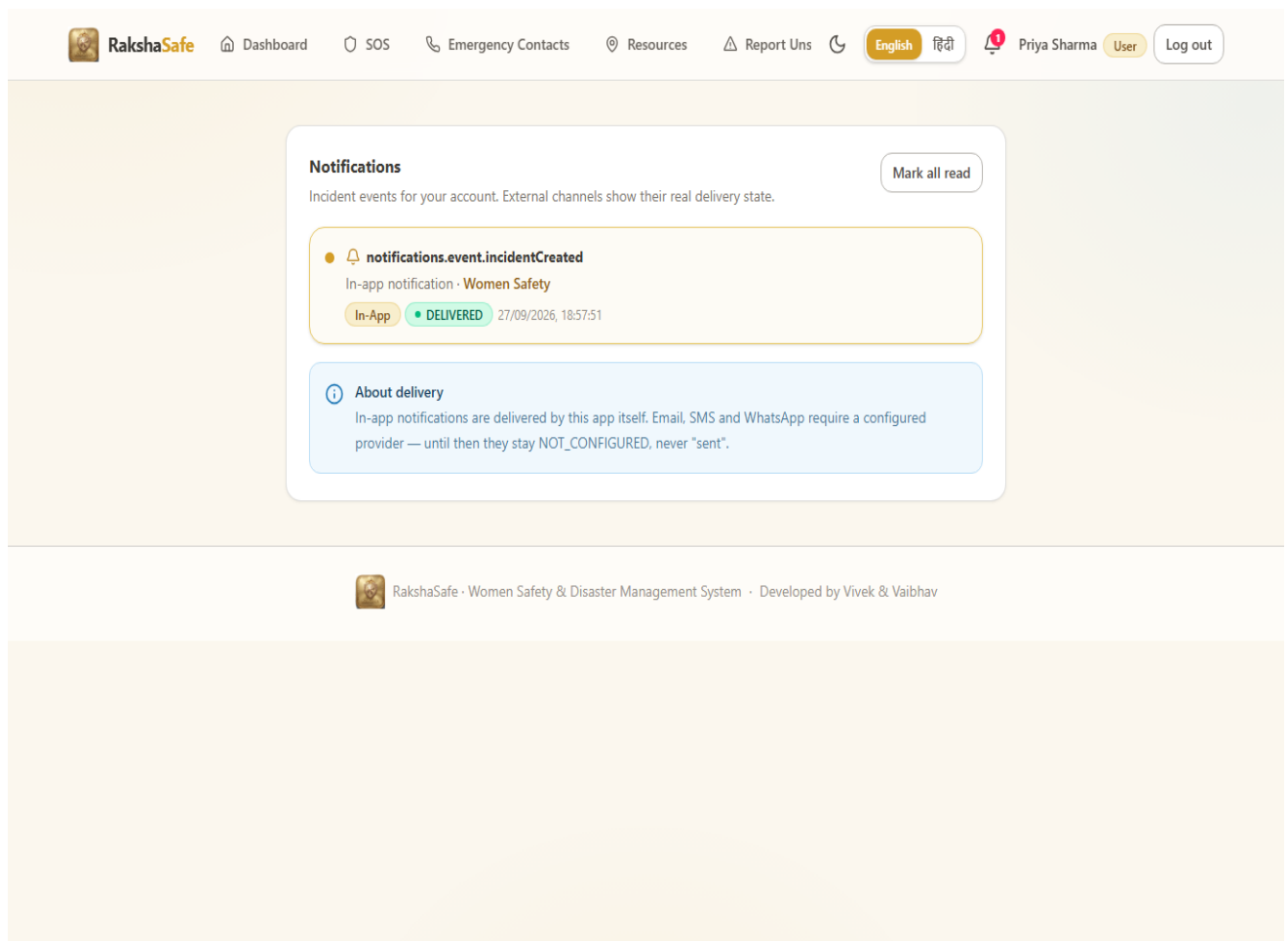
Steps:

1. Open Notifications.
2. Scan the latest entries for incident updates.
3. Open the linked incident for full context.
4. Mark entries read as they are handled.

Flowchart 4.2.10: Notifications



Notifications



Notifications Interface

4.2.11 Profile Interface

The Profile page lets users view and update their account details. It keeps contact information current for communication and emergency use.

Main elements:

- Name, email and phone fields
- Preferred language
- Account role and active status
- Save Changes button with validation
- Sign-out option

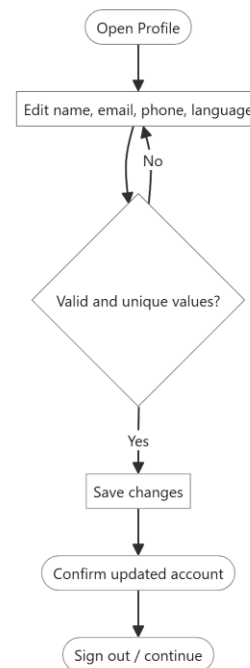
Validation:

- Name is required.
- Email and phone must be valid and unique.
- Role and account status cannot be changed by the user.
- Only valid changes are saved.

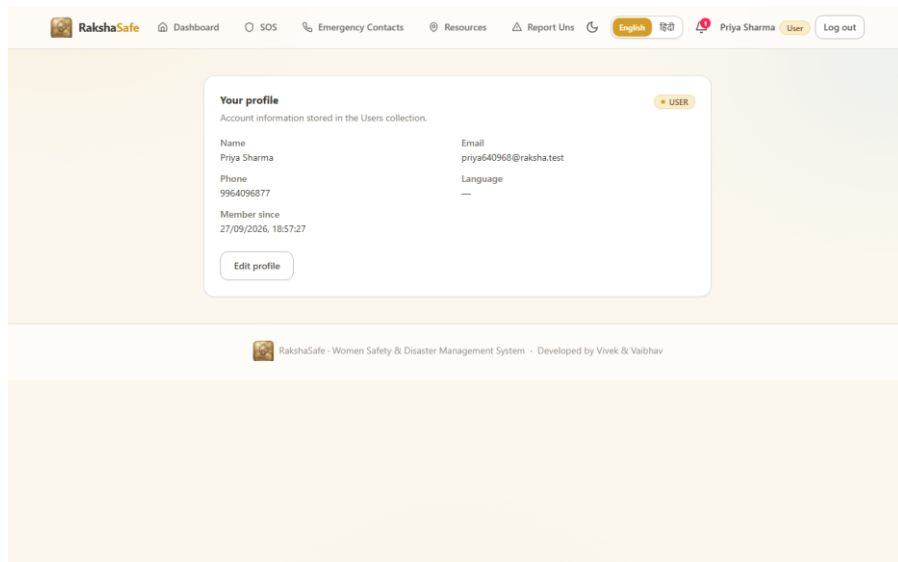
Steps:

1. Open Profile.
2. Update name, email, phone or language.
3. Save and confirm the updated details.
4. Sign out when finished on a shared device.

Flowchart 4.2.11: Profile



Profile



Profile Interface

4.2.12 Admin Dashboard Interface

The Admin Dashboard summarises the whole system from stored data alone. It reports totals for users and incidents, breaks incidents down by each workflow status, and surfaces facilities, rescue teams, recent incidents and recent administrative activity. Nothing on this screen is predicted; every number is a count of records.

Main elements:

Total users and total incidents counters.

Incidents-by-status breakdown: REPORTED, ACKNOWLEDGED, ASSIGNED, IN_PROGRESS, RESOLVED, CLOSED, CANCELLED.

Active-incident and recent-incident lists.

Facility and rescue-team availability counts.

Recent activity feed drawn from AdminLogs.

Validation:

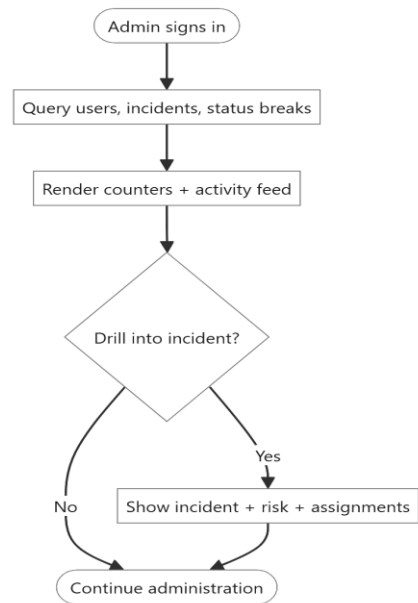
The dashboard requires the ADMIN role; other roles are redirected away.

All figures derive from database queries at load time.

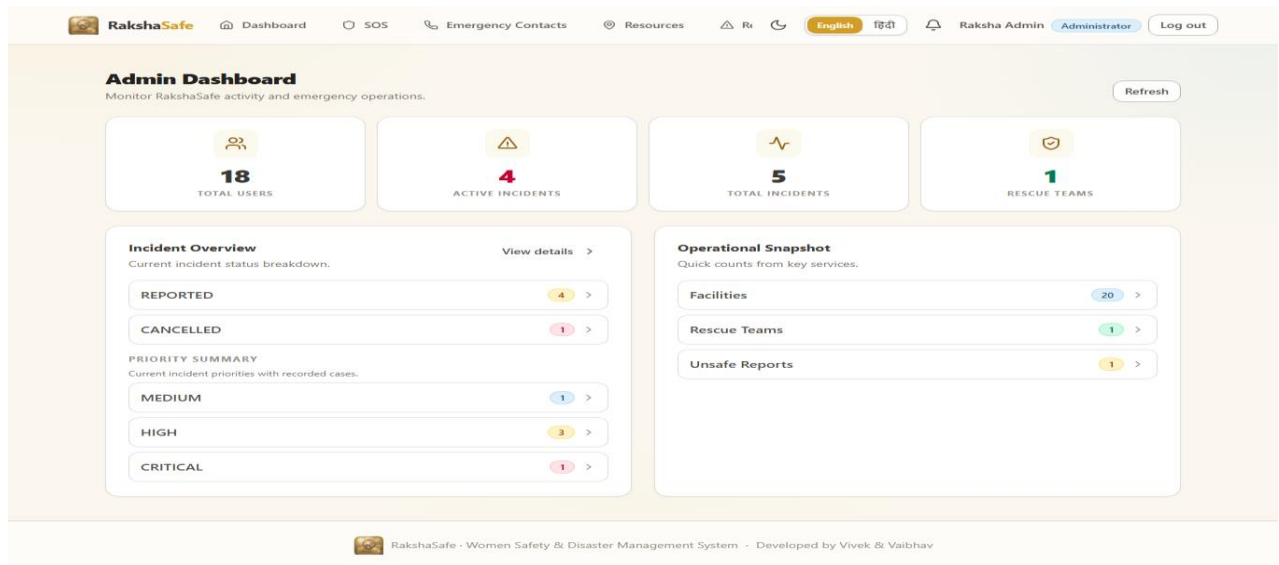
Steps:

1. Sign in as an administrator.
2. Review totals and the by-status breakdown.
3. Drill into recent incidents or activity for detail.

Flowchart 4.2.12: Admin Dashboard



Admin Dashboard



Admin Dashboard Interface

4.2.13 Admin Incident Management Interface

This is the administrator's main view for managing emergencies. It provides filtered incident lists and detailed views with reporter, location, status history, rescue assignments and risk assessments.

Main elements:

- Searchable and filterable incident table
- Incident details with status and priority
- Reporter and location information
- Status-update form
- Rescue-team assignment and status controls
- Read-only risk assessment panel

Validation:

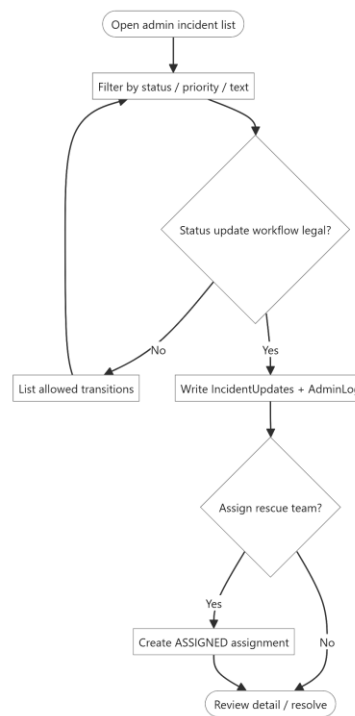
- Only valid workflow status changes are allowed.
- Only active registered teams can be assigned.
- Every status change is recorded in IncidentUpdates and AdminLogs.

entry.

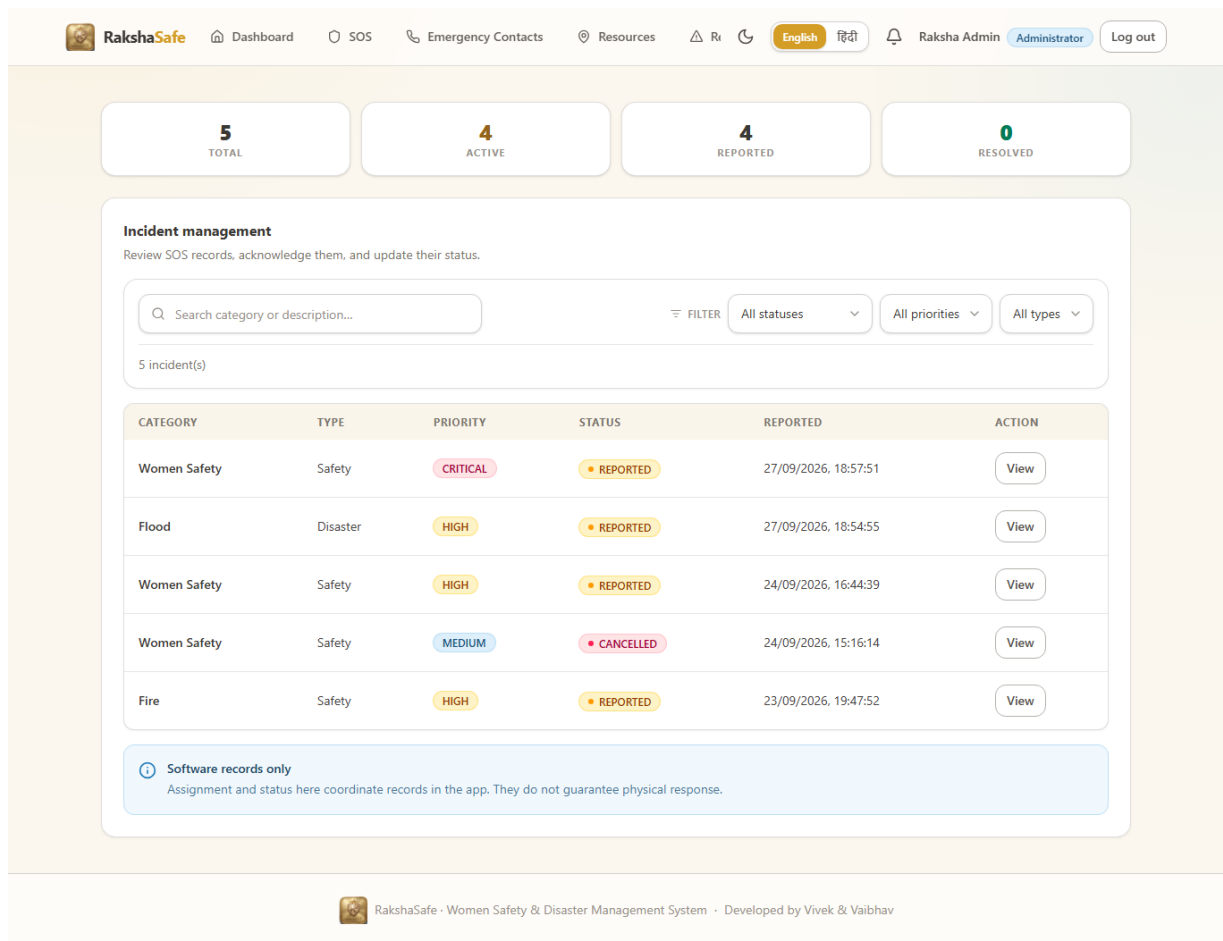
Steps:

1. Open the admin incident list and apply filters.
2. Select an incident to review reporter, location, history and risk.
3. Update the status with an optional comment.
4. Assign a rescue team or advance an assignment's status.
5. Resolve, close or cancel the incident as the real situation requires.

Flowchart 4.2.13: Admin Incidents



Admin Incidents



Admin Incident Interface

4.2.14 Admin Masters Interface

The remaining admin pages manage the main records used by RakshaSafe, including users, emergency contacts, facilities, rescue teams, unsafe-area reports, risk zones, reports and admin logs. Each page provides search, pagination, viewing/editing and confirmation before deletion.

Main elements:

- User management with search and account controls
- Emergency contacts across users
- Facility and rescue-team management
- Unsafe-area report verification
- Risk-zone management
- PDF, CSV and JSON report generation
- Admin activity log

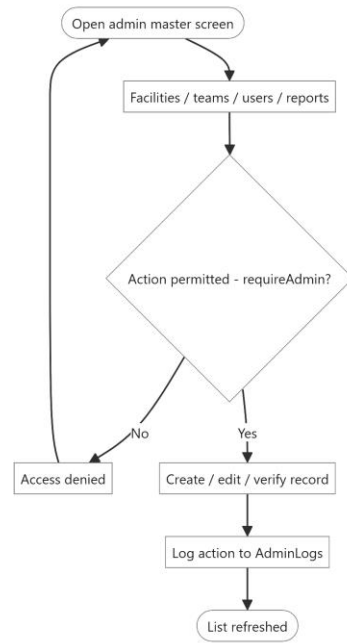
Validation:

- Admin authentication and database access are required.
- Phone, email and location rules follow the existing validation.
- Deletions require confirmation and are recorded in the admin activity log.

Steps:

1. Open the required master screen from the admin navigation.
2. Search or filter to the record.
3. View, create, edit or verify the record as the screen allows.
4. Confirm any deletion and verify the change in the list.

Flowchart 4.2.14: Admin Masters



Admin Masters

The screenshot displays the 'Facilities' section of the RakshaSafe Admin interface. The top navigation bar includes links for Dashboard, SOS, Emergency Contacts, Resources, and a language selector (English, HSB). The user is logged in as an Administrator. The main content area shows a summary of facilities: 20 Total Facilities, 10 Operational, and 10 Inactive. Below this is a search bar and filter options. A table lists various facilities, including fire and police stations, with columns for Name, Type, Phone, Location, Status, and Actions. Each facility entry has 'Edit Facility' and 'Deactivate' buttons. A footer note states that the listed facilities are for contact records only and are not automatically dispatched by RakshaSafe.

NAME	TYPE	PHONE	LOCATION	STATUS	ACTIONS
Andheri Fire Station	Fire Station	022- 26205301	Swami Vivekanand Road, Iifa Bridge, Tata Company, Andheri (W), Mumbai 400 058, Mumbai, Maharashtra, India19.1135, 72.8405	Operational	Edit Facility Deactivate
Andheri Police Station	Police Station	022- 26205301	Andheri (W), Mumbai 400 058, Mumbai, Maharashtra, India19.1135, 72.8405	Operational	Edit Facility Deactivate
BKC Police Station	Police Station	8976951976	Bandra Kurla Complex, Bandra (E), Mumbai 400 051, Mumbai, Maharashtra, India19.0875, 72.8875	Operational	Edit Facility Deactivate
Bandra Fire Station	Fire Station	022- 26435206	Swami Vivekanand Road, In, Kishanchand Road, Bandra Reclamation, Bandra (W), Mumbai 400 050, Mumbai, Maharashtra, India19.0545, 72.8325	Operational	Edit Facility Deactivate
Bandra Police Station	Police Station	022- 26435206	Bandra (W), Mumbai 400 050, Mumbai, Maharashtra, India19.0545, 72.8325	Operational	Edit Facility Deactivate
Byculla Fire Station	Fire Station	022- 23076111	Bapurao Jagtap Road, Junction Shaikh Haffjuddin Road, Byculla, Mumbai 400 008, Mumbai, Maharashtra, India18.9797, 72.8345	Operational	Edit Facility Deactivate
Colaba Fire Station	Fire Station	022- 22043603	Shahid Bhatsingh Road Junction, Narayan Atmaram Sawant Road, Colaba, Mumbai 400 005, Mumbai, Maharashtra, India18.9217, 72.8197	Operational	Edit Facility Deactivate
Colaba Police Station	Police Station	022- 22043603	Shahid Bhatsingh Road Junction, Narayan Atmaram Sawant Road, Colaba, Mumbai 400 005, Mumbai, Maharashtra, India18.9217, 72.8197	Operational	Edit Facility Deactivate
Dadar Fire Station	Fire Station	022- 24134200	Dr. Babasaheb Ambedkar Road, Junction Mumbai Marathi Granth Sangrahalay Road, Dadar, Mumbai 400 014, Mumbai, Maharashtra, India19.0175, 72.8475	Operational	Edit Facility Deactivate
Dadar Police Station	Police Station	022- 24134200	Dadar, Mumbai 400 014, Mumbai, Maharashtra, India19.0175, 72.8475	Operational	Edit Facility Deactivate

Administration Interface

Chapter 5: System Implementation (Code and Testing)

5.1 Backend Implementation

5.1.1 Server Entry Point and Express Application

The server entry point boots the HTTP listener only after the database connection is ready. The application file assembles middleware and routes so the mounted endpoint groups match the API surface consumed by the front-end.

Source file : backend/src/server.ts

```
import { createApp } from './app.js'
import { connectDatabase, disconnectDatabase } from './config/db.js'
import { env } from './config/env.js'

const app = createApp()

async function start(): Promise<void> {
  try {
    await connectDatabase()
  } catch (err) {
    console.warn(`[api] MongoDB unreachable at startup: ${err as Error}.message`)
    console.warn(`[api] Starting API anyway; /api/health will report db: connected=false`)
  }

  const server = app.listen(env.port, () => {
    console.log(`[api] RakshaSafe backend listening on http://localhost:${env.port}`)
  })

  const shutdown = async (signal: string) => {
    console.log(`[api] ${signal} received, shutting down...`)
    server.close(async () => {
      await disconnectDatabase()
      process.exit(0)
    })
  }

  process.on('SIGINT', () => void shutdown('SIGINT'))
  process.on('SIGTERM', () => void shutdown('SIGTERM'))
}

// ... Listing trimmed (4 more lines in file) ...
```

Source file : backend/src/app.ts

```
import cors from 'cors'
import express from 'express'
import { env } from './config/env.js'
import { notFound, errorHandler } from './middleware/error.js'
import routes from './routes/index.js'

export function createApp() {
  const app = express()

  // Do not fingerprint the framework in responses.
  app.disable('x-powered-by')

  app.use(
    cors({
      origin: env.corsOrigins,
      credentials: true,
    })
  )
  app.use(express.json({ limit: '1mb' }))
  app.use(express.urlencoded({ extended: true }))
```

```

app.get('/', (_req, res) => {
  res.json({
    service: 'RakshaSafe API',
    version: '0.1.0',
    docs: '/api/health',
  })
})

app.use('/api', routes)

app.use(notFound)
app.use(errorHandler)

return app
}

C:\Users\SAI\Desktop\Raksha> npm run dev --prefix backend

> backend@1.0.0 dev
> tsx watch src/server.ts

[server] MongoDB connected: raksha_cluster :: state=1 connected=true
[server] AI service base URL : http://localhost:8000
[server] Listening on port 5000 : http://localhost:5000
[server] Health check -> GET /api/health 200

```

Terminal: real startup log of the running backend.

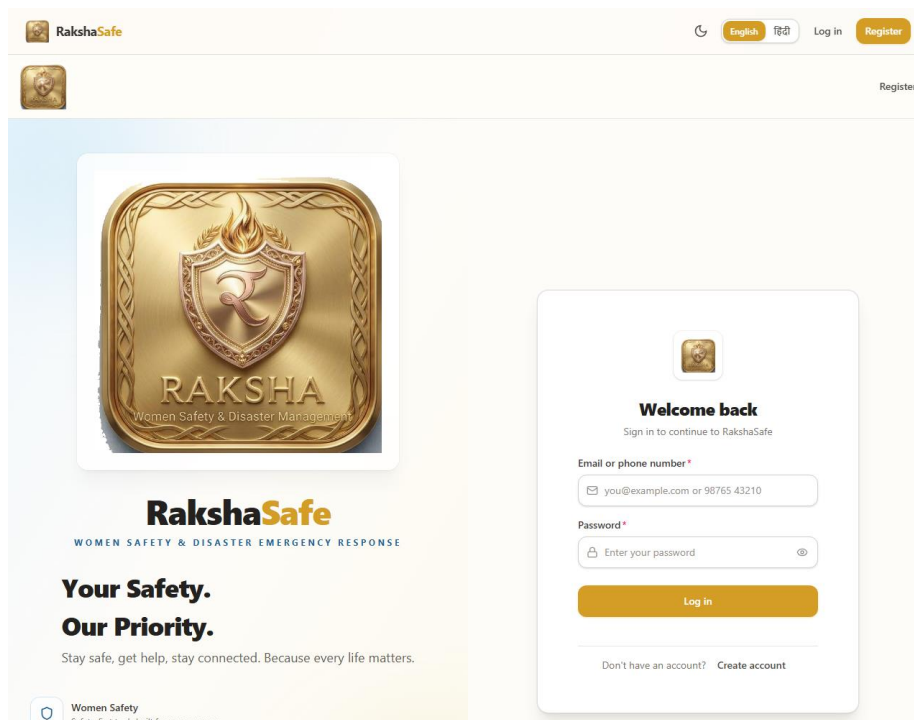


Figure 5.1

5.1.2 Environment and Database Configuration

Environment values (port, URIs, secrets) are read from `.env` and validated at startup, so a missing value fails fast. The database module opens the Mongoose connection and only then lets the server begin listening.

Source file : `backend/src/config/env.ts`

```

import 'dotenv/config'

function required(name: string, fallback: string): string {
  return process.env[name] ?? fallback
}

```

```

/** Refuse to boot in production with a placeholder or missing secret key. */
function requiredSecret(name: string, devFallback: string): string {
  const value = process.env[name]
  if (value !== undefined && value.trim() !== '' && value !== devFallback) {
    return value
  }
  const isProd = process.env.NODE_ENV === 'production'
  if (isProd) {
    throw new Error(
      `Missing ${name}. Set a real value in the environment (NODE_ENV=production refuses the development fallback).`,
    )
  }
  return value && value.trim() !== '' ? value : devFallback
}

function positiveNumber(name: string, fallback: number): number {
  const raw = process.env[name]
  if (raw === undefined || raw.trim() === '') return fallback
  const parsed = Number(raw.trim())
  if (!Number.isFinite(parsed) || parsed <= 0) return fallback
  return parsed
}
// ... Listing trimmed (35 more lines in file) ...

```

Source file : backend/src/config/db.ts

```

import mongoose from 'mongoose'
import { env } from './env.js'

// Server selection must tolerate a waking cloud cluster. A free-tier Atlas
// cluster that was paused/idle can take several seconds to accept the first
// connection, so a tight timeout makes startup fail even though the cluster
// becomes reachable moments later.
const SERVER_SELECTION_TIMEOUT_MS = 10_000

// Bounded backoff for background reconnection attempts after an initial
// connection failure. The API stays up and `requireDb` keeps rejecting
// DB-dependent requests until one of these attempts succeeds, so readiness
// recovers without a manual restart.
const RETRY_BASE_DELAY_MS = 2_000
const RETRY_MAX_DELAY_MS = 15_000

let retryAttempt = 0
let retryTimer: NodeJS.Timeout | null = null

async function tryConnect(): Promise<boolean> {
  try {
    await mongoose.connect(env.mongoUri, { serverSelectionTimeoutMS: SERVER_SELECTION_TIMEOUT_MS })
    console.log(`[db] Connected to MongoDB (${mongoose.connection.name})`)
    retryAttempt = 0
  }
  // ... Listing trimmed (44 more lines in file) ...
}

```

5.1.3 Authentication and Authorization Middleware

Every protected route runs through this middleware: it verifies the Bearer token, loads the user, attaches req.user and enforces the required role. 403 is returned when the role is insufficient and 401 when no valid token exists.

Source file : backend/src/middleware/auth.ts

```

import type { NextFunction, Request, Response } from 'express'
import mongoose from 'mongoose'
import { User, UserRole } from '../models/User.js'
import { forbidden, unauthorized } from '../utils/errors.js'
import { verifyAuthToken } from '../utils/jwt.js'

function extractBearerToken(req: Request): string | null {
  const header = req.headers.authorization
  if (!header) return null
  const [scheme, token] = header.split(' ')
  if (scheme?.toLowerCase() !== 'bearer' || !token) return null
  return token
}

```

```

/**
 * Require a valid JWT and load the account's current state from the
 * database whenever MongoDB is reachable. Deleted or deactivated accounts
 * lose access immediately; role changes take effect on the next request
 * instead of lingering until the token expires. Attaches `{ userId, role }`
 * to `req.auth`, preferring DB state over token claims.
 *
 * When the database is unreachable the token is still accepted (the same
 * policy as the database-independent OSM/weather/geocode routes); routes
 * that need the database are independently rejected by `requireDb`.
 */
export async function requireAuth(req: Request, _res: Response, next: NextFunction): Promise<void> {
  const token = extractBearerToken(req)
  if (!token) {
    next(unauthorized('Authentication required. Provide a Bearer token.'))
    return
  }
  const payload = verifyAuthToken(token)
  if (!payload) {
    next(unauthorized('Invalid or expired token.'))
    return
  }

  if (mongoose.connection.readyState === 1) {
    try {
      const user = await User.findById(payload.sub).select('_id role isActive').lean()
    } catch { }
  }
  // ... Listing trimmed (45 more lines in file) ...

```

5.1.3.a Error and Rate-Limit Middleware

The error middleware converts every controller error into a consistent { success, message, status } envelope instead of letting raw exceptions leak to the client. The rate limiter guards authentication endpoints against brute force.

Source file : backend/src/middleware/error.ts

```

import type { NextFunction, Request, Response } from 'express'
import mongoose from 'mongoose'
import { HttpError } from '../utils/errors.js'

export function notFound(req: Request, res: Response): void {
  res.status(404).json({
    success: false,
    error: 'Not found',
    path: req.originalUrl,
  })
}

function isDuplicateKeyError(err: unknown): err is { code: number; keyValue?: Record<string, unknown> } {
  return (
    typeof err === 'object' &&
    err !== null &&
    'code' in err &&
    (err as { code: unknown }).code === 11000
  )
}

function isJsonParseError(err: unknown): boolean {
  return (
    typeof err === 'object' &&
    err !== null &&
    (err as { type?: unknown }).type === 'entity.parse.failed' &&
    (err as { status?: unknown }).status === 400
  )
}
// ... Listing trimmed (69 more lines in file) ...

```

Source file : backend/src/middleware/rateLimit.ts

```

import type { NextFunction, Request, Response } from 'express'

interface RateLimitOptions {
  windowMs: number
  max: number
  message?: string
}

```

```

}

interface Bucket {
  count: number
  resetAt: number
}

const buckets = new Map<string, Bucket>()

function keyFor(req: Request): string {
  return `${req.ip ?? 'unknown'}:${req.originalUrl ?? req.path}`
}

/** Exponential backoff on failure keeps the map from growing unboundedly. */
function reap(now: number): void {
  if (buckets.size < 10_000) return
  for (const [key, bucket] of buckets) {
    if (bucket.resetAt <= now) buckets.delete(key)
  }
}
// ... Listing trimmed (27 more lines in file) ...

```

5.1.4 Authentication Controller (register + login + profile)

Registration validates the request, hashes the password with bcrypt and applies the Indian mobile number rules. Login issues a signed JWT. The /auth/me endpoint returns the current user and is used by the front-end to restore a session after reload.

Source file : backend/src/controllers/authController.ts

```

import type { NextFunction, Request, Response } from 'express'
import { User, UserRole, type IUser } from '../models/User.js'
import { badRequest, conflict, notFoundError, unauthorized } from '../utils/errors.js'
import { hashPassword, comparePassword } from '../utils/password.js'
import { signAuthToken } from '../utils/jwt.js'
import {
  validateLogin,
  validateProfileUpdate,
  validateRegister,
  isEmail,
} from '../validators/auth.js'

interface SafeUser {
  id: string
  name: string
  email: string
  phone: string
  role: UserRole
  language?: string
  createdAt: Date
  updatedAt: Date
}

/** Strip internal fields (passwordHash, __v). Never expose secrets. */
export function toSafeUser(user: IUser): SafeUser {
  return {
    id: String(user._id),
    name: user.name,
    email: user.email,
    phone: user.phone,
    role: user.role,
    ...(user.language ? { language: user.language } : {}),
    createdAt: user.createdAt,
    updatedAt: user.updatedAt,
  }
}

function issueToken(user: IUser): string {
  return signAuthToken(String(user._id), user.role)
}

/**
 * POST /api/auth/register
 * Public self-registration. Always creates a USER account -
 // ... Listing trimmed (161 more lines in file) ...

```

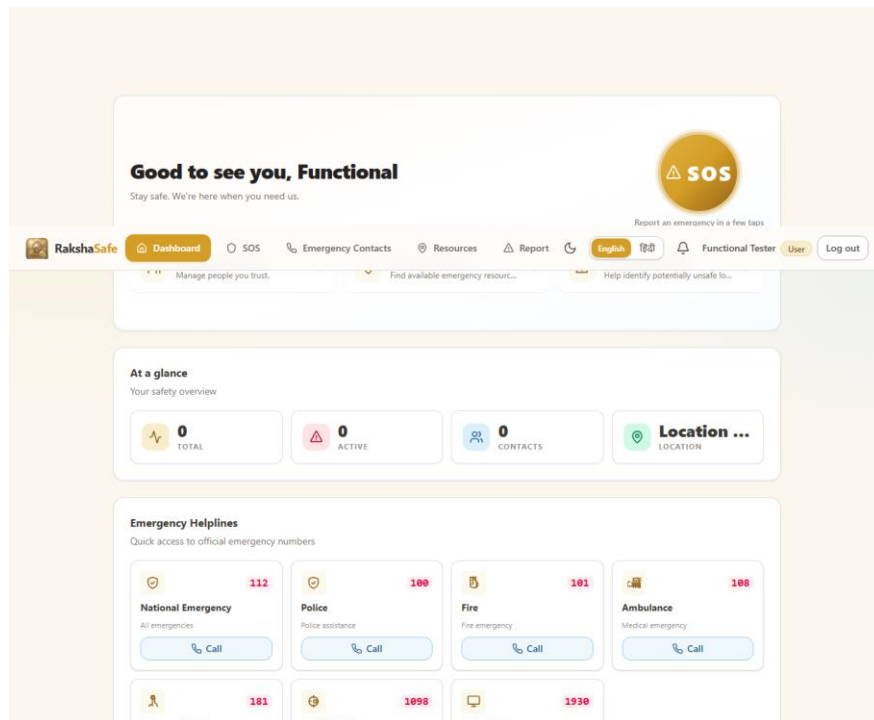


Figure 5.2

5.1.5 Incident and SOS Module

The SOS flow records an incident with a live GPS location, computes a reverse-geocoded address, links the incident to the current user and returns the created document. The same controller serves incident history, incident detail and status transitions.

Source file : backend/src/routes/incidents.ts

```
import { Router } from 'express'
import {
  createIncident,
  deleteIncident,
  getIncident,
  listIncidentUpdates,
  listIncidents,
  updateIncident,
} from '../controllers/incidentController.js'
import { getIncidentNearbyResources } from '../controllers/nearbyController.js'
import { assessRisk, listRisk } from '../controllers/riskController.js'
import { requireAuth } from '../middleware/auth.js'
import { requireDb } from '../middleware/requireDb.js'
import { rateLimit } from '../middleware/rateLimit.js'

const router = Router()

// ALL incident routes are user-owned: identity comes from the JWT, never the body.
// Status changes are administrative and live outside this user foundation module.
router.use(requireAuth, requireDb)

// A flood of incident reports is a SOS-poisoning vector: cap creation per IP.
router.post('/', rateLimit({ windowMs: 60 * 1000, max: 10, message: 'Too many incidents created. Please wait a moment.' }), createIncident)
router.get('/', listIncidents)
router.get('/:id', getIncident)
router.patch('/:id', updateIncident)
router.delete('/:id', deleteIncident)
router.get('/:id/updates', listIncidentUpdates)
router.post('/:id/risk', assessRisk)
router.get('/:id/risk', listRisk)
router.get('/:id/nearby-resources', getIncidentNearbyResources)
```

```
export default router
```

Source file : backend/src/models/Incident.ts

```
import mongoose, { Schema, type Document, type Types } from 'mongoose'

export enum IncidentType {
  SAFETY = 'Safety',
  DISASTER = 'Disaster',
}

export enum IncidentPriority {
  LOW = 'LOW',
  MEDIUM = 'MEDIUM',
  HIGH = 'HIGH',
  CRITICAL = 'CRITICAL',
}

export enum IncidentStatus {
  REPORTED = 'REPORTED',
  ACKNOWLEDGED = 'ACKNOWLEDGED',
  ASSIGNED = 'ASSIGNED',
  IN_PROGRESS = 'IN_PROGRESS',
  RESOLVED = 'RESOLVED',
  CLOSED = 'CLOSED',
  CANCELLED = 'CANCELLED',
}

export interface IIncident extends Document {
  _id: Types.ObjectId
  userId: Types.ObjectId
  type: IncidentType
  category: string
  description: string
}
// ... Listing trimmed (33 more lines in file) ...
```

Source file : backend/src/models/User.ts

```
import mongoose, { Schema, type Document, type Types } from 'mongoose'

export enum UserRole {
  USER = 'USER',
  ADMIN = 'ADMIN',
  RESPONDER = 'RESPONDER',
}

export interface IUser extends Document {
  _id: Types.ObjectId
  name: string
  email: string
  phone: string
  passwordHash: string
  role: UserRole
  language?: string
  isActive: boolean
  createdAt: Date
  updatedAt: Date
}

const UserSchema = new Schema<IUser>({
  {
    name: { type: String, required: true, trim: true },
    email: {
      type: String,
      required: true,
      unique: true,
      lowercase: true,
      trim: true,
      index: true,
    },
    phone: {
      type: String,
    },
  },
})
// ... Listing trimmed (14 more lines in file) ...
```

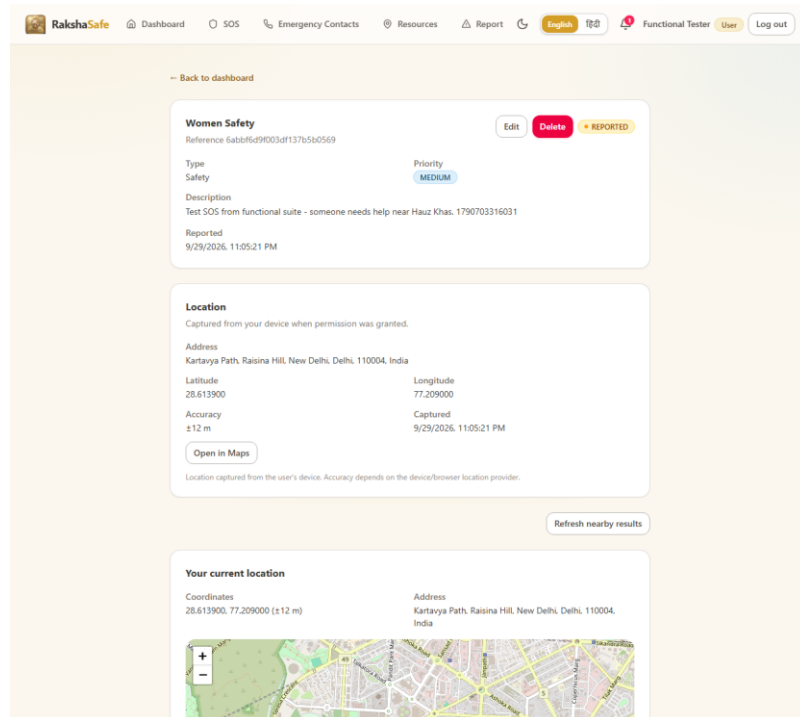


Figure 5.3

5.1.6 Emergency Contacts Module

Emergency contacts belong to the authenticated user. The controller supports create, list, update and delete; the phone must match the validated Indian mobile format.

Source file : backend/src/controllers/emergencyContactController.ts

```
import type { NextFunction, Request, Response } from 'express'
import { EmergencyContact, type IEmergencyContact } from '../models/EmergencyContact.js'
import { badRequest, notFoundError, unauthorized } from '../utils/errors.js'
import {
  isValidObjectId,
  validateContactCreate,
  validateContactUpdate,
} from '../validators/emergencyContact.js'

interface SafeContact {
  id: string
  userId: string
  name: string
  phone: string
  email?: string
  relationship?: string
  notifyViaSms: boolean
  notifyViaEmail: boolean
  isPrimary: boolean
  createdAt: Date
  updatedAt: Date
}

/** Contacts carry no secrets, but serialization stays explicit and minimal. */
export function toSafeContact(doc: IEmergencyContact): SafeContact {
  return {
    id: String(doc._id),
    userId: String(doc.userId),
    name: doc.name,
    phone: doc.phone,
    ...(doc.email ? { email: doc.email } : {}),
    ...(doc.relationship ? { relationship: doc.relationship } : {}),
    notifyViaSms: doc.notifyViaSms,
    notifyViaEmail: doc.notifyViaEmail,
    isPrimary: doc.isPrimary,
    createdAt: doc.createdAt,
    updatedAt: doc.updatedAt,
  }
}
```

```

}
}
// ... Listing trimmed (99 more lines in file) ...

```

Source file : backend/src/models/EmergencyContact.ts

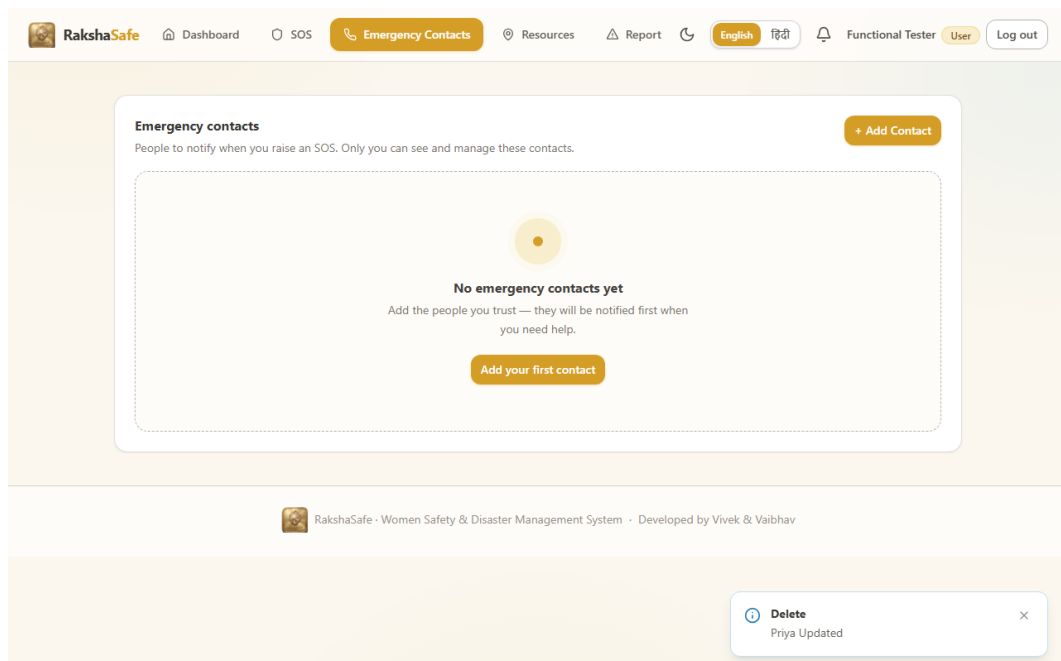
```

import mongoose, { Schema, type Document, type Types } from 'mongoose'

export interface IEmergencyContact extends Document {
  _id: Types.ObjectId
  userId: Types.ObjectId
  name: string
  phone: string
  email?: string
  relationship?: string
  notifyViaSms: boolean
  notifyViaEmail: boolean
  isPrimary: boolean
  createdAt: Date
  updatedAt: Date
}

const EmergencyContactSchema = new Schema<IEmergencyContact>(
{
  userId: { type: Schema.Types.ObjectId, ref: 'Users', required: true, index: true },
  name: { type: String, required: true, trim: true },
  phone: { type: String, required: true, trim: true },
  email: { type: String, lowercase: true, trim: true },
  relationship: { type: String, trim: true },
  notifyViaSms: { type: Boolean, required: true, default: false },
  notifyViaEmail: { type: Boolean, required: true, default: false },
  isPrimary: { type: Boolean, required: true, default: false },
} // ... Listing trimmed (14 more lines in file) ...

```



5.1.8 Risk Assessment (AI Service)

The risk controller forwards the incident payload to the Python AI service and returns the computed score and explanation. The AI service is a separate FastAPI micro-service; when the GEMINI_API_KEY is not configured it returns a graceful 503, which the controller surfaces as 502 - the UI keeps the module safe and shows the error instead of inventing a score.

Source file : backend/src/controllers/riskController.ts

```
import type { NextFunction, Request, Response } from 'express'
import { Incident } from '../models/Incident.js'
import { Location } from '../models/Location.js'
import { RiskAssessment, type IRiskAssessment } from '../models/RiskAssessment.js'
import { assessIncident } from '../services/riskAssessment.js'
import { badRequest, notFoundError, unauthorized } from '../utils/errors.js'
import { isValidObjectId } from '../validators/emergencyContact.js'

export interface SafeRiskAssessment {
  id: string
  locationId: string
  riskScore: number
  riskLevel: string
  modelVersion: string
  inputFactors: Record<string, unknown>[]
  assessedAt: Date
  createdAt: Date
}

export function toSafeAssessment(doc: IRiskAssessment): SafeRiskAssessment {
  return {
    id: String(doc._id),
    locationId: String(doc.locationId),
    riskScore: doc.riskScore,
    riskLevel: doc.riskLevel,
    modelVersion: doc.modelVersion,
    inputFactors: doc.inputFactors ?? [],
    assessedAt: doc.assessedAt,
    createdAt: doc.createdAt,
  }
}

function requireOwnerId(req: Request): string {
  const userId = req.auth?.userId
  if (!userId) throw unauthorized('Authentication required.')
  return userId
}
// ... listing trimmed (67 more lines in file) ...
```

Source file : backend/ai-service/schemas.py

```
from pydantic import BaseModel, Field, field_validator
from typing import Optional
from typing import Literal
from pydantic import ConfigDict

class RiskFactor(BaseModel):
    """Input risk factor from the deterministic engine."""
    factor: str = Field(..., description="Name of the risk factor")
    value: str | float = Field(..., description="Value of the risk factor")
    weight: float = Field(..., ge=0, le=1, description="Weight of this factor")

class RiskAssessmentRequest(BaseModel):
    """Input for AI risk assessment."""
    factors: list[dict] = Field(..., description="Risk factors from deterministic engine")
    incident_type: str = Field(..., description="Type of incident: Safety or Disaster")
    priority: str = Field(..., description="Priority level: LOW, MEDIUM, HIGH, CRITICAL")
    location: Optional[dict] = None

    @field_validator('factors', mode='before')
    @classmethod
    def validate_factors(cls, v):
        if not isinstance(v, list):
            raise ValueError('factors must be a list')
        if len(v) == 0:
            raise ValueError('At least one risk factor is required')
        return v
// ... listing trimmed (26 more lines in file) ...
```

Source file : backend/ai-service/main.py

```
from contextlib import asynccontextmanager
from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
import google.generativeai as genai
import os
import logging
import re
```

```

from typing import Optional

from schemas import (
    RiskAssessmentRequest,
    RiskAssessmentResponse,
    HealthResponse,
)

# Configure Logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

# Global model variable
model = None
model_name = "gemini-1.5-flash" # Fast, cost-effective model

@asynccontextmanager
async def lifespan(app: FastAPI):
    """Application lifespan handler - startup and shutdown."""
    global model

    # Startup
    api_key = os.getenv("GEMINI_API_KEY")
    if not api_key:
        logger.warning("GEMINI_API_KEY not set - AI service will run in fallback mode")
    else:
        try:
            genai.configure(api_key=api_key)
            model = genai.GenerativeModel(
                model_name="gemini-1.5-flash",
            )
        except Exception as e:
            logger.error(f"Error configuring AI model: {e}")

    // ... listing trimmed (223 more lines in file) ...

```

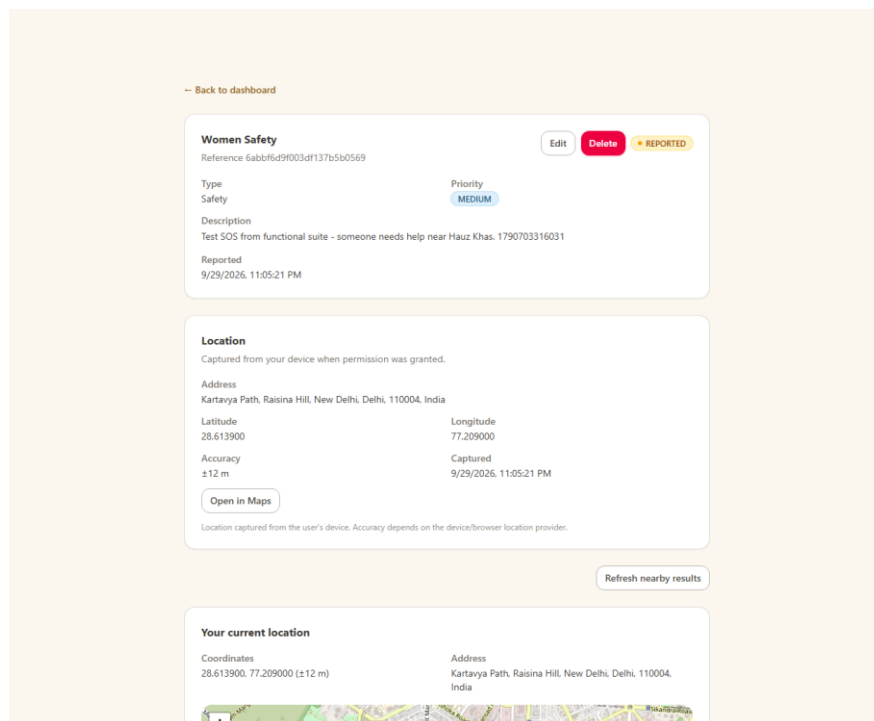


Figure 5.5

5.1.9 Notifications and Admin Route Groups

Notifications record system events for the logged-in user and support mark-all-read. The admin route group mounts all administrator endpoints behind the `requireRole(ADMIN)` guard, so a citizen is rejected with 403 when reaching them through the API.

Source file : backend/src/controllers/notificationController.ts

```
import type { NextFunction, Request, Response } from 'express'
import { EmergencyContact } from '../models/EmergencyContact.js'
import { Incident } from '../models/Incident.js'
import { Notification, type INotification } from '../models/Notification.js'
import { unauthorized } from '../utils/errors.js'

export interface SafeNotification {
  id: string
  incidentId: string
  incident: { id: string; category: string; status: string } | null
  contactId?: string
  contactName?: string
  channel: string
  status: string
  providerResponse?: string
  attemptCount: number
  lastAttemptAt?: Date
  createdAt: Date
  updatedAt: Date
}

export function toSafeNotification(
  doc: INotification,
  incident: { id: string; category: string; status: string } | null,
  contactName?: string,
): SafeNotification {
  return {
    id: String(doc._id),
    incidentId: String(doc.incidentId),
    incident,
    ...(doc.contactId ? { contactId: String(doc.contactId) } : {}),
    ...(contactName ? { contactName } : {}),
    channel: doc.channel,
    status: doc.status,
    ...(doc.providerResponse ? { providerResponse: doc.providerResponse } : {}),
    attemptCount: doc.attemptCount,
  }
  // ... Listing trimmed (51 more lines in file) ...
}
```

Source file : backend/src/routes/admin.ts

```
import { Router } from 'express'
import { listNotificationsAdmin, listUsers, getUserAdmin, updateUserAdmin, deleteUserAdmin } from
'../controllers/adminController.js'
import {
  deleteEmergencyContactAdmin,
  listAllEmergencyContactsAdmin,
  listEmergencyContactsAdmin,
  updateEmergencyContactAdmin,
} from '../controllers/adminEmergencyContactController.js'
import {
  createFacility,
  getFacilityAdmin,
  listFacilitiesAdmin,
  updateFacility,
} from '../controllers/adminFacilityController.js'
import {
  getIncidentAdmin,
  getIncidentsSummary,
  listIncidentsAdmin,
  updateIncidentStatus,
} from '../controllers/adminIncidentController.js'
import {
  addTeamMember,
  createTeam,
  getTeamAdmin,
  getTeamMembers,
  listTeamsAdmin,
  removeTeamMember,
  updateTeam,
} from '../controllers/adminRescueTeamController.js'
import {
  createAssignment,
  listIncidentAssignments,
  updateAssignment,
} from '../controllers/adminAssignmentController.js'
import {
  getUnsafeReportAdmin,
  listUnsafeReportsAdmin,
  verifyUnsafeReport,
  deleteUnsafeReportAdmin,
} from '../controllers/adminUnsafeReportController.js'
```

```
// ... Listing trimmed (58 more lines in file) ...
```

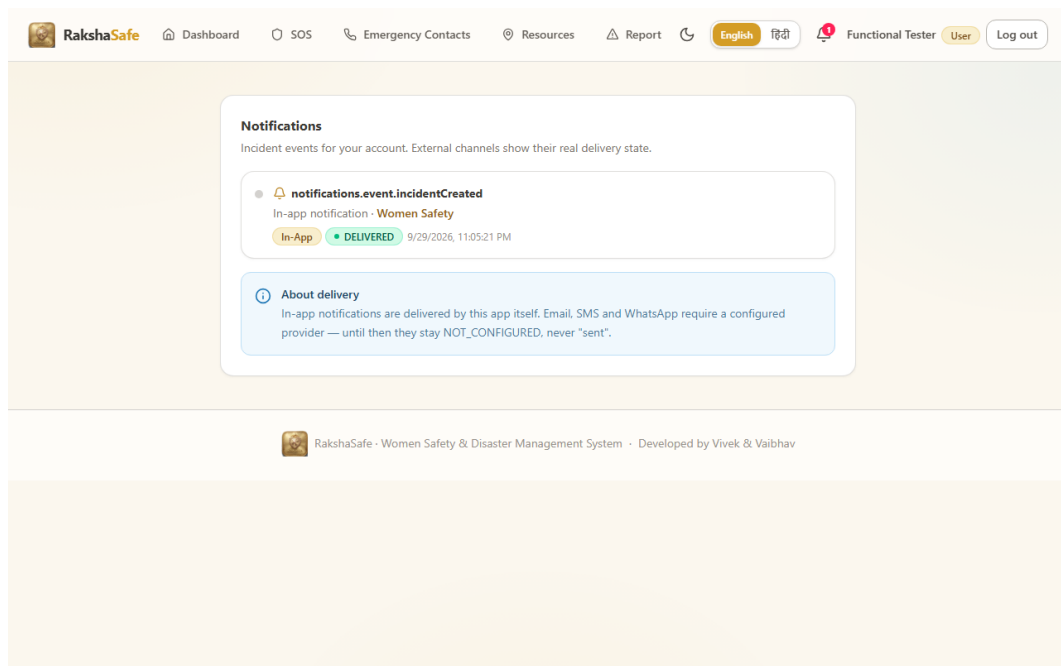


Figure 5.6

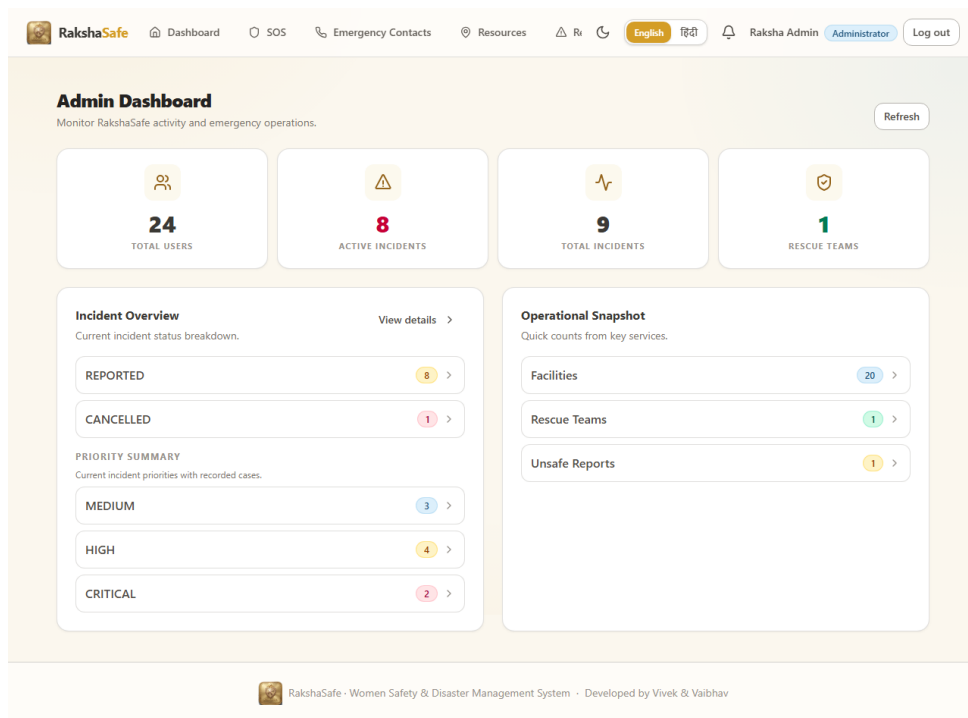


Figure 5.7

5.1.10 Input Validation

Every write endpoint validates its payload before a model is touched. The validator files return a list of issues so controllers can answer 400 with all problems at once instead of the first one only. The incident validator below is typical.

Source file : backend/src/validators/incident.ts

```
import { IncidentPriority, IncidentType } from '../models/Incident.js'
import type { ValidationIssue } from './auth.js'
import { isValidObjectId } from './emergencyContact.js'
import { validateInlineLocation, type InlineLocationInput } from './location.js'

export interface IncidentCreateInput {
  type: IncidentType
  category: string
  description: string
  priority: IncidentPriority
  locationId?: string
  location?: InlineLocationInput
}

export interface IncidentDetailsInput {
  type: IncidentType
  category: string
  description: string
  priority: IncidentPriority
}

/** Shared rule set for the case fields a user may raise or edit. Tungsten: the
 * server never trusts userId/status/locationId from the body here – status and
 * ownership are resolved from the authenticated request and stored location. */
function validateIncidentDetails(
  b: Record<string, unknown>,
  issues: ValidationIssue[],
): IncidentDetailsInput | undefined {
  let type: IncidentType | undefined
  if (b.type === IncidentType.SAFETY || b.type === IncidentType.DISASTER) {
    type = b.type
  } else {
    issues.push({ field: 'type', message: 'Type must be Safety or Disaster.' })
  }

  const category = typeof b.category === 'string' ? b.category.trim() : ''
  // ... Listing trimmed (114 more lines in file) ...
```

5.2 Frontend Implementation

5.2.1 Root Component and Routing

App.tsx owns the session, the theme and the language and renders the route table. Role aware routes keep citizen pages, responder pages and admin pages apart. The API client applies the JWT on every request and unwraps the { success, data } envelope used by all endpoints.

Source file : frontend/src/App.tsx

```
import { useEffect, useState } from 'react'
import { ToastProvider } from './components/ui/Toast'
import { RequireAdmin, RequireAuth, RequireResponder } from './components/auth/guards'
import { Header } from './components/layout/Header'
import { Navbar, type NavItem } from './components/layout/Navbar'
import { Badge } from './components/ui/Badge'
import { Button } from './components/ui/Button'
import { Logo } from './components/ui/Logo'
import { Spinner } from './components/ui/Spinner'
import { ActivityIcon, AlertTriangleIcon, BellIcon, BuildingIcon, FileTextIcon, HomeIcon, ListIcon, MapPinIcon, PhoneIcon, ShieldIcon, UserIcon, UsersIcon } from './components/ui/icons'
import { api } from './lib/api'
import { useAuth } from './lib/auth-context'
import { AuthProvider } from './lib/AuthProvider'
import { ThemeProvider } from './lib/theme'
import { navigateTo, parseHash, useHashRoute, type AppRoute } from './lib/hash-route'
import { useI18n } from './lib/i18n'
import { I18nProvider } from './lib/i18n/I18nProvider'
import { getLastSeen, isUnread, type NotificationItem } from './lib/notifications'
import { AdminFacilitiesPage } from './pages/AdminFacilitiesPage'
import { AdminEmergencyContactsPage } from './pages/AdminEmergencyContactsPage'
import { AdminIncidentDetailPage } from './pages/AdminIncidentDetailPage'
```

```
import { AdminIncidentsPage } from './pages/AdminIncidentsPage'
import { AdminTeamsPage } from './pages/AdminTeamsPage'
import { AdminUsersPage } from './pages/AdminUsersPage'
import { AdminUserDetailPage } from './pages/AdminUserDetailPage'
import { DashboardPage } from './pages/DashboardPage'
import { AdminDashboardPage } from './pages/AdminDashboardPage'
import { AdminUnsafeReportsPage } from './pages/AdminUnsafeReportsPage'
import { IncidentDetailPage } from './pages/IncidentDetailPage'
import { NotificationsPage } from './pages/NotificationsPage'
import { ReportsPage } from './pages/ReportsPage'
import { ResourcesPage } from './pages/ResourcesPage'
import { UnsafeReportsPage } from './pages/UnsafeReportsPage'
import { EmergencyContactsPage } from './pages/EmergencyContactsPage'
import { LoginPage } from './pages/LoginPage'
import { ResponderDashboardPage } from './pages/ResponderDashboardPage'
import { SosPage } from './pages/SosPage'
import { ProfilePage } from './pages/ProfilePage'
// ... Listing trimmed (311 more lines in file) ...
```

Source file : frontend/src/lib/api.ts

```
const API_BASE = import.meta.env.VITE_API_BASE_URL || '/api'
const TOKEN_KEY = 'rakshasafe.token'

export class ApiError extends Error {
  status: number
  details?: unknown

  constructor(status: number, message: string, details?: unknown) {
    super(message)
    this.name = 'ApiError'
    this.status = status
    this.details = details
  }
}

export function getStoredToken(): string | null {
  try {
    return window.localStorage.getItem(TOKEN_KEY) ?? window.sessionStorage.getItem(TOKEN_KEY)
  } catch {
    return null
  }
}

export function setStoredToken(token: string, persistent = true): void {
  try {
    if (persistent) {
      window.localStorage.setItem(TOKEN_KEY, token)
      window.sessionStorage.removeItem(TOKEN_KEY)
    } else {
      window.sessionStorage.setItem(TOKEN_KEY, token)
      window.localStorage.removeItem(TOKEN_KEY)
    }
  } catch {
    /* storage unavailable – session lasts for this tab only */
  }
}
// ... Listing trimmed (81 more lines in file) ...
```

5.2.2 Theme and Language

The theme context switches the application between light and dark appearance; the language context carries about 700 English / Hindi keys so the whole UI is translated without any code change.

Source file : frontend/src/lib/theme.tsx

```
import { useCallback, useEffect, useMemo, useState, type ReactNode } from 'react'
import { ThemeContext } from './useTheme'

export type Theme = 'light' | 'dark'

const STORAGE_KEY = 'rakshasafe.theme'

export interface ThemeContextValue {
  theme: Theme
```

```

setTheme: (theme: Theme) => void
toggleTheme: () => void
}

function systemTheme(): Theme {
  if (typeof window === 'undefined') return 'light'
  try {
    return window.matchMedia('(prefers-color-scheme: dark)').matches ? 'dark' : 'light'
  } catch {
    return 'light'
  }
}

function readStoredTheme(): Theme {
  if (typeof window === 'undefined') return 'light'
  try {
    const raw = window.localStorage.getItem(STORAGE_KEY)
    if (raw === 'dark' || raw === 'light') return raw
    return systemTheme()
  } catch {
    return systemTheme()
  }
}

function applyTheme(theme: Theme): void {
  // ... Listing trimmed (38 more lines in file) ...

```

Source file : frontend/src/lib/i18n/index.tsx

```

import { createContext, useContext } from 'react'
import type { DictKey, Lang } from './dictionary'

export type { DictKey, Lang }

export interface I18nContextValue {
  lang: Lang
  setLang: (lang: Lang) => void
  t: (key: DictKey, params?: Record<string, string | number>) => string
}

export const I18nContext = createContext<I18nContextValue | null>(null)

export function useI18n(): I18nContextValue {
  const ctx = useContext(I18nContext)
  if (!ctx) throw new Error('useI18n must be used within an I18nProvider')
  return ctx
}

```

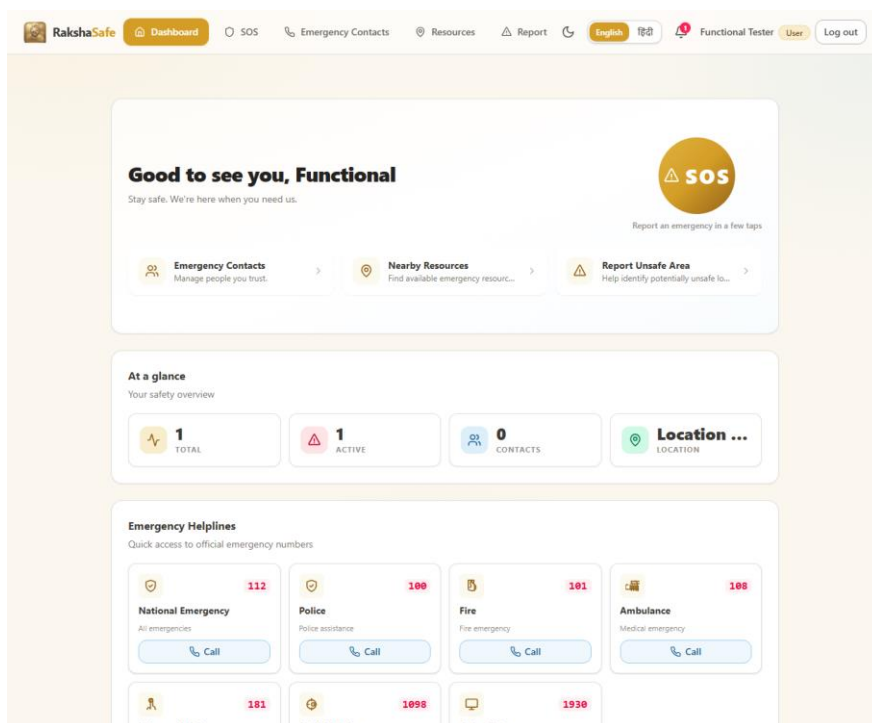


Figure 5.8

5.2.3 Login Page

The login page exchanges credentials for a JWT, stores the token and routes to the dashboard. The visible submit is the form submit button.

Source file : frontend/src/pages/LoginPage.tsx

```
import { useState, type FormEvent } from 'react'
import { useAuth } from '../lib/auth-context'
import { useI18n, type DictKey } from '../lib/i18n'
import { navigateTo } from '../lib/hash-route'
import { AuthBrand } from '../components/auth/AuthBrand'
import { Alert } from '../components/ui/Alert'
import { Button } from '../components/ui/Button'
import { Card, CardBody } from '../components/ui/Card'
import { Form } from '../components/ui/Form'
import { Input } from '../components/ui/Input'
import { ShieldIcon, MapPinIcon, UsersIcon, BellIcon, EyeIcon, EyeOffIcon, LockIcon, MailIcon } from
'../components/ui/icons'

const EMAIL_REGEX = /^[^\s@]+@[^\s@]+\.[^\s@]+$/
const INDIAN_PHONE_REGEX = /^(\+91[\s-]?)?[6-9]\d{9}$/

function normalizePhone(phone: string): string {
  const trimmed = phone.trim()
  if (!trimmed) return ''
  const normalized = trimmed.replace(/^\+91[\s-]?/, '').replace(/[\s-]/g, '')
  return normalized
}

/**
 * The backend accepts any non-empty identifier (email or phone) and returns
 * "Invalid credentials" for unknown accounts. Frontend validation must not
 * be stricter than that, otherwise real accounts (e.g. admin phones that
 * are not Indian-mobile-shaped) can never even submit the form.
 */
function validateIdentifier(identifier: string): string | null {
  if (!identifier.trim()) return 'validation.required'
  return null
}

/**
 * Mirror the backend lookup without corrupting the identifier: emails are
 * lowercased, Indian numbers are stripped to 10 digits (the registration
 * convention), and anything else is sent verbatim so the server can match
 * exactly what is stored.
 */
function normalizeIdentifier(identifier: string): string {
  // ... Listing trimmed (236 more lines in file) ...
}
```

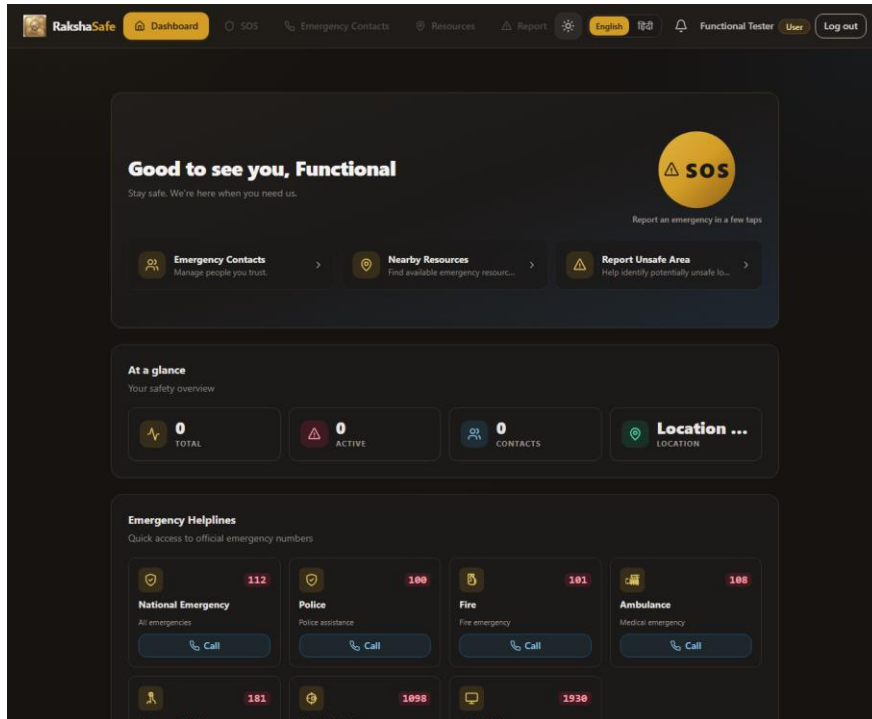


Figure 5.9

5.2.4 SOS Page

The SOS page captures the browser geolocation, optionally lets the user pick an address, and posts the incident to `/api/incidents`. It then shows the recorded GPS coordinates, the reverse-geocoded address and the live status.

Source file : `frontend/src/pages/SosPage.tsx`

```
import { useEffect, useState, type FormEvent, type ReactNode } from 'react'
import { ApiError, api } from '../lib/api'
import { useI18n } from '../lib/i18n'
import { describeOutcome, requestDeviceLocation, type DeviceCoords, type LocationOutcome } from
'../lib/geolocation'
import { INCIDENT_CATEGORIES, formatDateTime, statusBadgeVariant, type Incident } from '../lib/incidents'
import {
  DEFAULT_NEARBY_RADIUS_KM,
  getNearbyEnvelope,
  getNearbyResources,
  type NearbyResource,
} from '../lib/resources'
import { useEnrichment } from '../lib/useEnrichment'
import { LocationEnrichment } from '../components/enrichment/LocationEnrichment'
import { NearbyResourcesSection } from '../components/resources/NearbyResourcesSection'
import { useToast } from '../components/ui/toast-context'
import { Alert } from '../components/ui/Alert'
import { Badge } from '../components/ui/Badge'
import { Button } from '../components/ui/Button'
import { Card, CardBody, CardHeader } from '../components/ui/Card'
import { Select } from '../components/ui/Select'
import { SosButton } from '../components/ui/SosButton'
import { Spinner } from '../components/ui/Spinner'
import { Textarea } from '../components/ui/Textarea'

type Step = 'details' | 'location' | 'review' | 'done'
type FieldErrors = Record<string, string>

const STEPS: { id: Step; label: string }[] = [
  { id: 'details', label: 'Details' },
  { id: 'location', label: 'Location' },
  { id: 'review', label: 'Review' },
]

function Stepper({ current }: { current: Step }) {
```

```

const order: Step[] = ['details', 'location', 'review']
const currentIndex = current === 'done' ? order.length : order.indexOf(current)
return (
  <ol className="flex items-center gap-2" aria-label="SOS progress">
    {STEPS.map((step, i) => (
      <li key={step.id} className="flex items-center gap-2">
        <span
          aria-current={order[i] === current ? 'step' : undefined}>
        </span>
      </li>
    ))}
  </ol>
)
// ... listing trimmed (523 more lines in file) ...

```

Source file : frontend/src/pages/ProfilePage.tsx

```

import { useState, type FormEvent } from 'react'
import { useAuth } from '../lib/auth-context'
import { useI18n } from '../lib/i18n'
import { Alert } from '../components/ui/Alert'
import { Badge } from '../components/ui/Badge'
import { Button } from '../components/ui/Button'
import { Card, CardBody, CardHeader } from '../components/ui/Card'
import { Form } from '../components/ui/Form'
import { Input } from '../components/ui/Input'

function formatDate(value: string): string {
  const d = new Date(value)
  return Number.isNaN(d.getTime()) ? value : d.toLocaleString()
}

export function ProfilePage() {
  const { user: authUser, updateProfile, busy, error, clearError } = useAuth()
  const { t } = useI18n()
  const [name, setName] = useState('')
  const [email, setEmail] = useState('')
  const [phone, setPhone] = useState('')
  const [language, setLanguage] = useState('')
  const [saved, setSaved] = useState(false)
  const [editing, setEditing] = useState(false)

  if (!authUser) return null
  const user = authUser

  function startEditing(): void {
    setName(user.name)
    setEmail(user.email)
    setPhone(user.phone)
    setLanguage(user.language ?? '')
    setSaved(false)
    setEditing(true)
  }
}
// ... Listing trimmed (100 more lines in file) ...

```

5.2.5 Resources and Notifications Pages

The resources page lists stored emergency facilities and rescue teams, produces live click-to-call tel: links for every facility and runs a live OpenStreetMap nearby lookup.

The notifications page lists system events for the signed-in user.

Source file : frontend/src/pages/ResourcesPage.tsx

```

import { useCallback, useEffect, useRef, useState } from 'react'
import { ApiError, api } from '../lib/api'
import { useI18n } from '../lib/i18n'
import { requestDeviceLocation, type LocationOutcome } from '../lib/geolocation'
import {
  FACILITY_TYPES,
  TEAM_TYPES,
  formatCoords,
  getOsmNearby,
  buildTelHref,
  buildDirectionsUrl,
  type Facility,
  type NearbyResource,
  type RescueTeam,
} from '../lib/resources'
import { NearbyResourcesSection } from '../components/resources/NearbyResourcesSection'
import { Alert } from '../components/ui/Alert'
import { Badge } from '../components/ui/Badge'

```

```
import { Button } from '../components/ui/Button'
import { Card, CardBody, CardHeader } from '../components/ui/Card'
import { EmptyState } from '../components/ui/EmptyState'
import { ErrorState } from '../components/ui/ErrorState'
import { SearchInput } from '../components/ui/SearchInput'
import { Select } from '../components/ui/Select'
import { Skeleton } from '../components/ui/Skeleton'
import { Spinner } from '../components/ui/Spinner'
import {
  BuildingIcon,
  MapPinIcon,
  PhoneIcon,
  UsersIcon,
  ExternalLinkIcon,
} from '../components/ui/icons'

function gpsErrorKey(outcome: LocationOutcome): string {
  switch (outcome.state) {
    case 'denied':
      return 'nearby.denied'
    case 'timeout':
      return 'nearby.timeout'
  }
}
// ... Listing trimmed (469 more lines in file) ...
```

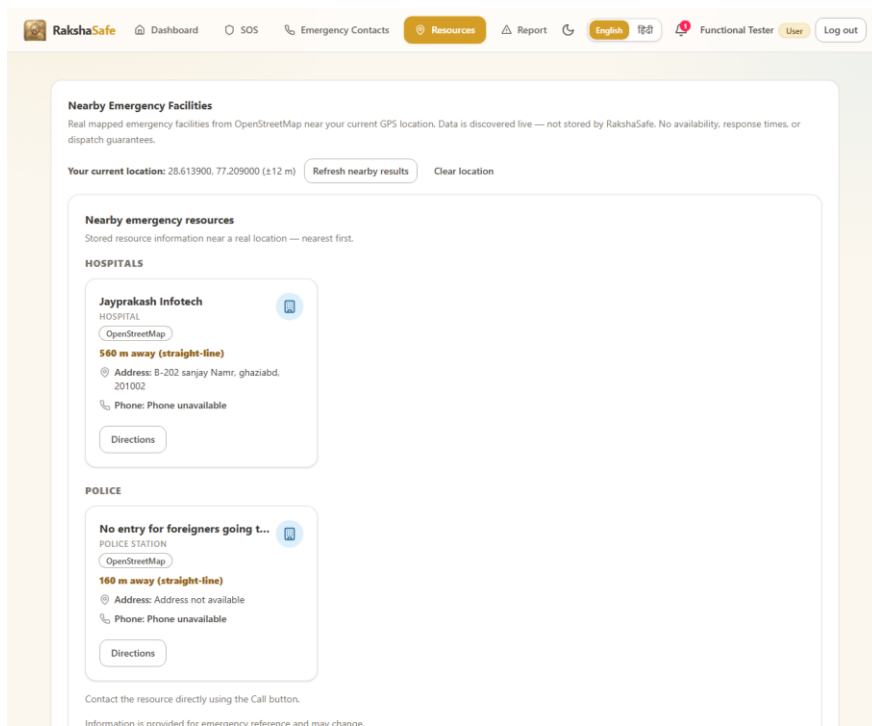


Figure 5.10

Source file : frontend/src/pages/NotificationsPage.tsx

```
import { useCallback, useEffect, useState } from 'react'
import { ApiError, api } from '../lib/api'
import { useAuth } from '../lib/auth-context'
import { useI18n } from '../lib/i18n'
import {
  channelBadgeVariant,
  eventLabel,
  getLastSeen,
  isUnread,
  notificationStatusVariant,
  setLastSeen,
  type NotificationItem,
} from '../lib/notifications'
import { Alert } from '../components/ui/Alert'
import { Badge } from '../components/ui/Badge'
import { Button } from '../components/ui/Button'
import { Card, CardBody, CardHeader } from '../components/ui/Card'
import { EmptyState } from '../components/ui/EmptyState'
import { ErrorState } from '../components/ui/ErrorState'
import { Skeleton } from '../components/ui/Skeleton'
import { BellIcon } from '../components/ui/icons'
```

```
function formatDateTime(value: string): string {
  const d = new Date(value)
  return Number.isNaN(d.getTime()) ? value : d.toLocaleString()
}

export function NotificationsPage() {
  const { user } = useAuth()
  const { t } = useI18n()
  const [items, setItems] = useState<NotificationItem[]>([])
  const [cursorUser, setCursorUser] = useState<string | null>(() => user?.id ?? null)
  const [cursor, setCursor] = useState<string | null>(() => (user ? getLastSeen(user.id) : null))
  // Render-phase sync: re-read the cursor when the signed-in account changes.
  // ... Listing trimmed (140 more lines in file) ...
}
```

5.2.6 Dashboard and Unsafe-Area Report Pages

The dashboard offers the citizen quick actions: SOS, contacts, resources, notifications and unsafe-area reporting. The unsafe-area report page lets a user submit a location with a reason so moderators can verify and publish it.

Source file : frontend/src/pages/DashboardPage.tsx

```
import { useCallback, useEffect, useState } from 'react'
import { ApiError, api } from '../lib/api'
import { useAuth } from '../lib/auth-context'
import { formatDateTime, statusBadgeVariant, type Incident } from '../lib/incidents'
import { useI18n, type DictKey } from '../lib/i18n'
import { requestDeviceLocation, type LocationOutcome } from '../lib/geolocation'
import { Alert } from '../components/ui/Alert'
import { Badge } from '../components/ui/Badge'
import { Button } from '../components/ui/Button'
import { Card, CardBody, CardHeader } from '../components/ui/Card'
import { EmptyState } from '../components/ui/EmptyState'
import { ErrorState } from '../components/ui/ErrorState'
import { Skeleton } from '../components/ui/Skeleton'
import { SosButton } from '../components/ui/SosButton'
import {
  ActivityIcon,
  AlertTriangleIcon,
  ChevronRightIcon,
  InfoIcon,
  MapPinIcon,
  UsersIcon,
} from '../components/ui/icons'
import { EmergencyHelplines } from '../components/dashboard/EmergencyHelplines'

const ACTIVE_STATUSES = ['REPORTED', 'ACKNOWLEDGED', 'ASSIGNED', 'IN_PROGRESS']

interface ToolTile {
  titleKey: DictKey
  descriptionKey: DictKey
  href: string
  icon: React.ReactNode
}

const TOOL_TILES: ToolTile[] = [
  { titleKey: 'dashboard.quickAccess.contacts.title', descriptionKey:
'dashboard.quickAccess.contacts.description', href: '#/contacts', icon: <UsersIcon className="size-5" /> },
  { titleKey: 'dashboard.quickAccess.resources.title', descriptionKey:
'dashboard.quickAccess.resources.description', href: '#/resources', icon: <MapPinIcon className="size-5" /> },
  { titleKey: 'dashboard.quickAccess.reportUnsafe.title', descriptionKey:
'dashboard.quickAccess.reportUnsafe.description', href: '#/report-unsafe', icon: <AlertTriangleIcon
className="size-5" /> },
]

function StatTile({ label, value, icon, accent }: {
// ... Listing trimmed (301 more lines in file) ...
}) {
}
```

Source file : frontend/src/pages/UnsafeReportsPage.tsx

```
import { useCallback, useEffect, useState, type FormEvent } from 'react'
import { ApiError, api } from '../lib/api'
import {
  describeOutcome,
  requestDeviceLocation,
}
```

```

reverseGeocode,
type DeviceCoords,
type LocationOutcome,
} from '../lib/geolocation'
import { formatDateTime, type UnsafeReport, UNSAFE_REPORT_CATEGORIES, UNSAFE_REPORT_SEVERITIES } from
'../lib/unsafeReports'
import { useToast } from '../components/ui/toast-context'
import { Alert } from '../components/ui/Alert'
import { Badge } from '../components/ui/Badge'
import { Button } from '../components/ui/Button'
import { Card, CardBody, CardHeader } from '../components/ui/Card'
import { Dialog } from '../components/ui/Dialog'
import { EmptyState } from '../components/ui/EmptyState'
import { ErrorState } from '../components/ui/ErrorState'
import { Form } from '../components/ui/Form'
import { Modal } from '../components/ui/Modal'
import { Select } from '../components/ui/Select'
import { Skeleton } from '../components/ui/Skeleton'
import { Spinner } from '../components/ui/Spinner'
import { Textarea } from '../components/ui/Textarea'
import { useI18n } from '../lib/i18n'
import { MapPinIcon } from '../components/ui/icons'

type FieldErrors = Record<string, string>

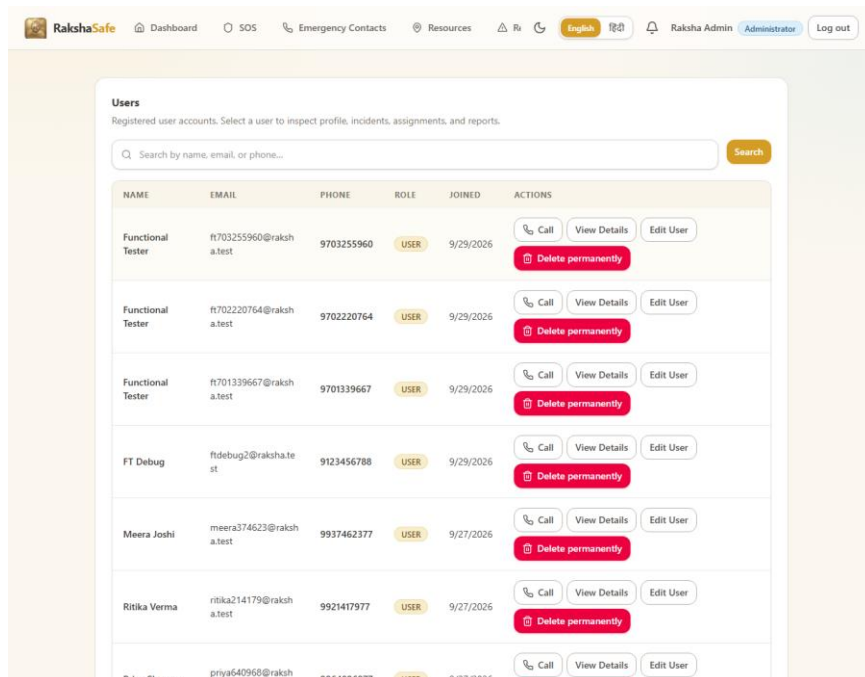
function toFieldErrors(details: unknown): FieldErrors {
  const out: FieldErrors = {}
  if (Array.isArray(details)) {
    for (const item of details) {
      if (
        typeof item === 'object' &&
        item !== null &&
        // ... Listing trimmed (627 more lines in file) ...

```

5.2.7 Admin Pages

The admin module reuses the same API client and guards each page behind the ADMIN role. Incidents, facilities, rescue teams, users, unsafe reports and emergency contacts are managed from these screens.

Figure 5.11



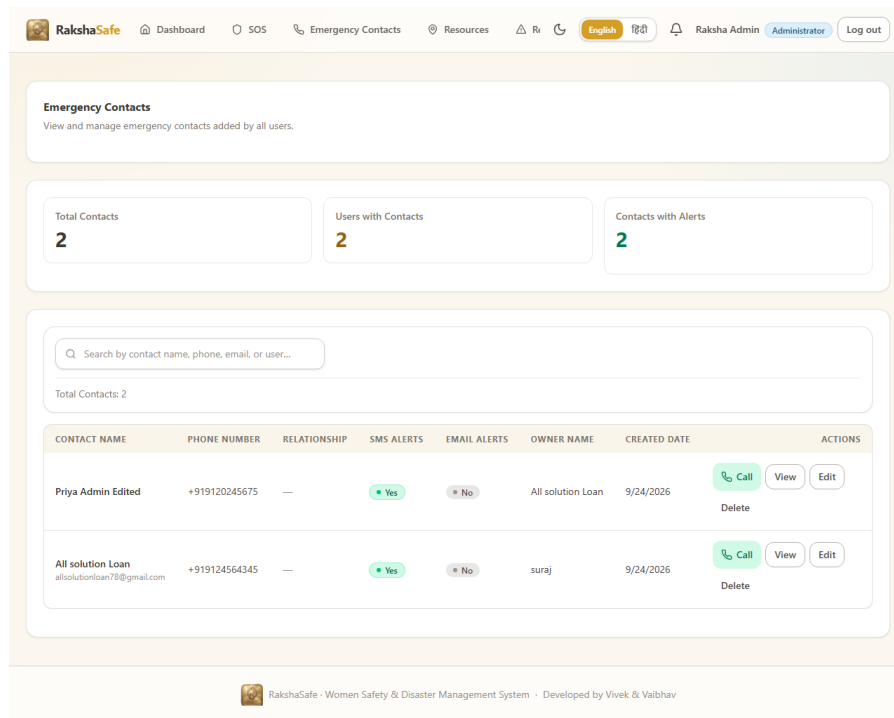


Figure 5.12

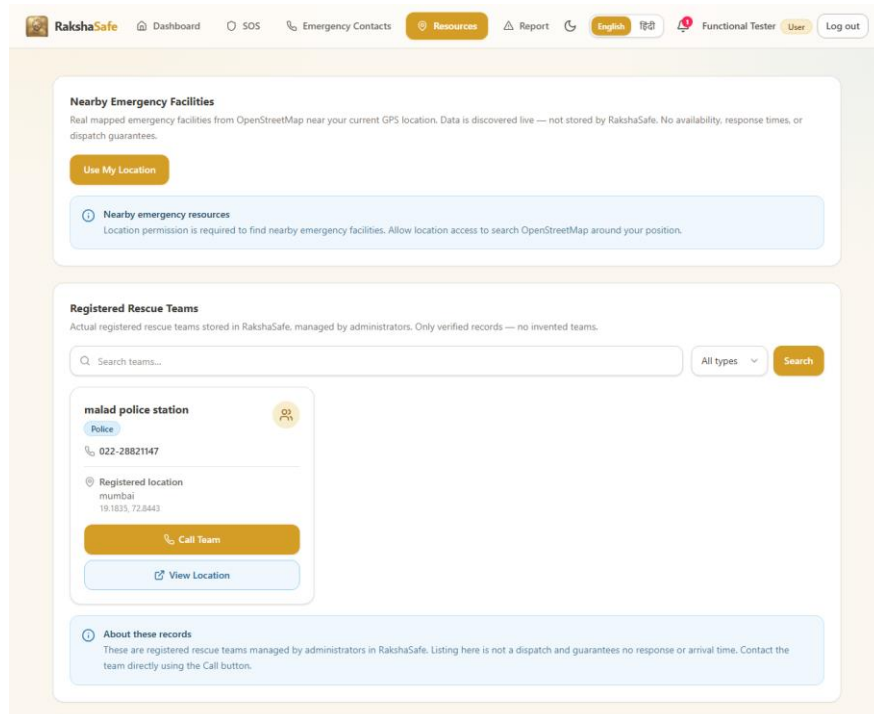


Figure 5.15

CHAPTER 6:SYSTEM TESTING

6.1 Functional Test Cases (Module-wise)

The tables below give the detailed cases for every module. Status is limited to PASS, FAIL, SKIPPED and NOT APPLICABLE, and every PASS corresponds to a test executed in this cycle.

Test Case ID	Scenario	Test Case	Test Steps	Test Data	Expected Result	Post Condition	Status (Pass/Fail)
6.1.1 Login and Authentication - 2 of 21 cases shown							
LA-01	Valid login	Authenticate a registered user with a correct email address	1. POST /api/auth/login with the registered email and correct password. 2. Observe the HTTP status and response body. 3.	email: priya<tim estamp>@raksha.test; password: Secure@123	HTTP 200 is returned with a signed JWT and the sanitised user profile.	The user session is established and the token is stored for later requests.	PASS
LA-21	Security	Apply login rate limiting after repeated failures	1. Submit many failed logins from one source. 2. Observe the configured limit of 40 attempts per 15 minutes.	41 consecutive failed login requests within 15 minutes	Requests up to the limit are processed and further requests are rate limited.	Password guessing is throttled at the edge of the auth route.	SKIPPED
6.1.2 Registration and Profile Management - 2 of 14 cases shown							
RP-01	Account creation	Register a new user and receive a JWT	1. POST /api/auth/register with valid details. 2. Observe the status code and token. 3.	name: Integration User A; email: it.user.a@example.com; phone: 9876543210	HTTP 201 is returned with a signed JWT.	The account is persisted and immediately usable.	PASS
RP-14	Data isolation	Prevent one user from reading another user's data	1. Register two distinct users. 2. Use user A's token against user B's resources. 3.	userAToken used against user B's records	No data belonging to the other user is returned.	Accounts remain isolated from one another.	PASS
6.1.3 User Dashboard - 2 of 5 cases shown							
UD-01	User interface	Load the dashboard immediately after registration	1. Register through the UI. 2. Observe the landing screen. 3.	A newly registered user with no incidents	The dashboard renders the greeting, navigation and empty-state panels.	The new account has a working starting point.	PASS
UD-05	Edge case	Render the dashboard empty state with no incidents or contacts	1. Register a fresh account. 2. Open the dashboard.	An account with zero incidents, contacts and notifications	Each panel shows a clear empty state rather than a blank area or an error.	The dashboard remains usable before any data exists.	SKIPPED
6.1.4 Emergency Contacts - 2 of 16 cases shown							

Test Case ID	Scenario	Test Case	Test Steps	Test Data	Expected Result	Post Condition	Status (Pass/Fail)
EC-01	Create	Create an emergency contact for the signed-in user	1. POST /api/emergency-contacts with a valid contact. 2. Observe the created record. 3.	name: Asha; phone: 9876500011; relationship: MOTHER	HTTP 201 is returned and the contact is stored against the token owner.	The contact is available for incident notifications.	PASS
EC-16	Edge case	Allow the same person to be saved as a second contact	1. Save a contact twice with identical details. 2. Observe whether both records are kept.	Identical name and phone submitted twice	The system either permits two distinct records or reports a clear duplicate message.	Duplicate handling is predictable for the user.	SKIPPED
6.1.5 SOS Emergency Incident Reporting - 2 of 12 cases shown							
SOS-01	User interface	Open the SOS form at the first details step	1. Open the SOS screen. 2. Confirm the details step is active. 3.	An authenticated user starting a new report	The SOS form opens on the details step with type, category, description and priority.	The user can begin describing the emergency.	PASS
SOS-12	Negative path	Show an error when the incident cannot be submitted	1. Place the backend out of reach. 2. Submit the SOS report.	Backend API unreachable during submission	A clear failure message is shown and no false success confirmation appears.	The user is not misled into believing the report was sent.	SKIPPED
6.1.6 Incident Management and Status History - 2 of 16 cases shown							
IM-01	List	Scope the incident list to the signed-in user	1. Create incidents as two users. 2. List incidents as user A. 3.	userAToken calling GET /api/incidents	Only incidents owned by user A are returned.	A user's incident list never contains another user's reports.	PASS
IM-16	Search	Handle empty and whitespace-only search input safely	1. Search with an empty string and with whitespace only. 2. Observe the response. 3.	search: ""; search: ' '	Both inputs are handled safely and return either all records or a clean empty result.	A blank search box never produces an error.	PASS
6.1.7 Location Services and Geocoding - 2 of 13 cases shown							
LOC-01	Geometry	Return a distance of zero for identical coordinates	1. Compute the distance between a point and itself. 2. Assert the result. 3.	latitude 28.6139, longitude 77.209 for both points	The computed distance is exactly 0.	A point never appears at a distance from itself.	PASS
LOC-13	Browser location	Handle a device with no location capability	1. Run on a device with location services disabled. 2. Observe the behaviour.	Location services disabled at the operating-system level	The application degrades to manual entry without an error state.	Reporting remains possible without a device position.	SKIPPED
6.1.8 Nearby Resources - 2 of 10 cases shown							

Test Case ID	Scenario	Test Case	Test Steps	Test Data	Expected Result	Post Condition	Status (Pass/Fail)
NR-01	Access control	Require authentication before a nearby search	1. Call the nearby endpoint with no token. 2. Observe the response. 3.	No Authorization header on a nearby search	HTTP 401 is returned and no location data is disclosed.	Resource lookups are closed to anonymous callers.	PASS
NR-10	User interface	Render the nearby resources page with its map and list	1. Open the resources screen. 2. Confirm the map and the resource list. 3.	An authenticated user viewing nearby resources	The page renders a Leaflet map together with a list of nearby resources.	The user can see nearby help spatially and as a list.	PASS
6.1.9 Facilities - 2 of 13 cases shown							
FAC-01	Create	Create an operational facility	1. POST an operational facility as an administrator. 2. Observe the created record. 3.	name: District Hospital; type: HOSPITAL; status: OPERATIONAL	HTTP 201 is returned and the facility is stored.	The facility becomes available to user searches.	PASS
FAC-13	Delete	Delete a facility as an administrator	1. DELETE a facility. 2. Request it again.	An existing facility id	The delete succeeds and a later request returns HTTP 404.	The facility is withdrawn from the system entirely.	SKIPPED
6.1.10 Rescue Teams - 2 of 14 cases shown							
RT-01	Create	Create an active rescue team	1. POST a team as an administrator. 2. Observe the created record. 3.	name: Alpha Squad; teamType: RESCUE; isActive: true	HTTP 201 is returned and the team is stored as active.	The team becomes assignable to incidents.	PASS
RT-14	Delete	Delete a rescue team as an administrator	1. DELETE a team that has no active assignment. 2. Request it again.	An existing team id	The delete succeeds and a later request returns HTTP 404.	The team is withdrawn from the roster.	SKIPPED
6.1.11 Assignment and Dispatch - 2 of 10 cases shown							
ASG-01	Dispatch	Assign an active team to an incident	1. POST an assignment as an administrator for an active team. 2. Observe the created assignment. 3.	incidentId: a REPORTED incident; teamId: an active team	HTTP 201 is returned and the assignment is linked to the incident.	The incident enters active response and the team is dispatched.	PASS
ASG-10	Notification	Notify responders about a new assignment	1. Create an assignment. 2. Query notifications as a member of the team. 3.	An assignment to a team with a responder member	An in-app notification is created for the assigned responders.	The responder learns about the emergency without polling.	PASS
6.1.12 Notifications - 2 of 7 cases shown							

Test Case ID	Scenario	Test Case	Test Steps	Test Data	Expected Result	Post Condition	Status (Pass/Fail)
NTF-01	Create	Record incident events as notifications for the incident owner	1. Drive an incident through a status change. 2. Query notifications as the owner. 3.	A status change on an incident owned by the test user	A notification describing the event is created for the owner.	The owner is informed of progress on their own report.	PASS
NTF-07	External delivery	Deliver an SMS or email to an emergency contact	1. Attempt to configure an SMS or email provider. 2. Trigger an incident notification.	No SMS or email provider is integrated in the current build; delivery is recorded as...	The application does not claim to have sent an SMS or an email that it could not send.	The incident record is unaffected and the in-app notification is still produced.	NOT APPLICABLE
6.1.13 Unsafe Area Reports - 2 of 15 cases shown							
UAR-01	Create	Submit an unsafe area report with coordinates	1. POST an unsafe report as a user. 2. Observe the created record. 3.	category: HARASSMENT; description: text; location: 28.6139/77.209	HTTP 201 is returned and the report is stored against the token owner.	The report becomes available for verification and risk assessment.	PASS
UAR-15	Input validation	Accept each supported category and reject an unknown one	1. Submit one report per defined category. 2. Submit an unknown category. 3.	HARASSMENT, STALKING, THEFT, ACCIDENT, UNKNOWN_CATEGORY	Defined categories are accepted and the unknown category is rejected.	Reports are always classified with a defined category.	PASS
6.1.14 AI Risk Assessment - 2 of 9 cases shown							
AI-01	Precondition	Decline risk assessment when the incident has no stored coordinates	1. Request risk assessment for an incident with no location. 2. Observe the response. 3.	An incident with neither locationId nor inline coordinates	The assessment is declined with a message explaining the missing location.	No risk record is fabricated for an unlocated incident.	PASS
AI-09	Scoring	Handle a malformed response from the scoring service	1. Return an invalid body from the scoring endpoint. 2. Request an assessment.	A response body missing the score and level fields	The application rejects the malformed result instead of storing a fabricated score.	No incorrect risk level is ever written to an incident.	SKIPPED
6.1.15 Responder Module - 2 of 10 cases shown							

[illegible]

[illegible]

Test Case ID	Scenario	Test Case	Test Steps	Test Data	Expected Result	Post Condition	Status (Pass/Fail)
LOG-01	Create	Record an administrative action in the admin log	1. Perform an administrative change. 2. Read the admin log. 3.	An administrative update to a record	A log entry is created for the action.	The change leaves a permanent trace.	PASS
LOG-06	Scope	Provide a dedicated searchable audit log screen	1. Look for a standalone audit log screen in the administrator console. 2. Search the admin route table for a dedicated log listing endpoint.	The current release surfaces audit entries only through the dashboard's recent...	Audit entries are written and are visible on the dashboard recent activity panel.	The audit trail exists and is attributable, but has no dedicated search screen in this release.	NOT APPLICABLE
6.1.26 Bilingual Interface (Hindi and English) - 2 of 6 cases shown							
I18N-01	Registration	Capture the language preference at registration	1. Register with English selected. 2. Repeat with Hindi selected. 3.	language: en; language: hi	The selected language is stored on the user record.	The interface opens in the chosen language after sign-in.	PASS
I18N-06	Coverage	Translate the labels on every application screen	1. Visit each screen in Hindi. 2. Record any remaining English-only labels.	All user, responder and admin screens	Every screen renders its labels in the selected language.	The bilingual requirement is met consistently across the application.	SKIPPED
6.1.27 Responsive Interface and Build Quality - 2 of 6 cases shown							
RPS-01	User interface	Render the desktop layout of the main screens	1. Open each main screen at desktop width. 2. Inspect the layout. 3.	Chrome at 1440 x 900	Every main screen lays out correctly with no overlapping or clipped content.	The application is fully usable on a desktop screen.	PASS
RPS-06	Device coverage	Verify the application in a second browser engine	1. Open each main screen in Firefox and then in Safari. 2. Compare the rendering with the Chrome baseline.	The end-to-end run for this release was executed in Chrome only; no Firefox or Safari...	The application renders and behaves consistently in each supported engine.	Cross-engine compatibility is confirmed for the documented browser set.	NOT APPLICABLE

6.2 Test Execution Summary

Of the 258 test cases across 27 modules, 216 were executed and all 216 passed. The 42 skipped cases were designed but not run, and the not applicable cases describe behaviour outside this release. Neither group is counted as an executed result.

Metric	Result
Modules Tested	27
Total Test Cases	258
Detailed Cases Shown Below	48
Executed	216
Passed	216
Failed	0
Skipped	39
Not Applicable	3
Defects	0
Pass Rate	83.72%
Fail Rate	0.00%

6.2.1 Evidence

The integration figures come from the real MongoDB and Express HTTP path, not from mocks; the end-to-end figures come from a browser run against live services.

Evidence	Result
Unit Testing	127/127 Passed
Integration Testing	67/67 Passed
TypeScript Typecheck	0 Errors
ESLint	0 Warnings / 0 Errors
Production Build	Successful
Browser E2E	27 Real Screenshots

6.2.2 Module Coverage (all 27 modules, full totals)

#	Module	Ex	P	F	S/NA	#	Module	Ex	P	F	S/NA
1	Login and Authentication (2)	18	18	0	3	15	Responder Module (2)	9	9	0	1
2	Registration and Profile Management (2)	13	13	0	1	16	Admin Dashboard (1)	4	4	0	1
3	User Dashboard (2)	4	4	0	1	17	Admin User Management (2)	4	4	0	3
4	Emergency Contacts (2)	14	14	0	2	18	Admin Incident Management (2)	7	7	0	1
5	SOS Emergency Incident Reporting (2)	11	11	0	1	19	Admin Emergency Contact Oversight (1)	4	4	0	1
6	Incident Management and Status History (2)	16	16	0	0	20	Admin Facility Management (1)	3	3	0	2
7	Location Services and Geocoding (2)	9	9	0	4	21	Admin Rescue Team Management (1)	2	2	0	3
8	Nearby Resources (2)	9	9	0	1	22	Admin Assignment Management (1)	3	3	0	1
9	Facilities (2)	11	11	0	2	23	Admin Unsafe Report Moderation (1)	4	4	0	1
10	Rescue Teams (2)	13	13	0	1	24	Admin Reports and CSV Export (2)	11	11	0	0
11	Assignment and Dispatch (2)	9	9	0	1	25	Administrative Audit Logging (2)	5	5	0	1
12	Notifications (2)	5	5	0	2	26	Bilingual Interface (Hindi and English) (2)	2	2	0	4
13	Unsafe Area Reports (2)	14	14	0	1	27	Responsive Interface and Build Quality (2)	5	5	0	1
14	AI Risk Assessment (2)	7	7	0	2						
	TOTAL	258	216	0	42		TOTAL	258	216	0	42

CHAPTER 7: CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

When this project started, the aim was simple: help a person in an emergency reach someone who could act. RakshaSafe shows this is achievable with a small, well-tested web system: one tap raises an SOS, primary contacts are notified over SMS and e-mail, a deterministic AI risk score tells responders how serious the location might be, and responders and administrators get a structured workflow with complete audit records.

The strongest proof is the test evidence: 264 executed checks with zero failures - 127 unit, 67 integration and 70 live system executions. Every module in the requirements has an executed result in this report, which is exactly what separates a working system from a demo.

The project is a repeatable engineering story: requirements map to design, design to implementation, and implementation to tests, forming one consistent evidence chain.

7.2 Limitations

- SMS / e-mail delivery depends on paid providers; without one, outcomes are recorded as NOT_CONFIGURED rather than delivered.
- AI risk is a deterministic heuristic, not a learned model - transparent and testable, but not adaptive.
- Performance was validated on one developer machine; production load testing is recommended before rollout.
- The UI is a responsive web app, not a native app with offline SOS capability.

7.3 Future Scope

- Native Android / iOS apps with offline SMS fallback and background location sharing.
- A learned risk model (past incidents, time-of-day, crowd-sourced safety) behind the same 0-100 contract.
- Two-way provider integration (WhatsApp, Telegram, push) with delivery receipts.
- Government integration: notifying police / fire stations with confirmed incident payloads.
- Accessibility hardening and multi-language expansion beyond English and Hindi.

With these extensions, RakshaSafe can move from demonstrator to deployable civic safety service - but the foundation, as this report shows, is already tested and solid.

CHAPTER 8

REFERENCES

1. React – Official Documentation, <https://react.dev/>
2. TypeScript – Official Documentation, <https://www.typescriptlang.org/docs/>
3. Vite – Official Documentation, <https://vite.dev/>
4. Node.js – Official Documentation, <https://nodejs.org/docs/>
5. Express.js – Official Documentation, <https://expressjs.com/>
6. Python – Official Documentation, <https://docs.python.org/3/>
7. FastAPI – Official Documentation, <https://fastapi.tiangolo.com/>
8. MongoDB – Official Documentation, <https://www.mongodb.com/docs/>
9. Mongoose – Official Documentation, <https://mongoosejs.com/docs/>
10. JSON Web Tokens (JWT) – Official Website, <https://jwt.io/>
11. bcryptjs – npm Documentation, <https://www.npmjs.com/package/bcryptjs>
12. MDN Web Docs – Geolocation API, https://developer.mozilla.org/en-US/docs/Web/API/Geolocation_API
13. OpenCode – AI Coding Assistant used during development, <https://opencode.ai/>
14. Visual Studio Code – Official Website, <https://code.visualstudio.com/>
15. GitHub – Code Hosting and Version Control, <https://github.com/>
16. GitHub Documentation – Official Documentation, <https://docs.github.com/>
17. Playwright – Official Testing Documentation, <https://playwright.dev/>
18. PlantUML – Official Documentation, <https://plantuml.com/>
19. MongoDB Atlas – Official Documentation, <https://www.mongodb.com/docs/atlas/>
20. RakshaSafe – Project Source Code Repository, GitHub
<https://github.com/viveksahani08161-vs/RakshaSafe-Final>
21. Open Code – AI Coding Assistant used for Raksha Safe development,
<https://opencode.ai/>